# Forging RISC-V Architectures

## Table of Contents


**1. Introduction: AI-Assisted Generation of RISC-V Processor Architectures ..... 1**
1.1 Introduction: AI Agents for Processor Design ..... 1
1.2 Two Hardware Design Approaches: Chisel and Verilog ..... 1
Chisel: Hardware Generation Through Scala ..... 1
Verilog: Direct RTL Generation ..... 1
1.3 Progressive Construction of the RISC-V Instruction Set ..... 1
Stage 1: Foundations of RV32I ..... 1
Stage 2: Complete RV32I Implementation ..... 1
Stage 3: RV32I with Multiplication ..... 2
Stage 4: Complete RV32IM with Sequential Division ..... 2
1.4 Evolution Toward Pipelined Processor Architectures ..... 2
Stage 5: Two-Stage RV32IM Pipeline ..... 2
Stage 6: Five-Stage RV32IM Pipeline ..... 2
Stage 7: Five-Stage Pipeline with Cache Memories ..... 2
1.5 Fully Automated Generation, Testing, and Waveform Analysis ..... 3
1.6 Conclusion ..... 3

**2. The RISC-V Ecosystem and Its Importance for Digital Systems Development ..... 4**
2.1 Introduction: An Open Instruction Set Architecture ..... 4
2.2 A Modular and Extensible Architecture ..... 4
2.3 A Rich Hardware Development Ecosystem ..... 4
2.4 Software Tools, Simulation, and Verification ..... 5
2.5 Importance for Digital Systems Development ..... 5
Technological Independence ..... 5
Reduced Development Barriers ..... 5
Application-Specific Processor Design ..... 5
Hardware/Software Co-Design ..... 5
2.6 RISC-V as a Platform for Education, Research, and AI-Assisted Design ..... 5
2.7 Conclusion ..... 6

**3. The RISC-V: Digital System and RISC-V Processor Design: Chisel versus Verilog ..... 7**
Principles, Design Methodologies, Advantages, and Limitations ..... 7
3.1 Introduction ..... 7
3.2 Verilog: Direct Description of Digital Hardware ..... 7
Advantages of Verilog ..... 7
Drawbacks of Verilog ..... 7
3.3 Chisel: Hardware Construction Using Scala ..... 7
Advantages of Chisel ..... 8
Drawbacks of Chisel ..... 8
3.4 Comparison for RISC-V Processor Development ..... 8
Instruction Decoder and Datapath ..... 8
Multiplication and Sequential Division ..... 8
Pipeline Development ..... 8
Cache Memory Integration ..... 9
3.5 Synthesis, Performance, and Verification ..... 9
3.6 Comparative Summary ..... 9
3.7 AI-Assisted RISC-V Development: A Complementary Approach ..... 9
3.8 Conclusion ..... 10
Automated RISC-V Architecture Generation ..... 11
Automatic Testing and Functional Verification ..... 11
Waveform Generation and Visual Analysis with GTKWave ..... 11
Conclusion ..... 11

**GLab 0. Installing Chisel 3 on Ubuntu Linux ..... 13**
Development Tools, Installation Procedure, Simulation, and Waveform Visualization ..... 13
0.1 Introduction ..... 13
2. Installing the Basic Development Tools ..... 13
3. Installing Scala and sbt ..... 14
4. Installing the RTL Simulation and Visualization Tools ..... 14
5. Creating a Chisel 3 Project ..... 14
Configure the Scala project ..... 15
6. Creating a First Chisel Module ..... 15
7. Creating Automated Unit Tests ..... 16
8. Generating VCD Waveforms ..... 16
9. Recommended Workflow for RISC-V Processor Development ..... 17
10. Summary ..... 18
Official documentation ..... 18

**GLab 1: Building simple RV32I (R-type only) RISC-V architecture ..... 18**
Test class: ..... 23
Understanding the Chisel ALU Test File (ALUSpec.scala) ..... 24
Purpose of the test file ..... 24
Testbench organization ..... 24
Definition of the ALU operations ..... 24

First test: Basic functional verification....25
Second test: Boundary-value verification....25
Third test: Randomized verification....25
Waveform generation....25
Summary of the verification strategy....26
Complete project: rv32i-stage1.zip....26
Understanding the Files Generated by a Chisel-Based RISC-V Processor Project....27
Main File Types....27
Chisel Hardware Description Files....27
ALU.scala....27
RegisterFile.scala....28
Decoder.scala....28
ImmediateGenerator.scala....28
RV32IRCore.scala....28
RV32ITestWrapper.scala....28
CoreParams.scala....28
RV32IEncoding.scala....28
Trace.scala....28
Emit.scala....28
RefModel.scala....28
Generated SystemVerilog Files....29
RV32IRCore.sv....29
RV32ITestWrapper.sv....29
Why generate SystemVerilog?....29
Automated Test Files....29
Supporting test utilities....29
Running the tests....30
Simulation Waveform Files....30
Files within each waveform directory....30
Opening the waveforms....31
Project Configuration and Documentation....31
Overall Organization of the Project....31
Conclusion....32

**GLab 2: Building simple RV32I (R and I types) RISC-V architecture....34**

**GLab 3: Building full RV32I RISC-V architecture....42**

**GLab 4: Building full RV32IM (MUL only) architecture....53**

**GLab 5: Building full RV32IM architecture with sequential DIV/REM unit....59**

**GLab 6: Building full RV32IM architecture with 2-stage pipeline....67**

**GLab 7: Building full RV32IM architecture with 5-stage pipeline....75**

**GLab 8: 5-stage pipeline with I-cache and D-cache....83**

# Forging RISC-V Architectures

## 1. Introduction: AI-Assisted Generation of RISC-V Processor Architectures

### 1.1 Introduction: AI Agents for Processor Design

The emergence of AI coding agents, such as **Claude Code**, introduces a new approach to digital hardware development. Instead of manually writing every hardware module, testbench, and simulation script, designers can describe a processor architecture through structured specifications and progressively delegate its implementation, verification, and refinement to an AI agent.
Our approach applies this methodology to the development of **RISC-V processor architectures**, starting with a minimal implementation of the RV32I instruction set and progressively extending it toward more sophisticated RV32IM processors featuring pipelining, sequential arithmetic units, and cache memories.
The objective is not simply to generate hardware description language (HDL) code. It is to establish an **automated design and verification workflow** in which each architectural evolution produces a complete, executable, and tested processor implementation.
At every development stage, the AI agent generates the hardware modules, simulation environment, test programs, and verification infrastructure. The resulting implementations are automatically compiled, simulated, and checked against expected architectural behavior. Timing diagrams are generated in VCD format and can be examined using **GTKWave**.
This methodology makes it possible to explore several processor architectures rapidly while preserving functional correctness across successive design iterations.

### 1.2 Two Hardware Design Approaches: Chisel and Verilog

Our development methodology supports two complementary hardware description approaches.

#### Chisel: Hardware Generation Through Scala

**Chisel** is a hardware construction language embedded in Scala. It allows designers to describe digital circuits using parameterized modules, strongly typed signals, and reusable hardware abstractions.
Chisel is particularly suitable for AI-assisted processor development because it supports hierarchical design, parameterization, and systematic reuse of components.
A RISC-V processor can be decomposed into modules such as the instruction decoder, register file, immediate generator, arithmetic logic unit (ALU), multiplier, divider, control and status registers (CSRs), and memory interfaces.
The AI agent generates these Chisel modules, integrates them into a processor core, and uses the Chisel compilation infrastructure to produce synthesizable Verilog or SystemVerilog.
Chisel also provides a convenient environment for automated unit testing and architectural verification.

#### Verilog: Direct RTL Generation

The alternative approach uses **Verilog or SystemVerilog** directly as the hardware description language.
In this workflow, the AI agent generates RTL modules, testbenches, and simulation scripts without requiring an intermediate Scala/Chisel representation.
This approach provides direct control over registers, combinational logic, finite-state machines, and clock-cycle behavior. It is particularly useful for studying microarchitectural details, understanding pipeline timing, and comparing alternative hardware implementations.
Both approaches ultimately produce RTL suitable for simulation and synthesis.
Our methodology uses **Chisel and Verilog Forge** as complementary automated generation environments. The same architectural specifications can guide the creation of implementations in either language, allowing comparisons of design complexity, simulation behavior, and hardware organization.

### 1.3 Progressive Construction of the RISC-V Instruction Set

Rather than requesting a complete processor implementation in a single step, our methodology follows a sequence of increasingly complex architectural milestones.

#### Stage 1: Foundations of RV32I

Development begins with the fundamental RV32I instruction formats, particularly **R-type and I-type instructions**.
The initial processor includes a program counter, instruction decoder, register file, immediate generator, and ALU.
The first supported instructions typically include arithmetic and logical operations such as ADD, SUB, AND, OR, XOR, and immediate arithmetic operations such as ADDI.
This initial implementation establishes the basic instruction execution path:
**Instruction Fetch → Decode → Register Read → Execute → Write-Back**
The AI agent generates the corresponding hardware modules and their unit tests. Each instruction is verified using directed test vectors and automatically generated test cases.

#### Stage 2: Complete RV32I Implementation

The architecture is then extended to support the complete RV32I base integer instruction set.
This requires the introduction of additional instruction formats:

- **S-type:** Store instructions.

- **B-type:** Conditional branches.
- **U-type:** Upper-immediate instructions.
- **J-type:** Unconditional jumps.

The processor gains load/store operations, conditional branches, jumps, comparison instructions, and the remaining base integer operations.
The AI agent updates the instruction decoder, control logic, immediate generator, memory interfaces, and program-counter selection circuitry.
Additional verification programs exercise branches, jumps, memory accesses, and interactions between successive instructions.
Where required by the chosen processor specification, the implementation also incorporates machine-mode control and status registers, exception handling, and privileged instructions.

### Stage 3: RV32I with Multiplication

The next architectural extension introduces the multiplication instructions from the RISC-V M extension.
The processor initially supports multiplication using a combinational multiplier, allowing the multiplication result to be computed within one execution cycle.
The implementation can progressively support MUL, MULH, MULHSU, and MULHU.
A shared multiplier structure can be used to generate the required low-order or high-order product bits, taking signed and unsigned operand interpretations into account.
The AI agent generates the multiplier module and verifies its operation using boundary values, signed and unsigned operands, and randomized tests against a software reference model.

### Stage 4: Complete RV32IM with Sequential Division

The processor is subsequently extended to the complete RV32IM instruction set by introducing:
**DIV, DIVU, REM, and REMU.**
Unlike multiplication, division is implemented using a **sequential restoring division algorithm**.
The divider processes one quotient bit per iteration, reducing the hardware resources required compared with a fully combinational divider.
The implementation includes operand sign handling, quotient and remainder generation, divide-by-zero behavior, signed overflow handling, and a control finite-state machine.
Because division requires multiple clock cycles, the processor must suspend instruction completion while the divider is active.
The AI agent automatically generates the divider, its handshake and control logic, and tests covering arithmetic correctness, exceptional cases, and execution latency.

## 1.4 Evolution Toward Pipelined Processor Architectures

Once the single-cycle RV32IM processor is validated, the methodology progresses toward pipelined implementations.

### Stage 5: Two-Stage RV32IM Pipeline

The first pipelined implementation separates instruction processing into two stages:
**IF  Instruction Fetch**
**EX  Decode, Register Read, Execute, Memory Access, and Write-Back**
An IF/EX pipeline register separates the stages.
Instruction fetching proceeds sequentially using static not-taken branch prediction. Branches and jumps are resolved in EX, with incorrect-path instructions discarded when control flow changes.
Because register reads and writes occur in the same execution stage, this architecture avoids the conventional interstage register data hazards found in deeper pipelines.
The sequential divider introduces a structural stall: both pipeline stages are frozen until the division result becomes available.
The AI agent generates the pipeline registers, stall logic, branch redirection circuitry, and verification tests for instruction timing and precise architectural behavior.

### Stage 6: Five-Stage RV32IM Pipeline

The next development stage introduces the conventional five-stage RISC-V pipeline:
**IF → ID → EX → MEM → WB**
These stages perform instruction fetch, decoding and register access, execution, memory access, and register write-back.
This architecture introduces additional challenges, including read-after-write data hazards, load-use dependencies, control hazards, and multicycle execution stalls.
The AI agent must therefore generate forwarding networks, hazard detection logic, pipeline interlocks, branch flushing mechanisms, and divider stall control.
Automated verification becomes especially important because correctness depends not only on instruction results but also on interactions between simultaneously executing instructions.

### Stage 7: Five-Stage Pipeline with Cache Memories

The final architectural extension introduces **instruction and data caches**.
The instruction cache accelerates instruction fetching, while the data cache reduces the effective latency of load and store operations.
The implementation can progressively incorporate cache-line organization, tag comparison, valid bits, hit/miss detection, refill control, and write policies.
Cache misses introduce variable-latency memory accesses, requiring additional pipeline stalls and coordination between the processor and memory hierarchy.
The AI agent generates cache controllers, memory interfaces, and verification scenarios covering hits, misses, refills, and interactions with pipeline execution.

## 1.5 Fully Automated Generation, Testing, and Waveform Analysis

A central feature of our methodology is that **every architectural stage is automatically generated and verified**.
Using Claude Code, the development process can be organized as a sequence of automated tasks:

1. **Specification:** Define the ISA subset, microarchitecture, interfaces, timing constraints, and expected behavior.
2. **Generation:** Produce Chisel or Verilog RTL modules, integration code, testbenches, and build scripts.
3. **Compilation:** Compile the generated hardware and detect syntax, elaboration, and structural errors.
4. **Simulation:** Execute directed instruction tests, randomized arithmetic tests, and complete RISC-V programs.
5. **Verification:** Compare processor execution against reference models, including register values, memory operations, exceptions, and instruction retirement.
6. **Waveform generation:** Produce VCD traces containing clock, program counter, instructions, pipeline registers, control signals, arithmetic results, and memory transactions.
7. **Debugging and refinement:** Analyze simulation failures, correct the generated implementation, and repeat the tests until the required checks pass.

For Verilog and SystemVerilog implementations, simulation can be performed using **Icarus Verilog** or **Verilator**. Chisel implementations can be tested through their associated Scala testing infrastructure and RTL simulation backends.
Generated VCD files are examined with **GTKWave**, allowing detailed observation of instruction execution, pipeline stalls, forwarding operations, divider iterations, branch redirections, and cache behavior.
This creates a reproducible workflow in which each processor version is accompanied by its RTL sources, test infrastructure, simulation results, and waveforms.

## 1.6 Conclusion

AI agents provide a powerful mechanism for accelerating the design and exploration of RISC-V processor architectures.
Our methodology demonstrates a progressive development path from a minimal RV32I processor supporting basic R-type and I-type instructions to a complete RV32IM implementation with sequential division, followed by two-stage and five-stage pipelined processors and, ultimately, architectures incorporating cache memories.
The use of both **Chisel and Verilog Forge** makes it possible to investigate alternative hardware description methodologies while maintaining a common architectural specification.
Most importantly, the approach integrates automatic code generation with systematic verification. Each development stage produces not merely hardware source code but a complete, testable implementation accompanied by simulation results and GTKWave timing diagrams.
The resulting workflow establishes a foundation for **AI-assisted hardware design, architectural experimentation, and automated RISC-V processor verification**, substantially reducing repetitive development effort while enabling increasingly sophisticated microarchitectural exploration.

# 2. The RISC-V Ecosystem and Its Importance for Digital Systems Development

## 2.1 Introduction: An Open Instruction Set Architecture

**RISC-V** is an open-standard Instruction Set Architecture (ISA) that is transforming the development of processors, embedded systems, and digital computing platforms. Initially developed at the University of California, Berkeley, RISC-V provides a common architectural foundation for designing processors without requiring proprietary ISA licenses.
Unlike commercial processor architectures, RISC-V is based on an open specification maintained by RISC-V International, a global organization bringing together semiconductor manufacturers, research institutions, software developers, and technology companies.
An important distinction is that **RISC-V defines an instruction set, not a particular processor implementation**. Designers can develop processors with different microarchitectures, performance levels, power consumption, and hardware complexity while maintaining compatibility with the same ISA.
This flexibility makes RISC-V particularly attractive for digital systems development, where applications range from simple microcontrollers and embedded controllers to high-performance computing systems, artificial intelligence accelerators, and specialized industrial processors.
The RISC-V ecosystem encompasses much more than processor hardware. It includes architectural specifications, open-source processor cores, development tools, compilers, operating systems, simulation environments, verification frameworks, FPGA platforms, and commercial semiconductor products.
Together, these components establish an increasingly mature environment for designing, implementing, testing, and deploying digital systems.

## 2.2 A Modular and Extensible Architecture

One of the principal characteristics of RISC-V is its **modular instruction set architecture**.
Rather than defining a single, monolithic instruction set, RISC-V provides a small base architecture that can be extended with standardized instruction groups.
The RV32I base instruction set, for example, defines a 32-bit integer architecture with fundamental arithmetic, logical, memory access, and control-flow instructions.
Additional extensions introduce specialized capabilities:

| Extension | Function |
|---|---|
| `RV32I / RV64I` | Base integer instruction sets |
| `M` | Integer multiplication and division |
| `A` | Atomic memory operations |
| `F / D` | Single- and double-precision floating-point arithmetic |
| `C` | Compressed instructions for improved code density |
| `V` | Vector processing |
| `B-related extensions` | Bit manipulation operations |
| `Zicsr` | Control and status register instructions |
| `Zifencei` | Instruction-fetch synchronization |

This modular organization allows processor designers to select the functionality required by a particular application.
For example, a small embedded controller may implement RV32I or RV32IMC, while a more powerful processor may include floating-point arithmetic, virtual memory, vector instructions, and multicore capabilities.
RISC-V also supports custom instruction extensions, allowing designers to introduce application-specific operations without modifying the standardized base instruction set.
This capability is particularly important for digital systems incorporating cryptographic engines, digital signal processing, artificial intelligence, or specialized industrial control functions.
The resulting architecture can combine **software compatibility with hardware specialization**, providing a valuable balance between standardization and innovation.

## 2.3 A Rich Hardware Development Ecosystem

The RISC-V ecosystem includes numerous open-source and commercial processor implementations.
These range from compact microcontroller-oriented cores to sophisticated superscalar and out-of-order processors.
Representative open-source projects include Rocket Chip, BOOM, Ibex, and CVA6.
These implementations demonstrate the diversity of possible microarchitectures, including simple pipelines, multistage pipelines, cache hierarchies, memory management units, and advanced execution mechanisms.
The availability of open-source processor cores provides several important advantages.
First, designers can study complete processor implementations rather than relying exclusively on architectural documentation.
Second, existing hardware modules can be reused or adapted for new applications.
Third, alternative implementations can be compared in terms of performance, silicon area, power consumption, and design complexity.
Finally, open hardware facilitates independent verification and experimentation.
RISC-V processors can be described using traditional hardware description languages such as Verilog and SystemVerilog, or higher-level hardware construction languages such as Chisel.
These designs can be simulated, synthesized, and implemented on FPGA platforms before eventual integration into ASICs.
Consequently, RISC-V supports the complete digital design process, from architectural exploration to physical hardware realization.

## 2.4 Software Tools, Simulation, and Verification

A major strength of the RISC-V ecosystem is the availability of a broad software development infrastructure.
Established compiler technologies such as GCC and LLVM support RISC-V code generation, enabling programs written in C, C++, and other languages to execute on compatible processors.
Debugging tools such as GDB, together with embedded development environments, support software development and hardware/software integration.
Operating systems ranging from lightweight real-time kernels to Linux can execute on suitable RISC-V implementations.
The ecosystem also provides architectural simulators and emulators.
Spike is a RISC-V ISA simulator commonly used as an architectural reference. QEMU supports emulation of RISC-V systems and enables software development before physical hardware becomes available.
For RTL verification, tools such as Icarus Verilog, Verilator, and GTKWave provide compilation, simulation, and waveform visualization capabilities.
These tools enable designers to verify individual hardware modules, execute complete programs, and analyze processor behavior cycle by cycle.
Architectural compliance tests and differential testing against reference models are particularly important for ensuring that independently developed processors implement the instruction set correctly.
This verification infrastructure substantially reduces the barriers to developing new processor architectures.

## 2.5 Importance for Digital Systems Development

RISC-V has significant implications for the development of modern digital systems.

### Technological Independence

Because the ISA specification is open, organizations can develop compatible processors without depending on a single proprietary ISA owner.
This supports technological independence, particularly for universities, research laboratories, semiconductor companies, and public institutions seeking greater control over their computing platforms.
However, the open ISA does not automatically make every processor implementation open-source or free of third-party intellectual property restrictions. Individual implementations and associated technologies may have their own licensing conditions.

### Reduced Development Barriers

RISC-V enables developers to begin processor design using publicly available specifications, open-source tools, and reference implementations.
This is especially valuable for small companies, academic laboratories, and educational institutions that may lack access to expensive proprietary processor IP.
Although hardware implementation, verification, fabrication, and certification remain costly, the absence of ISA licensing fees can reduce an important barrier to entry.

### Application-Specific Processor Design

RISC-V is particularly well suited to specialized digital systems.
A designer can start with a minimal processor and progressively introduce additional functionality, including multiplication, division, floating-point arithmetic, vector processing, or custom accelerators.
This flexibility enables hardware architectures to be optimized for specific application requirements.
Examples include low-power sensor nodes, industrial controllers, robotics, communication systems, cybersecurity devices, and embedded artificial intelligence.

### Hardware/Software Co-Design

RISC-V also facilitates hardware/software co-design.
Because the ISA and many implementation tools are openly accessible, designers can evaluate the impact of architectural modifications on both hardware complexity and software execution.
Custom instructions can be developed alongside compiler support, simulation models, and application software.
This approach is particularly valuable for specialized computing systems in which software algorithms and hardware accelerators must be optimized together.

## 2.6 RISC-V as a Platform for Education, Research, and AI-Assisted Design

RISC-V is especially important in education and research because it provides a complete and accessible framework for studying computer architecture.
Students can begin with a simple RV32I processor implementing arithmetic and logical instructions, then progressively introduce memory operations, branches, multiplication, division, exceptions, and privileged operations.
The same processor can subsequently evolve from a single-cycle architecture to a two-stage or five-stage pipeline, incorporating forwarding logic, hazard detection, and cache memories.
Each architectural modification can be evaluated through simulation and FPGA implementation.
This progressive methodology makes it possible to connect theoretical computer architecture concepts with concrete digital hardware implementations.
RISC-V is also particularly compatible with emerging **AI-assisted hardware design methodologies**.
AI coding agents such as Claude Code can generate Chisel, Verilog, or SystemVerilog modules from architectural specifications, create testbenches, execute simulation tools, analyze failures, and iteratively refine implementations.
For example, an AI-assisted workflow can progressively generate:

1. A basic RV32I processor supporting R-type and I-type instructions.
2. A complete RV32I processor with memory and control-flow instructions.
3. An RV32IM processor with combinational multiplication and sequential division.
4. A two-stage pipelined RV32IM processor.
5. A five-stage processor with forwarding and hazard management.
6. A pipelined processor incorporating instruction and data caches.

At every stage, the generated hardware can be verified using automated test programs and reference models, with VCD waveforms visualized through GTKWave.
The openness and modularity of RISC-V make it particularly suitable for this approach because architectural specifications, software tools, and many hardware reference implementations are publicly available.
Nevertheless, AI-generated hardware must undergo rigorous functional verification, timing analysis, and implementation validation. Successful compilation and simulation alone do not guarantee that a processor is correct or suitable for fabrication.

## 2.7 Conclusion

The RISC-V ecosystem represents an important development in digital computing because it combines an open-standard instruction set with modular architectural extensions, a growing collection of processor implementations, and a comprehensive software and verification infrastructure.
Its principal strengths are **openness, flexibility, extensibility, interoperability, and accessibility**.
These characteristics enable organizations to develop processors adapted to their requirements while benefiting from a shared technological foundation.
For digital systems development, RISC-V supports the entire design process, from simple embedded processors to advanced pipelined architectures and specialized computing accelerators.
It also provides an excellent platform for education, architectural research, hardware/software co-design, and automated processor generation.
Combined with modern AI coding agents, Chisel, Verilog, simulation tools, and automated verification frameworks, RISC-V opens new possibilities for rapidly exploring processor architectures and developing increasingly sophisticated digital systems.
**RISC-V is therefore more than an alternative processor instruction set: it is an enabling technology for open, customizable, and increasingly automated digital hardware development.**

# 3. The RISC-V

# Digital System and RISC-V Processor Design: Chisel versus Verilog

### Principles, Design Methodologies, Advantages, and Limitations

## 3.1 Introduction

The development of digital systems, and particularly RISC-V processors, traditionally relies on Hardware Description Languages (HDLs) such as Verilog and SystemVerilog. These languages describe the structure and behavior of digital circuits, including combinational logic, registers, finite-state machines, memories, and processor datapaths.
More recently, higher-level hardware construction languages such as **Chisel (Constructing Hardware in a Scala Embedded Language)** have introduced a different methodology. Instead of describing hardware directly in RTL, Chisel uses the Scala programming language to construct and generate hardware circuits.
The essential distinction is that **Verilog describes hardware directly, whereas Chisel is a programming language for generating hardware descriptions**.
Both approaches can ultimately produce synthesizable RTL implementations suitable for FPGA or ASIC development. However, they differ significantly in their abstraction mechanisms, development workflows, parameterization capabilities, debugging methods, and suitability for architectural exploration.
These differences become particularly important when developing RISC-V processors, where designers must coordinate instruction decoding, register files, arithmetic units, memory interfaces, control logic, pipelines, exceptions, and cache hierarchies.

## 3.2 Verilog: Direct Description of Digital Hardware

Verilog is a hardware description language specifically developed to model digital circuits. Its fundamental constructs correspond closely to hardware concepts such as signals, registers, combinational logic, and clocked processes.
For example, a 32-bit arithmetic operation can be expressed directly:

```
module Adder (
    input  wire [31:0] a,
    input  wire [31:0] b,
    output wire [31:0] result
);
    assign result = a + b;
endmodule
```

This description corresponds to a combinational 32-bit adder.
Sequential behavior is expressed using clock-sensitive processes. In SystemVerilog, constructs such as `always_comb` and `always_ff` make the distinction between combinational and sequential logic more explicit.
For RISC-V processor design, Verilog allows the designer to describe every hardware component directly: the instruction decoder, register file, ALU, multiplier, divider, control unit, and pipeline registers.

### Advantages of Verilog

The principal advantage of Verilog is its **direct correspondence with RTL hardware structures**.
Designers explicitly control signal widths, registers, clock cycles, and combinational dependencies. This makes Verilog particularly suitable for understanding and optimizing hardware timing, resource utilization, and control logic.
Verilog also benefits from a mature industrial ecosystem. Most FPGA and ASIC synthesis tools accept Verilog or SystemVerilog, and numerous simulators, verification frameworks, and debugging tools are available.
Another important advantage is accessibility. A relatively small collection of language constructs is sufficient to describe a complete RV32I processor.
For educational purposes, direct RTL descriptions make it easier to connect architectural concepts with actual digital circuits.

### Drawbacks of Verilog

Verilog becomes less convenient when architectures grow in complexity.
A five-stage RISC-V processor requires many interconnected control signals, pipeline registers, forwarding paths, and stall mechanisms. Managing these connections manually can produce repetitive code and increase the probability of integration errors.
Traditional Verilog also provides relatively limited abstraction and parameterization mechanisms compared with general-purpose programming languages.
SystemVerilog improves this situation through packages, structures, interfaces, parameters, and generate constructs. Nevertheless, constructing families of highly configurable processors can still require substantial engineering effort.
Consequently, Verilog is especially effective for **direct RTL implementation and detailed hardware control**, but may become cumbersome for large-scale architectural generation.

## 3.3 Chisel: Hardware Construction Using Scala

Chisel takes a fundamentally different approach.
Rather than being a conventional HDL, Chisel is a hardware construction language embedded in Scala. Designers write Scala programs that elaborate hardware structures.
For example, the same 32-bit adder can be written as:

```
import chisel3._
```

```
class Adder extends Module {
  val io = IO(new Bundle {
    val a      = Input(UInt(32.W))
    val b      = Input(UInt(32.W))
    val result = Output(UInt(32.W))
  })

  io.result := io.a + io.b
}
```

The Chisel toolchain elaborates this description and generates RTL, typically Verilog or SystemVerilog, for subsequent simulation and synthesis.
An important distinction is that Scala executes during hardware elaboration, whereas the generated circuit operates during hardware execution.
For example, a Scala loop can generate multiple hardware instances, but it does not necessarily represent a sequential loop executed by the circuit.
This separation between **hardware generation time and hardware execution time** is one of the most important conceptual differences between Chisel and Verilog.

### Advantages of Chisel

The main advantage of Chisel is its ability to describe hardware using powerful programming abstractions.
Scala functions, classes, collections, and parameterized generators can be used to construct reusable hardware components.
For example, a processor generator can support different configurations:

- RV32I or RV32IM instruction sets.
- Single-cycle, two-stage, or five-stage pipelines.
- Combinational or sequential arithmetic units.
- Different register-file organizations.
- Optional instruction and data caches.
- Configurable cache sizes and memory interfaces.

Instead of maintaining completely separate RTL implementations, designers can share modules and generate architectural variants from common descriptions.
Chisel also provides strong support for structured interfaces through `Bundle`, collections through `Vec`, and reusable parameterized modules.
These features are particularly useful when developing complex RISC-V processors containing numerous interconnected components.

### Drawbacks of Chisel

Chisel introduces an additional level of abstraction between the source code and the final hardware.
Designers must understand Scala, Chisel semantics, and the hardware elaboration process.
Debugging can also be more complex because errors may originate from Scala compilation, Chisel elaboration, RTL generation, or the behavior of the generated circuit.
Furthermore, the generated RTL may be less readable than carefully written Verilog. Signal names and intermediate structures can reflect the generation process rather than the original architectural organization.
This can complicate waveform analysis, manual RTL inspection, and integration with external verification environments.
Finally, Chisel requires a more elaborate development infrastructure, including a Scala build environment and the Chisel compiler toolchain.

## 3.4 Comparison for RISC-V Processor Development

The differences between Chisel and Verilog become particularly visible when progressively developing a RISC-V processor.
Consider an architectural sequence beginning with a minimal RV32I processor supporting R-type and I-type instructions.
The processor is subsequently extended to the complete RV32I instruction set, then to RV32IM with multiplication and sequential division, followed by two-stage and five-stage pipelined implementations.

### Instruction Decoder and Datapath

In Verilog, the decoder is typically implemented using `case` statements and combinational control logic.
Each instruction format and opcode must be explicitly handled.
In Chisel, the decoder can use similar hardware constructs, but Scala abstractions can additionally generate decoding tables, instruction definitions, and reusable control structures.
For a small RV32I decoder, the difference may be limited. For a larger processor supporting multiple extensions, Chisel's generation capabilities become more valuable.

### Multiplication and Sequential Division

Both languages can describe a combinational multiplier or a sequential restoring divider.
In Verilog, the divider is implemented directly using registers, arithmetic operations, and a finite-state machine.
In Chisel, the same circuit can be constructed using `Reg`, `Wire`, `Mux`, and `when` constructs.
**The resulting hardware architecture is determined primarily by the design choices, not by the source language.**
A sequential divider written in Chisel does not inherently consume fewer resources or operate faster than an equivalent Verilog divider.

### Pipeline Development

Pipeline implementation highlights a more significant difference.
In a two-stage processor, pipeline control is relatively simple. The principal mechanisms are instruction-register management, branch redirection, and divider stalls.

A five-stage processor introduces forwarding networks, hazard detection, pipeline flushing, and more complex control interactions.
In Verilog, these mechanisms are typically implemented through explicit RTL connections and control equations.
Chisel allows designers to define reusable pipeline-stage interfaces, parameterized registers, and common control structures.
This can reduce repetitive code and facilitate the generation of multiple pipeline configurations.
However, the designer must still define correct hazard-handling behavior. Chisel does not automatically solve pipeline hazards merely because the processor is described at a higher abstraction level.

### Cache Memory Integration

Cache memories introduce additional architectural complexity, including tag arrays, data arrays, replacement policies, refill controllers, and memory arbitration.
Both Verilog and Chisel can implement these mechanisms.
Chisel is particularly attractive when generating configurable cache structures because parameters such as cache capacity, associativity, line size, and address width can be managed through Scala abstractions.
Verilog remains advantageous when detailed control over memory inference, technology-specific RAM blocks, and timing-critical RTL is required.

## 3.5 Synthesis, Performance, and Verification

An essential point is that **Chisel does not directly produce a physical circuit**. It generates RTL that is subsequently processed by conventional synthesis and implementation tools.
Thus, both methodologies eventually use similar downstream tools:
**Verilog approach:**
Verilog/SystemVerilog → Simulation → Synthesis → FPGA/ASIC Implementation
**Chisel approach:**
Scala/Chisel → Hardware Elaboration → Generated RTL → Simulation → Synthesis → FPGA/ASIC Implementation
Because both flows ultimately produce RTL, neither language guarantees superior performance, area, or power consumption.
These characteristics depend on the generated hardware structure, synthesis optimizations, target technology, and physical implementation.
Verification also remains essential in both approaches.
Verilog designs can be tested using SystemVerilog testbenches, Icarus Verilog, Verilator, and waveform viewers such as GTKWave.
Chisel designs can use ChiselTest or other compatible testing frameworks, as well as conventional RTL simulators after hardware generation.
For RISC-V processors, both approaches should include instruction-level tests, randomized arithmetic verification, exception tests, pipeline timing checks, and comparisons against architectural reference models.

## 3.6 Comparative Summary

| Criterion | Verilog / SystemVerilog | Chisel |
|---|---|---|
| Design philosophy | Direct hardware description | Programmatic hardware construction |
| Main language | HDL | Scala with Chisel |
| Hardware abstraction | RTL-oriented | RTL with higher-level generation |
| Learning requirements | Digital logic and HDL | Digital logic, Scala, and Chisel |
| Small processor implementation | Simple and direct | Additional toolchain overhead |
| Parameterized architectures | Supported, but often more verbose | Strong support through Scala |
| Reusable hardware generators | Possible | Particularly well suited |
| Pipeline implementation | Explicit RTL connections | Reusable stage and control abstractions |
| Cache parameterization | Parameters and generate constructs | Scala-based generators |
| RTL readability | Generally direct | Generated RTL may be harder to inspect |
| Simulation | Direct | Native tests or generated RTL |
| Synthesis | Direct | Through generated RTL |
| Performance advantage | No inherent advantage | No inherent advantage |
| Industrial tool compatibility | Excellent | Excellent after RTL generation |
| Architectural exploration | Effective | Particularly effective |

## 3.7 AI-Assisted RISC-V Development: A Complementary Approach

The emergence of AI coding agents, such as Claude Code, introduces another dimension to this comparison.
AI agents can generate complete RISC-V implementations in either Chisel or Verilog, including hardware modules, testbenches, simulation scripts, and verification programs.
With Verilog, the agent produces RTL directly. This facilitates immediate inspection of the generated circuits and direct simulation using tools such as Icarus Verilog or Verilator.
With Chisel, the agent can exploit Scala abstractions to generate reusable and configurable processor architectures.
This becomes especially interesting when developing a family of processors, progressing from a simple RV32I implementation to complete RV32IM processors with sequential division, multiple pipeline depths, and optional caches.
The two approaches are therefore complementary.
Verilog provides transparency and precise RTL-level control, while Chisel facilitates architectural parameterization and systematic hardware generation.
In both cases, AI-generated implementations require rigorous verification. Automated test execution, reference-model comparison, and GTKWave waveform analysis are essential for detecting functional and timing errors.

## 3.8 Conclusion

Verilog and Chisel represent two different but compatible approaches to digital hardware development.
**Verilog emphasizes direct RTL description, explicit hardware control, and close interaction with established simulation and synthesis tools.**
**Chisel emphasizes hardware generation, reusable abstractions, parameterization, and rapid exploration of architectural alternatives.**
For small RISC-V processors, Verilog often provides the most straightforward and transparent implementation.
For larger processor families involving multiple ISA configurations, pipeline organizations, arithmetic units, and cache hierarchies, Chisel can significantly simplify design reuse and architectural exploration.
Neither approach inherently produces faster or smaller circuits. The quality of the resulting processor depends primarily on its microarchitecture, implementation decisions, and verification methodology.
For AI-assisted RISC-V development, the most effective strategy may therefore be to **combine both approaches**: use Chisel to explore and generate configurable architectures, while retaining Verilog/SystemVerilog as the common RTL representation for detailed simulation, verification, synthesis, and hardware implementation.
This complementary methodology provides a powerful foundation for the automated development of increasingly sophisticated RISC-V digital systems.

# Part I

# Building RISC-V architectures with Chisel

The emergence of artificial intelligence coding agents, such as **Claude Code**, opens new possibilities for the automated development of digital systems and RISC-V processor architectures. Combined with **Chisel**, a hardware construction language embedded in Scala, these agents can generate complete processor implementations from architectural specifications, including hardware modules, testbenches, simulation scripts, and verification environments. This approach significantly accelerates processor development while facilitating the exploration of alternative microarchitectures.

## Automated RISC-V Architecture Generation

Chisel is particularly well suited to AI-assisted hardware development because it combines hardware description capabilities with the abstraction and parameterization mechanisms of Scala. AI agents can generate reusable hardware components, including instruction decoders, register files, immediate generators, arithmetic logic units (ALUs), multipliers, dividers, control and status registers (CSRs), and memory interfaces.

Our methodology follows a **progressive architectural development strategy**. The process begins with a minimal RV32I processor supporting basic R-type and I-type arithmetic instructions. It then evolves toward the complete RV32I instruction set, incorporating memory accesses, conditional branches, jumps, and system instructions.

The next step introduces the RISC-V M extension, initially implementing multiplication through a combinational multiplier. The architecture is subsequently extended to the complete RV32IM instruction set by introducing a sequential divider supporting DIV, DIVU, REM, and REMU instructions.

Once the single-cycle processor is operational and verified, the AI agent generates more advanced microarchitectures, beginning with a **two-stage pipeline (IF/EX)** and progressing toward a conventional **five-stage pipeline (IF/ID/EX/MEM/WB)**. Further extensions introduce instruction and data caches, forwarding networks, hazard detection, pipeline stalls, and memory management mechanisms.

At each stage, the AI agent generates Chisel source code, elaborates the hardware, and produces synthesizable Verilog or SystemVerilog through the Chisel compilation infrastructure.

## Automatic Testing and Functional Verification

An essential characteristic of this methodology is the integration of **automatic verification throughout the development process**. Rather than generating hardware code alone, the AI agent also produces the corresponding unit tests, integration tests, and complete processor test programs.

Individual modules are verified independently. For example, ALU tests exercise arithmetic and logical operations, multiplier and divider tests cover signed and unsigned calculations, and register-file tests verify register access and the special behavior of register x0.

At the processor level, automatically generated RISC-V programs exercise instruction execution, memory accesses, branches, jumps, exceptions, and control-flow changes.

For pipelined architectures, additional tests verify instruction timing, forwarding, hazard detection, pipeline flushing, and multicycle divider stalls.

A particularly effective verification technique consists of comparing the processor execution against a **software reference model**. At every retired instruction, the program counter, instruction word, destination register, computed result, memory accesses, and exception information can be compared against the expected architectural behavior.

The AI agent can automatically execute the tests, analyze compilation or simulation failures, modify the generated Chisel code, and repeat the verification process. This iterative workflow reduces repetitive debugging effort while improving implementation reliability. Nevertheless, verification quality remains dependent on the completeness of the test suite and the correctness of the reference model.

## Waveform Generation and Visual Analysis with GTKWave

Beyond functional verification, **waveform visualization is an important component of AI-assisted processor development**.

The generated simulation environment can produce Value Change Dump (**VCD**) files containing the evolution of internal processor signals over successive clock cycles.

These waveforms can be displayed using **GTKWave**, providing a graphical representation of processor operation.

Typical signals include the clock, program counter, fetched instruction, decoded operation, register operands, ALU results, memory addresses, write-back data, and instruction retirement indicators.

For pipelined processors, the waveforms additionally illustrate pipeline-stage contents, instruction dependencies, forwarding operations, stalls, and branch redirections.

The sequential divider provides a particularly interesting example: GTKWave can display the progression of division iterations, the divider's busy and completion signals, and the resulting pipeline stall.

Similarly, cache-equipped processors can be analyzed through waveforms showing cache hits, misses, refill operations, and memory-interface transactions.

These timing diagrams provide **visual evidence of the generated architecture's dynamic behavior**, complementing automated functional tests. They are also valuable for teaching, debugging, documentation, and comparing alternative microarchitectures.

## Conclusion

The combination of **AI coding agents, Chisel, automated testing, and GTKWave visualization** establishes a powerful methodology for RISC-V processor development.

It enables the progressive generation of increasingly sophisticated architectures, from elementary RV32I processors to complete RV32IM implementations with sequential arithmetic units, multistage pipelines, and cache memories.

Each development stage produces a complete hardware implementation accompanied by automated tests, simulation results, and timing waveforms.

This approach transforms processor development into an iterative process of **specification, generation, simulation, verification, visualization, and refinement**.
By reducing repetitive implementation tasks and accelerating architectural experimentation, AI-assisted Chisel development offers significant opportunities for digital design education, computer architecture research, and the rapid prototyping of customizable RISC-V processors.
Add a concrete Chisel testing exampleShow the VCD generation workflow

# GLab 0

## Installing Chisel 3 on Ubuntu Linux

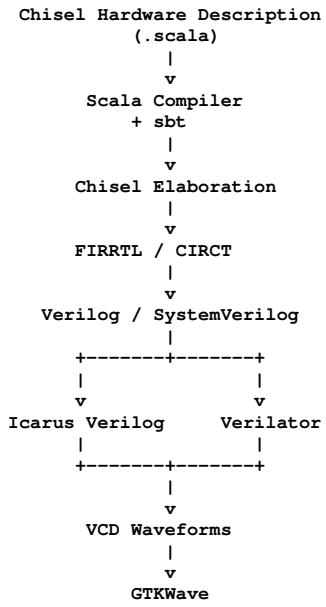
### Development Tools, Installation Procedure, Simulation, and Waveform Visualization

## 0.1 Introduction

**Chisel (Constructing Hardware in a Scala Embedded Language)** is a hardware construction language that enables digital circuits to be described using Scala. It is particularly suitable for developing RISC-V processors because it supports modular hardware design, reusable components, parameterized architectures, and automatic generation of Verilog.
A typical Chisel development environment includes several complementary tools:

| Tool | Function |
|---|---|
| **Java Development Kit (JDK)** | Executes the Scala compiler and Chisel development tools |
| **Scala** | Programming language used to describe and generate hardware |
| **sbt** | Scala build tool for compiling Chisel projects and running tests |
| **Chisel 3** | Hardware construction library |
| **FIRRTL** / **CIRCT** | Hardware intermediate representations and compilation infrastructure |
| **Icarus Verilog** | Simulation of generated Verilog/SystemVerilog |
| **Verilator** | High-performance RTL simulation and verification |
| **GTKWave** | Visualization of digital simulation waveforms |
| **Git** | Version control and retrieval of example projects |
| **Make** | Automation of compilation, simulation, and testing |

The main development workflow is:




**Important version distinction:** Chisel 3 refers to the older Chisel library release series, whereas current Chisel releases use newer version numbers and different dependency coordinates. A project intended specifically for Chisel 3 should use a compatible Chisel 3 dependency rather than automatically adopting the latest Chisel release.
The following procedure targets Ubuntu and Ubuntu-derived Linux distributions, including Linux Mint and Pop!_OS.

## 2. Installing the Basic Development Tools

First, update the Ubuntu package database:

```
sudo apt update
sudo apt upgrade
```

Install the essential development packages:

```
sudo apt install -y \
    git \
    curl \
    wget \
    unzip \
    build-essential \
    make \
```

```
    openjdk-17-jdk
```

These packages provide the Java runtime, compiler tools, version control, and standard utilities required for Chisel development.
Check the Java installation:

```
java -version
javac -version
```

For example:

```
openjdk version "17..."
javac 17...
```

Java 17 is a practical choice for many recent Chisel 3 projects. However, older Chisel 3 projects may require Java 8 or Java 11, depending on their Scala and sbt versions.

## 3. Installing Scala and sbt

Scala is the programming language used to construct Chisel hardware modules.
Although Scala can be installed independently, **sbt normally downloads and manages the Scala compiler automatically** according to the project's configuration.
Therefore, installing a separate system-wide Scala compiler is generally unnecessary.
The recommended approach is to install sbt and specify the desired Scala version in the project configuration.
Install sbt using its official Ubuntu repository:

```
sudo apt install -y ca-certificates gnupg

sudo install -d -m 0755 /etc/apt/keyrings

curl -fsSL \
  "https://keyserver.ubuntu.com/pks/lookup?op=get&search=0x2EE0EA64E40A89B84B2DF73499E82A75642AC823" \
  | sudo gpg --dearmor -o /etc/apt/keyrings/scalasbt.gpg

echo "deb [signed-by=/etc/apt/keyrings/scalasbt.gpg] https://repo.scala-sbt.org/scalasbt/debian all main" \
  | sudo tee /etc/apt/sources.list.d/sbt.list

sudo apt update
sudo apt install -y sbt
```

These commands follow the official sbt Linux installation instructions.
Check the installation:

```
sbt -version
```

The first execution may download additional components.

## 4. Installing the RTL Simulation and Visualization Tools

Install Icarus Verilog, Verilator, and GTKWave:

```
sudo apt install -y \
    iverilog \
    verilator \
    gtkwave
```

Verify the installations:

```
iverilog -V
verilator --version
gtkwave --version
```

These tools have complementary purposes.
**Icarus Verilog** provides a convenient environment for compiling and simulating Verilog/SystemVerilog designs.
For example:

```
iverilog -g2012 -s tb_core \
    -o core.vvp \
    Core.sv tb_core.sv

vvp core.vvp
```

**Verilator** is particularly useful for fast simulation of complex processors and extensive regression testing.
**GTKWave** displays waveform files generated during simulation:

```
gtkwave core.vcd
```

For the development of RV32I and RV32IM processors, these three tools provide a useful RTL verification environment.

## 5. Creating a Chisel 3 Project

We will use **Chisel 3.6.0**, Scala 2.13.10, and chiseltest 0.6.0. This is a documented compatible combination from the Chisel project. GitHub
Create the project:

```
mkdir -p ~/chisel-riscv
cd ~/chisel-riscv

mkdir -p src/main/scala
```

```
mkdir -p src/test/scala
mkdir -p project
```

The directory structure is:

```
chisel-riscv/
├── build.sbt
├── project/
│   └── build.properties
├── src/
│   ├── main/
│   │   └── scala/
│   │       └── Adder.scala
│   └── test/
│       └── scala/
│           └── AdderSpec.scala
└── generated/
```

### Configure the Scala project

Create `build.sbt`:

```
cat > build.sbt <<'EOF'
ThisBuild / scalaVersion := "2.13.10"

ThisBuild / version := "0.1.0"

ThisBuild / organization := "rv32i"

val chiselVersion = "3.6.0"

libraryDependencies ++= Seq(
  "edu.berkeley.cs" %% "chisel3" % chiselVersion,
  "edu.berkeley.cs" %% "chiseltest" % "0.6.0" % Test
)

addCompilerPlugin(
  "edu.berkeley.cs" % "chisel3-plugin" %
    chiselVersion cross CrossVersion.full
)

scalacOptions ++= Seq(
  "-deprecation",
  "-feature"
)
EOF
```

Create `project/build.properties`:

```
echo 'sbt.version=1.9.7' > project/build.properties
```

The version declared in `build.properties` determines the sbt version used for the project.
The project configuration automatically downloads Chisel, Scala, and the testing libraries.
**Note:** Chisel 3.6.x also supports a CIRCT-based RTL emission flow. When using that flow, `firtool` must be available. For Chisel 3.6.x, the documented compatible `firtool` version is 1.37.0. The example below uses the traditional Chisel 3 Verilog emission API, which avoids requiring a separate `firtool` installation for this introductory project. Chisel

## 6. Creating a First Chisel Module

Create `src/main/scala/Adder.scala`:

```
import chisel3._
import chisel3.stage.ChiselStage

class Adder extends Module {

  val io = IO(new Bundle {
    val a = Input(UInt(32.W))
    val b = Input(UInt(32.W))
    val result = Output(UInt(32.W))
  })

  io.result := io.a + io.b
}

object GenerateAdder extends App {

  (new ChiselStage).emitVerilog(
    new Adder,
    Array("--target-dir", "generated")
  )
}
```

This module describes a combinational 32-bit adder.
Compile the project:

```
sbt compile
```

Generate the Verilog implementation:
```
sbt "runMain GenerateAdder"
```

The generated RTL is written to the `generated` directory.
Inspect it:

```
ls generated
cat generated/Adder.v
```

The output contains a Verilog module implementing the adder.
This demonstrates the fundamental principle of Chisel:
**Scala/Chisel source → Hardware elaboration → Verilog RTL**

## 7. Creating Automated Unit Tests

Chisel 3 can use the **chiseltest** library to verify hardware behavior.
Create **`src/test/scala/AdderSpec.scala:`**

```
import chisel3._
import chiseltest._
import org.scalatest.flatspec.AnyFlatSpec

class AdderSpec extends AnyFlatSpec
    with ChiselScalatestTester {

  behavior of "Adder"

  it should "add two 32-bit numbers" in {

    test(new Adder) { dut =>

      dut.io.a.poke(10.U)
      dut.io.b.poke(20.U)

      dut.io.result.expect(30.U)

      dut.io.a.poke(100.U)
      dut.io.b.poke(200.U)

      dut.io.result.expect(300.U)

      dut.io.a.poke("hFFFFFFFF".U)
      dut.io.b.poke(1.U)

      dut.io.result.expect(0.U)
    }
  }
}
```

Execute the tests:
```
sbt test
```

The testbench verifies the expected addition results, including 32-bit wraparound.
For RISC-V processor development, the same approach can be extended to verify:

- ALU operations and immediate generation.
- Instruction decoding.
- Register-file behavior.
- Signed and unsigned multiplication.
- Sequential division and remainder operations.
- CSR operations and exceptions.
- Pipeline stalls, forwarding, and branch redirection.
- Instruction and data cache operations.

## 8. Generating VCD Waveforms

Waveform generation is particularly useful when studying pipelined processors and multicycle arithmetic units.
Create an additional test file, **`src/test/scala/AdderWaveSpec.scala:`**

```
import chisel3._
import chiseltest._
import org.scalatest.flatspec.AnyFlatSpec

class AdderWaveSpec extends AnyFlatSpec
    with ChiselScalatestTester {

  behavior of "Adder waveform"
```

```
  it should "generate simulation waveforms" in {

    test(new Adder)
      .withAnnotations(Seq(WriteVcdAnnotation)) {
        dut =>

        dut.io.a.poke(10.U)
        dut.io.b.poke(20.U)
        dut.clock.step()

        dut.io.a.poke(100.U)
        dut.io.b.poke(200.U)
        dut.clock.step()

        dut.io.a.poke(1234.U)
        dut.io.b.poke(5678.U)
        dut.clock.step()
      }
  }
}
```

Execute:
`sbt "testOnly AdderWaveSpec"`
The simulation generates a VCD waveform file under the test output directory.
Locate it:
`find test_run_dir -name "*.vcd"`
Open the waveform:
`gtkwave path/to/generated_waveform.vcd`
Replace the example path with the actual filename returned by `find`.
GTKWave allows the designer to inspect the evolution of the input operands and the corresponding arithmetic results.
For a RISC-V processor, the same mechanism can display the program counter, fetched instruction, decoded instruction, ALU inputs, register-file writes, memory transactions, and pipeline control signals.
In particular, a sequential divider can be analyzed by observing its iteration counter, busy signal, quotient, remainder, and completion handshake.

## 9. Recommended Workflow for RISC-V Processor Development

A practical Chisel-based processor project can use the following structure:

```
riscv-core/
├── build.sbt
├── project/
│   └── build.properties
├── src/
│   ├── main/scala/
│   │   ├── Alu.scala
│   │   ├── Decoder.scala
│   │   ├── ImmGen.scala
│   │   ├── RegFile.scala
│   │   ├── Multiplier.scala
│   │   ├── DivUnit.scala
│   │   ├── CsrFile.scala
│   │   └── Core.scala
│   └── test/scala/
│       ├── AluSpec.scala
│       ├── DecoderSpec.scala
│       ├── MultiplierSpec.scala
│       ├── DivUnitSpec.scala
│       ├── CoreSpec.scala
│       └── CoreTimingSpec.scala
├── generated/
├── programs/
└── waveforms/
```

The typical development sequence is:
**Step 1 Generate hardware**
`sbt "runMain GenerateCore"`
This assumes the project defines a `GenerateCore` object that elaborates the processor.
**Step 2 Execute automated tests**
`sbt test`
**Step 3 Run a specific processor test**
`sbt "testOnly CoreTimingSpec"`
**Step 4 Generate and inspect waveforms**
Enable `WriteVcdAnnotation` in the relevant test, run it, and open the resulting VCD file with GTKWave.
**Step 5 Simulate the generated RTL independently**

For a generated processor and a compatible SystemVerilog testbench:

```
iverilog -g2012 \
    -s tb_core \
    -o core.vvp \
    generated/Core.v \
    tb_core.sv
```

```
vvp core.vvp
```
For more complex designs, Verilator may offer better simulation performance and broader support for SystemVerilog constructs.

## 10. Summary

The essential Ubuntu development environment for **Chisel 3** consists of Java, sbt, Chisel, and a compatible Scala version.
For complete RISC-V processor development, it is useful to add Icarus Verilog, Verilator, and GTKWave.
These tools support a complete workflow:
**Chisel Design → Verilog Generation → Automated Testing → RTL Simulation → VCD Waveforms → GTKWave Visualization**
The combination is particularly appropriate for progressively developing RV32I and RV32IM processors, from basic single-cycle implementations to pipelined architectures with sequential division and cache memories.

### Official documentation

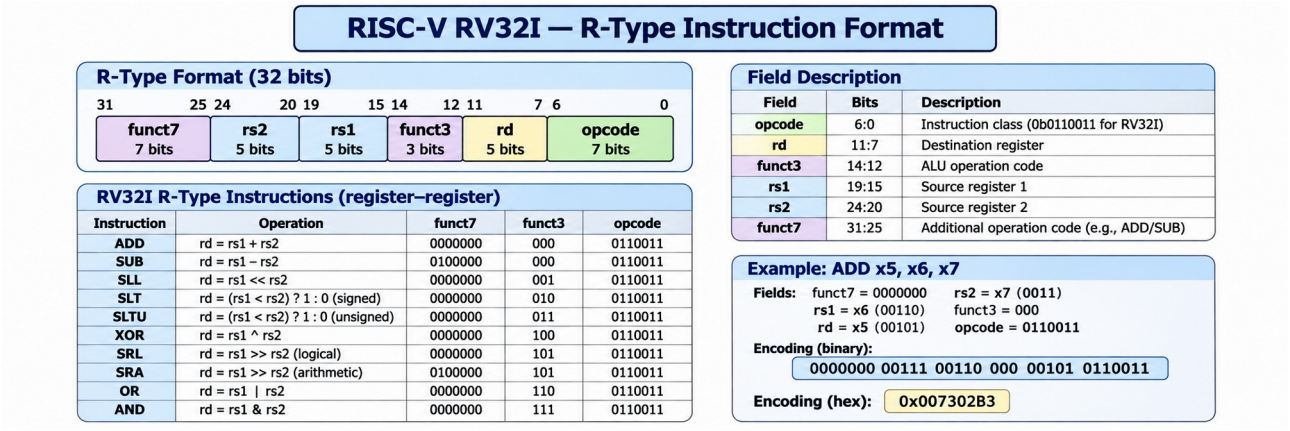
The following references provide additional installation and development information:

- Chisel installation guide
- Chisel source repository
- Chisel version compatibility
- sbt installation guide
- Verilator documentation
- GTKWave project

# GLab 1

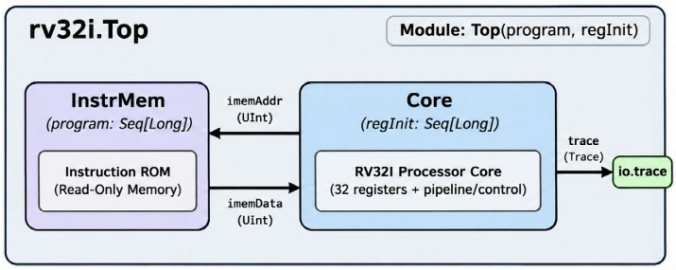
## Building simple RV32I (R-type only) RISC-V architecture

Let us start with simple RV32I (R-types) obly architectures. Ther following figure illustartes the RV32I instruction formats relative to R-type instructions.




The generated architecture contains at the highest level - **`RV32ITestWrapper`** - two modules:

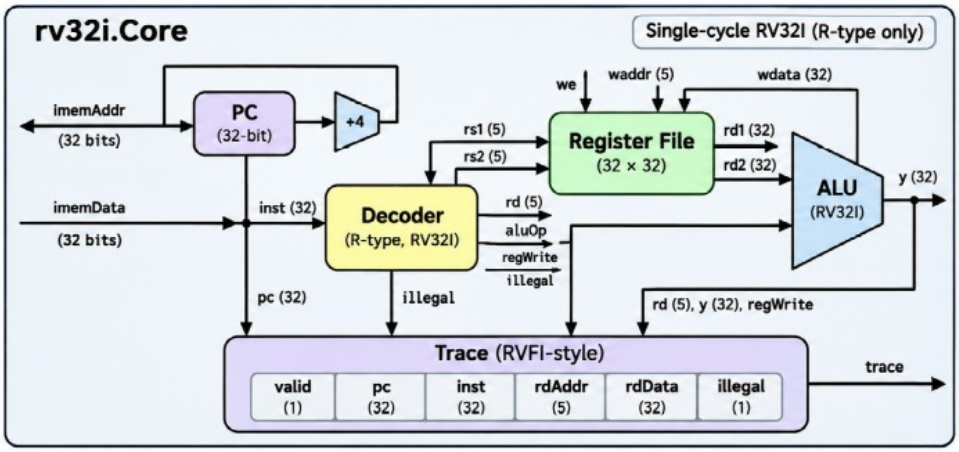
- **`InstructionMemory`** and
- **`RV32IRCore`**




```
-------------------------------------------------------------------------------------------
package rv32i
import chisel3._
/** RV32IRCore + InstructionMemory loaded from a hex file */
class RV32ITestWrapper(
  p: CoreParams,
  memoryFile: String,
  initialRegs: Seq[BigInt] = Seq.empty
) extends Module {
  val io = IO(new Bundle {
    val debugPC = Output(UInt(p.xlen.W))
    val debugX1 = Output(UInt(p.xlen.W))
    val debugX2 = Output(UInt(p.xlen.W))
    val debugX3 = Output(UInt(p.xlen.W))
    val debugX4 = Output(UInt(p.xlen.W))
    val trace   = Output(new Trace(p))
  })
  val core = Module(new RV32IRCore(p, initialRegs))
  val imem = Module(new InstructionMemory(memoryFile))
  imem.io.addr     := core.io.imemAddr
  core.io.imemInst := imem.io.inst
  io.debugPC := core.io.debugPC
  io.debugX1 := core.io.debugX1
  io.debugX2 := core.io.debugX2
  io.debugX3 := core.io.debugX3
  io.debugX4 := core.io.debugX4
  io.trace   := core.io.trace
}
-------------------------------------------------------------------------------------------
```

The **`Core`** module contains other three modules:

- **`RegisterFile.scala`**
- **`ALU.scala`**
- **`Decoder.scala`**

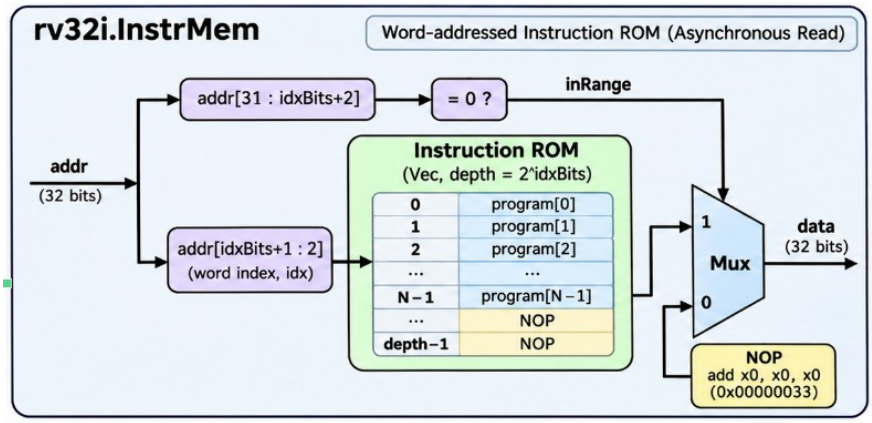



---

```
package rv32i
import chisel3._
import chisel3.util._
/** Single-cycle RV32I ALU core (lab1): executes the 19 RV32I ALU instructions, one per clock cycle.
  *
  *  - R-type: ADD SUB SLL SLT SLTU XOR SRL SRA OR AND        (ALU b = rs2)
  *  - I-type: ADDI SLTI SLTIU XORI ORI ANDI SLLI SRLI SRAI   (ALU b = sign-extended immediate)
  * There are no loads, stores, branches or jumps: the pc always advances by 4. An instruction the decoder does
not
  * recognise is executed as a no-op (no register write, trace.valid = 0). Registers can be preset with
  * `initialRegs` (x0 stays 0).
  */
class RV32IRCore(
  p: CoreParams = CoreParams(),
  initialRegs: Seq[BigInt] = Seq.empty,
  resetPc: BigInt = 0
) extends Module {
  require(p.xlen == 32, "RV32IRCore implements RV32I: xlen must be 32")
  require(p.numRegs == 32, "RV32IRCore implements RV32I: 32 registers")
  val io = IO(new Bundle {
    // Instruction-memory interface (combinational read).
    val imemAddr = Output(UInt(p.xlen.W))
    val imemInst = Input(UInt(32.W))
    // Debug outputs for laboratory verification.
    val debugPC = Output(UInt(p.xlen.W))
    val debugX1 = Output(UInt(p.xlen.W))
    val debugX2 = Output(UInt(p.xlen.W))
    val debugX3 = Output(UInt(p.xlen.W))
    val debugX4 = Output(UInt(p.xlen.W))
    // What was executed this cycle (used by the test harness and in waveforms).
    val trace = Output(new Trace(p))
  })
  // ---------------------------------------------------------------- program counter
  val pc = RegInit(resetPc.U(p.xlen.W))
  io.imemAddr := pc
  io.debugPC  := pc
  // ---------------------------------------------------------------- decode
  val decoder = Module(new Decoder)
  decoder.io.instr := io.imemInst
  val immGen = Module(new ImmediateGenerator(p))
  immGen.io.instr := io.imemInst
  // ---------------------------------------------------------------- register file
  val regFile = Module(new RegisterFile(p, initialRegs))
  regFile.io.rs1 := decoder.io.rs1
  regFile.io.rs2 := decoder.io.rs2
  // ---------------------------------------------------------------- ALU
  val alu = Module(new ALU(p))
  alu.io.a  := regFile.io.readData1
  alu.io.b  := Mux(decoder.io.aluSrc, immGen.io.imm, regFile.io.readData2)
  alu.io.op := decoder.io.aluOp
  // ---------------------------------------------------------------- write-back
  regFile.io.rd          := decoder.io.rd
  regFile.io.writeData   := alu.io.y
  regFile.io.writeEnable := decoder.io.regWrite
  // ---------------------------------------------------------------- next pc
  pc := pc + 4.U
  // ---------------------------------------------------------------- debug and trace
  io.debugX1 := regFile.io.debugRead(1)
  io.debugX2 := regFile.io.debugRead(2)
  io.debugX3 := regFile.io.debugRead(3)
  io.debugX4 := regFile.io.debugRead(4)
  val writes = decoder.io.regWrite && decoder.io.rd =/= 0.U
  io.trace.pc     := pc
  io.trace.inst   := io.imemInst
  io.trace.valid  := decoder.io.valid
  io.trace.rdAddr := Mux(writes, decoder.io.rd, 0.U)
```

```
  io.trace.rdData := Mux(writes, alu.io.y, 0.U)
  assert(!(decoder.io.regWrite && !decoder.io.valid), "register write by an unrecognised instruction")
}
```

---



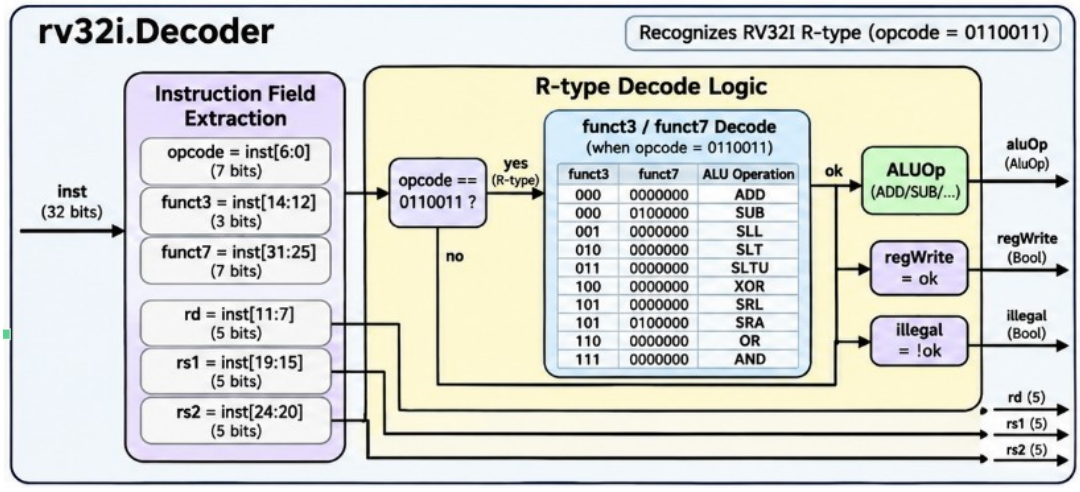

---

```
package rv32i
import chisel3._
import scala.io.Source

/** Read-only instruction memory initialised from a hex file at elaboration time (one 32-bit word per line;
  * blank lines, `#` comments and `_` separators allowed). Combinational read; word address = addr >> 2;
  * words past the end of the file read as 0 (not a valid instruction: executed as a no-op by the core).
  */
class InstructionMemory(
  fileName: String,
  depth: Int = 1024
) extends Module {
  val io = IO(new Bundle {
    val addr = Input(UInt(32.W))
    val inst = Output(UInt(32.W))
  })
  val instructions: Seq[BigInt] = InstructionMemory.readHex(fileName)
  require(instructions.nonEmpty, s"Instruction memory file '$fileName' is empty")
  require(instructions.length <= depth, s"Instruction memory contains ${instructions.length} words, but depth is
only $depth")
  val mem = VecInit(instructions.map(_.U(32.W)) ++ Seq.fill(depth - instructions.length)(0.U(32.W)))
  val wordAddr = io.addr >> 2
  io.inst := Mux(wordAddr < depth.U, mem(wordAddr(chisel3.util.log2Ceil(depth) - 1, 0)), 0.U)
}

object InstructionMemory {
  /** Parse a hex file: one word per line, `#` comments, `_` separators. */
  def readHex(fileName: String): Seq[BigInt] = {
    val source = Source.fromFile(fileName)
    try {
      source.getLines().toVector
        .map(_.split("#").headOption.getOrElse("").trim)
        .filter(_.nonEmpty)
        .map(line => BigInt(line.replace("_", ""), 16))
    } finally source.close()
  }
}
```

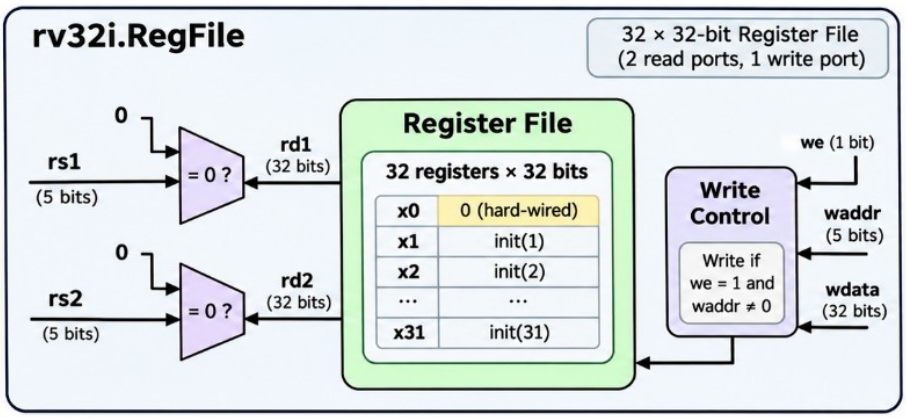
---




---

```
package rv32i
import chisel3._
import chisel3.util._
/** Decoder for the RV32I ALU instructions:
  *  - R-type (opcode 0110011): ADD SUB SLL SLT SLTU XOR SRL SRA OR AND
  *  - I-type (opcode 0010011): ADDI SLTI SLTIU XORI ORI ANDI SLLI SRLI SRAI
  * `valid` is high only for these 19 encodings; anything else (other opcodes, other funct7 values such as the
  * M extension, bad shift encodings) gives valid = 0, aluOp = NOP and no register write.
  */
class Decoder extends Module {
  val io = IO(new Bundle {
    val instr    = Input(UInt(32.W))
    val rs1      = Output(UInt(5.W))
    val rs2      = Output(UInt(5.W))
    val rd       = Output(UInt(5.W))
    val aluOp    = Output(UInt(ALUOp.width.W))
    val aluSrc   = Output(Bool())   // 0: ALU b = rs2, 1: ALU b = immediate
    val regWrite = Output(Bool())
    val valid    = Output(Bool())
  })
  val opcode = io.instr(6, 0)
  val funct3 = io.instr(14, 12)
  val funct7 = io.instr(31, 25)
  io.rs1 := io.instr(19, 15)
  io.rs2 := io.instr(24, 20)
  io.rd  := io.instr(11, 7)
  io.aluOp  := ALUOp.NOP
  io.aluSrc := false.B
  io.valid  := false.B
  // R-type: funct7 ## funct3 selects the operation
  when(opcode === "b0110011".U) {
    io.aluOp := MuxLookup(Cat(funct7, funct3), ALUOp.NOP)(Seq(
      "b0000000000".U -> ALUOp.ADD,
      "b0100000000".U -> ALUOp.SUB,
      "b0000000001".U -> ALUOp.SLL,
      "b0000000010".U -> ALUOp.SLT,
      "b0000000011".U -> ALUOp.SLTU,
      "b0000000100".U -> ALUOp.XOR,
      "b0000000101".U -> ALUOp.SRL,
      "b0100000101".U -> ALUOp.SRA,
      "b0000000110".U -> ALUOp.OR,
      "b0000000111".U -> ALUOp.AND
    ))
    io.valid := io.aluOp =/= ALUOp.NOP
  }
  // I-type ALU: the second operand is the immediate
  when(opcode === "b0010011".U) {
    io.aluSrc := true.B
    io.aluOp := MuxLookup(funct3, ALUOp.NOP)(Seq(
      "b000".U -> ALUOp.ADD,                                                    // ADDI
      "b010".U -> ALUOp.SLT,                                                    // SLTI
      "b011".U -> ALUOp.SLTU,                                                   // SLTIU
      "b100".U -> ALUOp.XOR,                                                    // XORI
      "b110".U -> ALUOp.OR,                                                     // ORI
      "b111".U -> ALUOp.AND,                                                    // ANDI
      "b001".U -> Mux(funct7 === "b0000000".U, ALUOp.SLL, ALUOp.NOP),           // SLLI
      "b101".U -> Mux(funct7 === "b0000000".U, ALUOp.SRL,                       // SRLI
                  Mux(funct7 === "b0100000".U, ALUOp.SRA, ALUOp.NOP))           // SRAI
    ))
    io.valid := io.aluOp =/= ALUOp.NOP
  }
  io.regWrite := io.valid
}
```

---

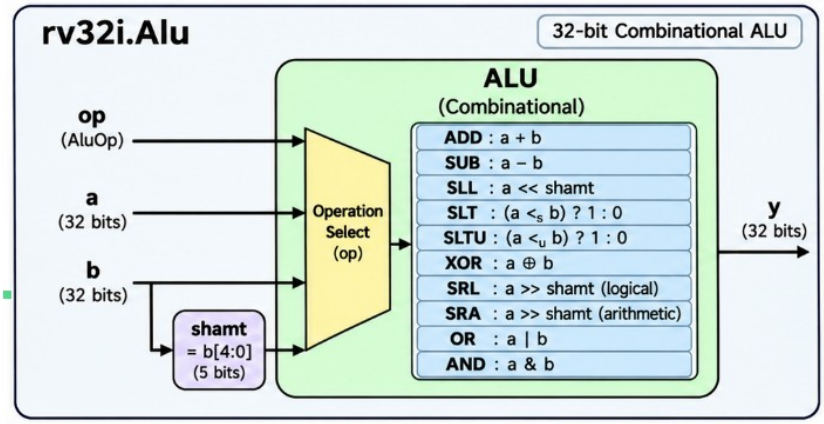



---

```
package rv32i
import chisel3._
import chisel3.util._
/** Register file: two combinational read ports, one write port, x0 hardwired to zero, optional reset values,
  * and every register visible on `debugRead` for laboratory verification.*/
```

```
class RegisterFile(
  p: CoreParams,
  initialValues: Seq[BigInt] = Seq.empty
) extends Module {
  require(initialValues.isEmpty || initialValues.length == p.numRegs,
    "initialValues must be empty or contain exactly numRegs entries")
  val io = IO(new Bundle {
    val rs1         = Input(UInt(log2Ceil(p.numRegs).W))
    val rs2         = Input(UInt(log2Ceil(p.numRegs).W))
    val rd          = Input(UInt(log2Ceil(p.numRegs).W))
    val writeData   = Input(UInt(p.xlen.W))
    val writeEnable = Input(Bool())
    val readData1   = Output(UInt(p.xlen.W))
    val readData2   = Output(UInt(p.xlen.W))
    val debugRead   = Output(Vec(p.numRegs, UInt(p.xlen.W)))
  })
  val resetValues =
    if (initialValues.nonEmpty) initialValues.zipWithIndex.map
      { case (v, i) => (if (i == 0) BigInt(0) else v).U(p.xlen.W) }
    else Seq.fill(p.numRegs)(0.U(p.xlen.W))
  val regs = RegInit(VecInit(resetValues))
  io.readData1 := Mux(io.rs1 === 0.U, 0.U, regs(io.rs1))
  io.readData2 := Mux(io.rs2 === 0.U, 0.U, regs(io.rs2))
  when(io.writeEnable && io.rd =/= 0.U) {
    regs(io.rd) := io.writeData
  }
  for (i <- 0 until p.numRegs) {
    io.debugRead(i) := (if (i == 0) 0.U else regs(i))
  }
}
```

---




---

```
import chisel3._
import chisel3.util._
/** ALU operation codes (UInt constants, as in lab1). */
object ALUOp {
  val width = 4
  val ADD  = 0.U(width.W)
  val SUB  = 1.U(width.W)
  val SLL  = 2.U(width.W)
  val SLT  = 3.U(width.W)
  val SLTU = 4.U(width.W)
  val XOR  = 5.U(width.W)
  val SRL  = 6.U(width.W)
  val SRA  = 7.U(width.W)
  val OR   = 8.U(width.W)
  val AND  = 9.U(width.W)
  val NOP  = 15.U(width.W)   // result 0 (used for instructions the decoder does not recognise)
}
/** Combinational ALU. Shifts use the low log2(xlen) bits of `b`. */
class ALU(p: CoreParams) extends Module {
  val io = IO(new Bundle {
    val a  = Input(UInt(p.xlen.W))
    val b  = Input(UInt(p.xlen.W))
    val op = Input(UInt(ALUOp.width.W))
    val y  = Output(UInt(p.xlen.W))
  })
  val shamt = io.b(log2Ceil(p.xlen) - 1, 0)
  io.y := MuxLookup(io.op, 0.U(p.xlen.W))(Seq(
    ALUOp.ADD  -> (io.a + io.b),
    ALUOp.SUB  -> (io.a - io.b),
    ALUOp.SLL  -> (io.a << shamt)(p.xlen - 1, 0),
    ALUOp.SLT  -> (io.a.asSInt < io.b.asSInt).asUInt,
    ALUOp.SLTU -> (io.a < io.b).asUInt,
    ALUOp.XOR  -> (io.a ^ io.b),
    ALUOp.SRL  -> (io.a >> shamt),
    ALUOp.SRA  -> (io.a.asSInt >> shamt).asUInt,
    ALUOp.OR   -> (io.a | io.b),
    ALUOp.AND  -> (io.a & io.b)
```

```
  ))
}
```

---

```
package rv32i
/** Core configuration. The instruction set is RV32I, so `xlen` must be 32 for `RV32IRCore`;
  * the ALU and the register file are written for any width and register count.
  */
case class CoreParams(
  xlen: Int = 32,
  numRegs: Int = 32
)
```

---

### Test class:

---

```
package rv32i
import chisel3._
import chiseltest._
import org.scalatest.flatspec.AnyFlatSpec
import scala.util.Random
import TestUtil._

class ALUSpec extends AnyFlatSpec with ChiselScalatestTester {
  private val ops: Seq[(UInt, String, (Long, Long) => Long)] = Seq(
    (ALUOp.ADD, "ADD", (a, b) => a + b),
    (ALUOp.SUB, "SUB", (a, b) => a - b),
    (ALUOp.SLL, "SLL", (a, b) => a << (b & 31)),
    (ALUOp.SLT, "SLT", (a, b) => if (s32(a) < s32(b)) 1 else 0),
    (ALUOp.SLTU, "SLTU", (a, b) => if (a < b) 1 else 0),
    (ALUOp.XOR, "XOR", (a, b) => a ^ b),
    (ALUOp.SRL, "SRL", (a, b) => a >>> (b & 31)),
    (ALUOp.SRA, "SRA", (a, b) => s32(a) >> (b & 31)),
    (ALUOp.OR, "OR", (a, b) => a | b),
    (ALUOp.AND, "AND", (a, b) => a & b),
    (ALUOp.NOP, "NOP", (_, _) => 0L)
  )
  "ALU" should "pass the lab1 checks" in {
    test(new ALU(CoreParams())).withAnnotations(TestUtil.waves) { dut =>
      def check(a: BigInt, b: BigInt, op: UInt, expected: BigInt): Unit = {
        dut.io.a.poke(a.U); dut.io.b.poke(b.U); dut.io.op.poke(op)
        dut.io.y.expect(expected.U)
      }
      check(10, 3, ALUOp.ADD, 13); check(10, 3, ALUOp.SUB, 7); check(10, 3, ALUOp.AND, 2)
      check(10, 3, ALUOp.OR, 11); check(10, 3, ALUOp.XOR, 9); check(1, 4, ALUOp.SLL, 16);check(16,2,ALUOp.SRL, 4)
      check(BigInt("ffffffff", 16), 1, ALUOp.SLT, 1); check(BigInt("ffffffff", 16), 1, ALUOp.SLTU, 0)
      check(BigInt("80000000", 16), 1, ALUOp.SRA, BigInt("c0000000", 16)); check(123, 456, ALUOp.NOP, 0)
    }
  }
  it should "compute every operation on all edge-value pairs (overflow, shift by 31/32, sign)" in {
    test(new ALU(CoreParams())).withAnnotations(TestUtil.waves) { c =>
      for ((op, name, f) <- ops; a <- edges; b <- edges) {
        c.io.op.poke(op); c.io.a.poke(a.U(32.W)); c.io.b.poke(b.U(32.W))
        val got = c.io.y.peek().litValue.toLong
        assert(got == (f(a, b) & M), f"$name 0x$a%08x 0x$b%08x: got 0x$got%08x")
      }
    }
  }
  it should "match the reference on random operands and give 0 for unused operation codes" in {
    test(new ALU(CoreParams())).withAnnotations(TestUtil.waves) { c =>
      val r = new Random(1)
      for (_ <- 0 until 3000) {
        val (op, name, f) = ops(r.nextInt(ops.length))
        val a = rnd32(r); val b = rnd32(r)
        c.io.op.poke(op); c.io.a.poke(a.U(32.W)); c.io.b.poke(b.U(32.W))
        val got = c.io.y.peek().litValue.toLong
        assert(got == (f(a, b) & M), f"$name 0x$a%08x 0x$b%08x: got 0x$got%08x")
      }
      for (code <- 10 to 14) { c.io.op.poke(code.U); c.io.a.poke(5.U); c.io.b.poke(7.U); c.io.y.expect(0.U) }
    }
  }
}
```

---

# Understanding the Chisel ALU Test File (`ALUSpec.scala`)

### Purpose of the test file

ALUSpec.scala is an automated verification testbench for the Arithmetic Logic Unit (ALU) of an RV32I RISC-V processor. It uses Chisel 3 to describe the hardware, **`ChiselTest`** to apply inputs and inspect outputs, and **`ScalaTest`** to organize tests and report failures.
The testbench checks three aspects of correctness:

(1) directed tests with known examples,
(2) boundary-value tests covering every operation and special 32-bit operand pairs, and
(3) 3,000 randomized tests against a software reference. It also requests waveform generation through **`TestUtil.waves`**.

## Testbench organization

`class ALUSpec extends AnyFlatSpec with ChiselScalatestTester`

`AnyFlatSpec` provides readable test descriptions; `ChiselScalatestTester` provides hardware simulation methods. The ALU is instantiated with `test(new ALU(CoreParams()))`.

- Signal: `io.a` Direction: Input Function: First 32-bit operand
- Signal: `io.b` Direction: Input Function: Second 32-bit operand
- Signal: `io.op` Direction: Input Function: ALU operation selection
- Signal: `io.y` Direction: Output Function: Computed 32-bit result

## Definition of the ALU operations

The `ops` sequence contains `tuples(UInt,String,(Long,Long)=>Long)`: an operation code, a descriptive name, and a Scala function computing the **expected result.**

`(ALUOp.ADD, "ADD", (a, b) => a + b)`

- Operation: ADD Reference calculation: `a + b` Meaning: Addition
- Operation: SUB Reference calculation: `a - b` Meaning: Subtraction
- Operation: SLL Reference calculation: `a << (b & 31)` Meaning: Logical left shift
- Operation: SLT Reference calculation: `if (s32(a) < s32(b)) 1 else 0` Meaning: Signed comparison
- Operation: SLTU Reference calculation: `if (a < b) 1 else 0` Meaning: Unsigned comparison
- Operation: XOR Reference calculation: `a ^ b` Meaning: Bitwise exclusive OR
- Operation: SRL Reference calculation: `a >>> (b & 31)` Meaning: Logical right shift
- Operation: SRA Reference calculation: `s32(a) >> (b & 31)` Meaning: Arithmetic right shift
- Operation: OR Reference calculation: `a | b` Meaning: Bitwise OR
- Operation: AND Reference calculation: `a & b` Meaning: Bitwise AND
- Operation: NOP Reference calculation: `0L` Meaning: No operation

`s32()` interprets a 32-bit value as signed. The expression `b & 31` retains the five low bits of the shift amount, so a shift by 32 behaves like a shift by zero in RV32I.

## First test: Basic functional verification

The first test, “pass the lab1 checks,” applies predefined inputs and verifies expected results using a local check() helper. poke() drives hardware inputs; expect() asserts the expected output.

```
def check(a: BigInt, b: BigInt, op: UInt, expected: BigInt): Unit = {
dut.io.a.poke(a.U); dut.io.b.poke(b.U); dut.io.op.poke(op)
dut.io.y.expect(expected.U)
}
```

- Operation: ADD Input A: `10` Input B: `3` Expected: `13`
- Operation: SUB Input A: `10` Input B: `3` Expected: `7`
- Operation: AND Input A: `10` Input B: `3` Expected: `2`
- Operation: OR Input A: `10` Input B: `3` Expected: `11`
- Operation: XOR Input A: `10` Input B: `3` Expected: `9`
- Operation: SLL Input A: `1` Input B: `4` Expected: `16`
- Operation: SRL Input A: `16` Input B: `2` Expected: `4`
- Operation: SLT Input A: `0xFFFFFFFF` Input B: `1` Expected: `1`
- Operation: SLTU Input A: `0xFFFFFFFF` Input B: `1` Expected: `0`
- Operation: SRA Input A: `0x80000000` Input B: `1` Expected: `0xC0000000`
- Operation: NOP Input A: `123` Input B: `456` Expected: `0`

Signed versus unsigned comparison is especially important: `0xFFFFFFFF` represents `-1` as a signed 32-bit integer, so SLT(`0xFFFFFFFF`, `1`) is 1; as an unsigned integer it represents 4,294,967,295, so SLTU returns `0`.

## Second test: Boundary-value verification

The second test executes every operation against every pair of values in edges, a collection defined in `TestUtil.scala`. The supplied code does not show its exact contents. Typical values include **zero**, **one**, `0xFFFFFFFF`, `0x7FFFFFFF`, and `0x80000000`.

```
for ((op, name, f) <- ops; a <- edges; b <- edges) {
        c.io.op.poke(op); c.io.a.poke(a.U(32.W)); c.io.b.poke(b.U(32.W))
        val got = c.io.y.peek().litValue.toLong
        assert(got == (f(a, b) & M), f"$name 0x$a%08x 0x$b%08x: got 0x$got%08x")
}
```

The software function `f(a,b)` computes the expected result; M is expected to be the 32-bit mask `0xFFFFFFFF`.

For example, (`0xFFFFFFFF + 1`) `& 0xFFFFFFFF = 0`, matching 32-bit wraparound. If edges contains **N** values, this test performs $11 \times N^2$ comparisons.

## Third test: Randomized verification

The third test uses new Random(1), a fixed seed that makes the random sequence reproducible. It performs 3,000 iterations, randomly choosing an ALU operation and two 32-bit operands (via rnd32(r)), then compares the hardware output with the software reference function.

```
val r = new Random(1)
for (_ <- 0 until 3000) {
```

```
        val (op, name, f) = ops(r.nextInt(ops.length))
        val a = rnd32(r); val b = rnd32(r)
        c.io.op.poke(op); c.io.a.poke(a.U(32.W)); c.io.b.poke(b.U(32.W))
        val got = c.io.y.peek().litValue.toLong
        assert(got == (f(a, b) & M), f"$name 0x$a%08x 0x$b%08x: got 0x$got%08x")
}
for (code <- 10 to 14) {
        c.io.op.poke(code.U); c.io.a.poke(5.U); c.io.b.poke(7.U)
        c.io.y.expect(0.U)
}
```

The final loop checks that operation codes 10 through 14 return zero for the selected operands. Their precise interpretation depends on **`ALUOp`** definitions not supplied here.

## Waveform generation

```
.withAnnotations(TestUtil.waves)
```

This passes simulation annotations from **`TestUtil.scala`**. The name suggests VCD waveform recording, but its exact configuration is not visible in the supplied code. A typical **`ChiselTest`** configuration uses **`WriteVcdAnnotation`**.

```
        .withAnnotations(Seq(WriteVcdAnnotation))
        gtkwave waveform.vcd
```

GTKWave can display **`io.a`**, **`io.b`**, **`io.op`**, and **`io.y`** as inputs change. Because this ALU is combinational, the result depends on current input values rather than a multicycle execution sequence. Waveforms are useful for inspecting shifts, signed comparisons, and arithmetic overflow.

## Summary of the verification strategy

| | |
|---|---|
| Test: Lab 1 checks Verification method: 11 directed cases | Objective: Basic ALU functionality |
| Test: Edge-value pairs Verification method: All operations × all operand pairs | Objective: Overflow, sign, and shift behavior |
| Test: Randomized tests Verification method: 3,000 random cases | Objective: Comparison against software reference |
| Test: Unused operation codes Verification method: 5 additional cases | Objective: Zero output for codes 10–14 |

ALUSpec.scala verifies the hardware implemented in ALU.scala; together with the waveform configuration in TestUtil.scala, it forms a reproducible testing and visual-analysis environment. The same methodology can be extended to instruction decoding, register access, multiplication, sequential division, and pipeline control.

## Complete project: `rv32i-stage1.zip`

```
$ unzip rv32i-stage1.zip
Archive:  rv32i-stage1.zip
replace rv32i-stage1/DESIGN_NOTES.md? [y]es, [n]o, [A]ll, [N]one, [r]ename: A
  inflating: rv32i-stage1/DESIGN_NOTES.md
  inflating: rv32i-stage1/generated/RV32IRCore.sv
  inflating: rv32i-stage1/generated/RV32ITestWrapper.sv
  inflating: rv32i-stage1/src/test/scala/rv32i/Disasm.scala
  inflating: rv32i-stage1/src/test/scala/rv32i/Asm.scala
  inflating: rv32i-stage1/src/test/scala/rv32i/TestUtil.scala
  inflating: rv32i-stage1/src/test/scala/rv32i/InstructionMemorySpec.scala
  inflating: rv32i-stage1/src/test/scala/rv32i/ALUSpec.scala
  inflating: rv32i-stage1/src/test/scala/rv32i/RegisterFileSpec.scala
  inflating: rv32i-stage1/src/test/scala/rv32i/CoreSim.scala
  inflating: rv32i-stage1/src/test/scala/rv32i/DecoderSpec.scala
  inflating: rv32i-stage1/src/test/scala/rv32i/WaveformSpec.scala
  inflating: rv32i-stage1/src/test/scala/rv32i/RV32IRCoreSpec.scala
  inflating: rv32i-stage1/src/test/scala/rv32i/Gtkw.scala
  inflating: rv32i-stage1/src/test/scala/rv32i/TimingSpec.scala
  inflating: rv32i-stage1/src/test/scala/rv32i/ImmediateGeneratorSpec.scala
  inflating: rv32i-stage1/src/main/scala/rv32i/InstructionMemory.scala
  inflating: rv32i-stage1/src/main/scala/rv32i/RegisterFile.scala
  inflating: rv32i-stage1/src/main/scala/rv32i/RefModel.scala
  inflating: rv32i-stage1/src/main/scala/rv32i/RV32IEncoding.scala
  inflating: rv32i-stage1/src/main/scala/rv32i/CoreParams.scala
  inflating: rv32i-stage1/src/main/scala/rv32i/ALU.scala
  inflating: rv32i-stage1/src/main/scala/rv32i/Emit.scala
  inflating: rv32i-stage1/src/main/scala/rv32i/Decoder.scala
  inflating: rv32i-stage1/src/main/scala/rv32i/ImmediateGenerator.scala
  inflating: rv32i-stage1/src/main/scala/rv32i/RV32IRCore.scala
  inflating: rv32i-stage1/src/main/scala/rv32i/Trace.scala
  inflating: rv32i-stage1/src/main/scala/rv32i/RV32ITestWrapper.scala
  inflating: rv32i-stage1/build.sbt
  inflating: rv32i-stage1/README.md
  inflating: rv32i-stage1/.gitignore
 extracting: rv32i-stage1/project/build.properties
  inflating: rv32i-stage1/tools/rv32im_filter.py
  inflating: rv32i-stage1/memory.hex
  inflating: rv32i-stage1/waves/lab1_rtype/program.lst
  inflating: rv32i-stage1/waves/lab1_rtype/aluop.filter
  inflating: rv32i-stage1/waves/lab1_rtype/core.gtkw
  inflating: rv32i-stage1/waves/lab1_rtype/inst.filter
  inflating: rv32i-stage1/waves/lab1_rtype/RV32ITestWrapper.vcd
  inflating: rv32i-stage1/waves/mixed/program.lst
  inflating: rv32i-stage1/waves/mixed/aluop.filter
  inflating: rv32i-stage1/waves/mixed/core.gtkw
  inflating: rv32i-stage1/waves/mixed/inst.filter
  inflating: rv32i-stage1/waves/mixed/RV32IRCore.vcd
  inflating: rv32i-stage1/waves/alu_imm/program.lst
  inflating: rv32i-stage1/waves/alu_imm/aluop.filter
  inflating: rv32i-stage1/waves/alu_imm/core.gtkw
  inflating: rv32i-stage1/waves/alu_imm/inst.filter
  inflating: rv32i-stage1/waves/alu_imm/RV32IRCore.vcd
--------------------------------------------------------------------------------
```

## Understanding the Files Generated by a Chisel-Based RISC-V Processor Project

The displayed output corresponds to the extraction of a ZIP archive containing a **Chisel-based RV32I RISC-V processor project**, named **`rv32i-stage1.`**
The project contains several categories of files: Chisel hardware descriptions, generated SystemVerilog, automated tests, build configurations, reference models, documentation, and simulation waveforms. Together, these files form a complete development environment for designing, generating, testing, and visually analyzing a RISC-V processor.

### Main File Types

The following table summarizes the file extensions appearing in the project.

| Extension | File type | Purpose |
|---|---|---|
| `.scala` | Scala / Chisel source | Hardware descriptions, test programs, and simulation utilities |
| `.sv` | SystemVerilog | Generated RTL hardware implementation |
| `.sbt` | Scala build configuration | Defines dependencies and project compilation |
| `.properties` | Configuration | Specifies the sbt version |
| `.md` | Markdown | Documentation and design notes |
| `.hex` | Hexadecimal data | Memory initialization |
| `.vcd` | Value Change Dump | Simulation waveforms |
| `.gtkw` | GTKWave configuration | Waveform display settings |
| `.lst` | Program listing | Human-readable instruction listing |
| `.filter` | GTKWave translation filter | Converts encoded signal values into readable names |
| `.py` | Python | Utility scripts |
| `.gitignore` | Git configuration | Excludes generated or temporary files from version control |

The most important distinction is between **hardware source files**, **generated RTL files**, and **simulation files**.

## Chisel Hardware Description Files

Directory:
**`src/main/scala/rv32i/`**
This directory contains the principal hardware modules written in Scala using Chisel.

### `ALU.scala`

Describes the Arithmetic Logic Unit.
The ALU performs arithmetic and logical operations, potentially including:

- **`ADD`**
- **`SUB`**
- **`AND`**
- **`OR`**
- **`XOR`**
- **`SLL`**
- **`SRL`**
- **`SRA`**
- **`SLT`**
- **`SLTU`**

The exact supported operations depend on the implementation.

### `RegisterFile.scala`

Describes the processor register file.
A conventional RV32I register file contains:

- 32 general-purpose registers.
- 32 bits per register.
- Two read ports.
- One write port.
- Register `x0`, permanently equal to zero.

### `Decoder.scala`

Implements instruction decoding.
The decoder examines fields of a RISC-V instruction, such as:

- **`opcode`**
- **`funct3`**
- **`funct7`**
- **`rs1`**
- **`rs2`**
- **`rd`**

It generates the control signals required to execute the instruction.

### `ImmediateGenerator.scala`

Extracts immediate values from instruction encodings.
For the complete RV32I instruction set, these include:

- **`I-type`**
- **`S-type`**
- **`B-type`**
- **`U-type`**
- **`J-type`**

The immediate generator reconstructs and sign-extends the immediate according to the instruction format.
InstructionMemory.scala
Describes the instruction memory interface or implementation.
Its role is to supply the instruction corresponding to the program counter.

### `RV32IRCore.scala`

This is the **main processor module**.
It integrates the hardware components into a complete RV32I processor.
Depending on the implementation, it manages instruction fetching, decoding, execution, register write-back, and program-counter updates.

### `RV32ITestWrapper.scala`

Provides a wrapper around the processor for simulation and testing.
A wrapper may expose internal signals, connect instruction memory, or simplify testbench interactions.

### `CoreParams.scala`

Contains processor configuration parameters.
These may include register widths, reset addresses, or other architectural constants.

### `RV32IEncoding.scala`

Defines instruction encodings and constants used by the processor.
For example, it may define opcodes or instruction-type identifiers.

### `Trace.scala`

Defines information used to observe processor execution.
Execution traces can include the program counter, instruction, destination register, and computed result.
Such information is useful for verification against a reference model.

### `Emit.scala`

Provides the entry point for generating Verilog or SystemVerilog from the Chisel hardware descriptions.
For example, the generation process may be launched with:

```
sbt "runMain rv32i.Emit"
```

The exact command depends on the definition of the `Emit` object.

### `RefModel.scala`

Contains a software reference model of the processor.
Unlike the Chisel hardware modules, this file is likely intended to model the expected architectural behavior.
The reference model can be used to compare actual processor execution against the RISC-V specification.
**Important:** These descriptions are based on the filenames and conventional Chisel project organization. The precise implementation details would require examining the source code.

## Generated SystemVerilog Files

Directory:
**`generated/`**
The archive contains:
**`generated/RV32IRCore.sv`**
**`generated/RV32ITestWrapper.sv`**

These files are the **generated RTL representations** of the Chisel hardware.
The generation process is:

```
        Chisel Source (.scala)
                |
                v
        Chisel Elaboration
                |
                v
        FIRRTL / CIRCT
                |
                v
        SystemVerilog (.sv)
```

### `RV32IRCore.sv`

Contains the generated SystemVerilog implementation of the processor core.
This file can be used with compatible RTL simulators and synthesis tools.

### `RV32ITestWrapper.sv`

Contains the generated SystemVerilog representation of the test wrapper.
It can simplify processor simulation by exposing useful signals or connecting the core to its test environment.

## Why generate SystemVerilog?

Chisel is a hardware generation language, whereas SystemVerilog is a standard hardware description language supported by many digital design tools. The generated files allow the processor to be simulated using tools such as Icarus Verilog or Verilator.
For example:

```
iverilog -g2012 \
    -s RV32ITestWrapper \
    -o processor.vvp \
    generated/RV32IRCore.sv \
   generated/RV32ITestWrapper.sv
```

This command compiles the RTL, but a separate simulation testbench or appropriate simulation harness is generally needed to provide clock, reset, and stimulus signals.

## Automated Test Files

Directory:
**`src/test/scala/rv32i/`**

This directory contains the automated verification environment.
The project includes several types of tests.

| File | Expected verification role |
|---|---|
| **`ALUSpec.scala`** | Tests arithmetic and logical operations |
| **`RegisterFileSpec.scala`** | Tests register reads, writes, and x0 |
| **`DecoderSpec.scala`** | Tests instruction decoding |
| **`ImmediateGeneratorSpec.scala`** | Tests immediate extraction |
| **`InstructionMemorySpec.scala`** | Tests instruction memory behavior |
| **`RV32IRCoreSpec.scala`** | Tests the integrated processor |
| **`TimingSpec.scala`** | Checks execution timing |
| **`WaveformSpec.scala`** | Generates or verifies simulation waveforms |

These tests are particularly important because processor correctness cannot be established simply by compiling the hardware. For example, an ALU may compile successfully but produce an incorrect result for signed comparisons or arithmetic right shifts. Similarly, a processor may execute arithmetic instructions correctly while incorrectly handling branches or register dependencies.

## Supporting test utilities

The archive also contains:
`Asm.scala`
Likely provides assembly instruction encoding utilities for constructing test programs.
`Disasm.scala`
Likely converts binary instruction words into readable RISC-V assembly mnemonics.
`TestUtil.scala`
Provides shared testing functions.
`CoreSim.scala`
Likely contains utilities for simulating the complete processor.
`Gtkw.scala`
Likely generates GTKWave configuration files or supports waveform visualization.
These utilities help automate the verification workflow and make test programs easier to develop.

## Running the tests

From the project directory:
```
cd rv32i-stage1
sbt test
```
This command compiles the Chisel project and executes the configured Scala tests.
To run a specific test:
```
sbt "testOnly rv32i.ALUSpec"
```
Or:
```
sbt "testOnly rv32i.RV32IRCoreSpec"
```
The actual test class names must match those declared in the source files.

## Simulation Waveform Files

Directory:
**`waves/`**

This directory is especially interesting because it contains **pre-generated simulation results**.
The archive contains three waveform subdirectories:

```
waves/
├── lab1_rtype/
├── mixed/
└── alu_imm/
```

These names suggest three different instruction test scenarios.
**`lab1_rtype`**

Likely contains waveforms for R-type register-register arithmetic instructions.
Examples include:
```
add x3, x1, x2
sub x4, x3, x1
and x5, x1, x2
or  x6, x1, x2
```

Likely contains tests for immediate arithmetic instructions.
Examples include:
```
addi x3, x1, 10
andi x4, x2, 15
ori  x5, x3, 7
```

## Files within each waveform directory

Each directory contains several files that work together.
**1. `RV32IRCore.vcd` or `RV32ITestWrapper.vcd`**

These are simulation waveform files.
VCD stands for **Value Change Dump**.
A VCD file records signal transitions over simulation time.
For example:

```
clock
reset
pc
instruction
rs1
rs2
alu_result
rd
write_enable
```

The exact recorded signals depend on the simulation configuration.
**2. core.gtkw**
This file stores GTKWave visualization settings.
It can specify which signals to display, their ordering, formatting, and grouping.
**3. inst.filter**
Likely provides a translation filter for displaying instruction encodings as readable assembly instructions.
For example, a hexadecimal instruction value could be displayed as:
**add x3, x1, x2**
**4. aluop.filter**
Likely translates numeric ALU operation codes into readable operation names.
For example:
**0 → ADD**
**1 → SUB**
**2 → AND**
The actual mapping depends on the implementation.
**5. program.lst**
Contains a human-readable representation of the test program.
This file helps associate waveform events with the executed instructions.

## Opening the waveforms

For example:

```
cd rv32i-stage1
gtkwave \
    waves/lab1_rtype/RV32ITestWrapper.vcd \
    waves/lab1_rtype/core.gtkw
```

For the immediate arithmetic test:

```
gtkwave \
    waves/alu_imm/RV32IRCore.vcd \
    waves/alu_imm/core.gtkw
```

For the mixed instruction test:

```
gtkwave \
    waves/mixed/RV32IRCore.vcd \
    waves/mixed/core.gtkw
```

GTKWave displays the signal values over successive clock cycles.
This allows the designer to visually inspect the relationship between instruction execution and hardware behavior.
**Note:** The `.gtkw` configuration may reference translation filters using relative paths. If the filters do not load correctly, their paths may need adjustment.

## Project Configuration and Documentation

Several files support the development environment rather than describing hardware directly.
**build.sbt**
This is the main Scala build configuration file.
It specifies dependencies such as Chisel, ScalaTest, and ChiselTest, along with compiler settings.
The file is interpreted by **sbt**.
project/build.properties
Defines the sbt version used by the project.
For example:
**sbt.version=1.9.7**
The actual version must be read from the project's file.
**README.md**
Usually contains installation instructions, project organization, and commands for compiling, testing, and generating hardware.
**This is generally the first file to read when starting with an unfamiliar project.**
**DESIGN_NOTES.md**
Contains design decisions, architectural explanations, and implementation details.
For an educational RISC-V project, it may describe the supported instruction set, processor organization, execution timing, and verification strategy.
**memory.hex**
Contains hexadecimal data used to initialize a memory.
In a processor project, this file commonly contains machine-code instructions.

For example:

```
00500093
00A00113
002081B3
```

These values can represent RISC-V instructions loaded into instruction memory.

**`tools/rv32im_filter.py`**

This Python script is likely a utility for processing RISC-V instruction data or simulation output.
Its precise purpose cannot be determined from the filename alone.

**`.gitignore`**

Specifies files that Git should ignore, such as temporary build directories, compiled classes, and generated simulation artifacts.

## Overall Organization of the Project

The complete project can be summarized as follows:

```
rv32i-stage1/
├── src/
│   │
│   ├── main/scala/rv32i/
│   │   │
│   │   ├── ALU.scala
│   │   ├── Decoder.scala
│   │   ├── RegisterFile.scala
│   │   ├── ImmediateGenerator.scala
│   │   ├── InstructionMemory.scala
│   │   ├── RV32IRCore.scala
│   │   └── ...
│   │
│   └── test/scala/rv32i/
│       │
│       ├── ALUSpec.scala
│       ├── DecoderSpec.scala
│       ├── RV32IRCoreSpec.scala
│       ├── WaveformSpec.scala
│       └── ...
├── generated/
│   ├── RV32IRCore.sv
│   └── RV32ITestWrapper.sv
├── waves/
│   ├── lab1_rtype/
│   ├── alu_imm/
│   └── mixed/
├── tools/
│   └── rv32im_filter.py
├── memory.hex
├── build.sbt
├── README.md
└── DESIGN_NOTES.md
```

The different directories correspond to successive stages of the hardware development workflow:

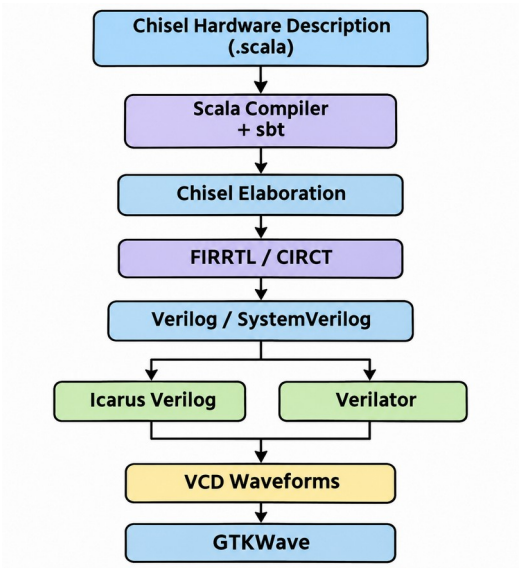



**Fig.1.1:** Development workflow

In practice, Chisel tests can also run directly on the Chisel design through a simulation backend, so generated SystemVerilog is not necessarily a prerequisite for executing the Scala tests.

## Conclusion

The extracted archive contains the main elements required for developing and verifying a Chisel-based RISC-V processor.
Three file categories are particularly important:

**1. `.scala` Hardware design and testing**

These files contain the Chisel hardware descriptions and the Scala verification environment. They are the principal source files of the project.

**2. `.sv` Generated hardware**

These files contain the SystemVerilog RTL produced from Chisel. They can be used with conventional simulation and synthesis tools.

**3. `.vcd and .gtkw` Visual verification**
These files record and display the dynamic behavior of the processor, allowing instruction execution and internal signal activity to be examined cycle by cycle.
The remaining files provide documentation, configuration, memory initialization, instruction listings, and supporting utilities.
**The important characteristic of this project is that it combines hardware generation, automated verification, and waveform visualization in a single development environment.**
This organization is particularly valuable for AI-assisted RISC-V development because each architectural stage can be delivered as a complete, reproducible project containing the processor source code, generated RTL, automated tests, and GTKWave simulation results.

### Note:

We will find the same types of the generated design files in all following Glabs starting from this simple case RV32I (R-type only) to quite complex RV32IM with 5-stage pipeline and cache memories.

## 2. Running the project

We start by execution of global test with

```
$ sbt test
[info] welcome to sbt 1.10.2 (Ubuntu Java 17.0.20.1)
[info] loading project definition from /media/bako/0969-2ABE/books_2026/CL.Book.Chisel.2026/RV32.Steps/rv32i-
stage1/project
[info] loading settings for project root from build.sbt ...
[info] set current project to rv32i-stage1 (in build
file:/media/bako/0969-2ABE/books_2026/CL.Book.Chisel.2026/RV32.Steps/rv32i-stage1/)
[info] compiling 1 Scala source to /media/bako/0969-2ABE/books_2026/CL.Book.Chisel.2026/RV32.Steps/rv32i-stage1/
target/scala-2.13/classes ...
[info] compiling 1 Scala source to /media/bako/0969-2ABE/books_2026/CL.Book.Chisel.2026/RV32.Steps/rv32i-stage1/
target/scala-2.13/test-classes ...
[info] DecoderSpec:
[info] Decoder
[info] - should decode the 10 R-type instructions (lab1 cases)
[info] - should decode the 9 I-type ALU instructions (lab1 cases)
[info] - should reject every other encoding: valid = 0, aluOp = NOP, no register write
[info] - should accept only the OP and OP-IMM major opcodes among all 128
[info] WaveformSpec:
[info] Waveforms
[info] - should show the lab1 program from memory.hex in RV32ITestWrapper (waves/lab1_rtype)
[info] - should show the I-type ALU instructions and constants built with ALU instructions (waves/alu_imm)
[info] - should show R-type operations, a dependency chain and unsupported words executed as no-ops (waves/mixed)
[info] TimingSpec:
[info] RV32IRCore
[info] - should execute one instruction per cycle, the 19 ALU instructions and any other word alike
[info] RegisterFileSpec:
[info] RegisterFile
[info] - should read, write and protect x0 (lab1 cases)
[info] - should start from the initial values, keep x0 at 0 even if a value is given, and show all registers on
debugRead
[info] - should write all 31 registers and return the old value during a write
[info] VerilogVectorSpec:
[info] Vectors for tb_ALU
[info] - should match the ALU (every operation on edge values and random operands)
[info] Vectors for tb_ImmediateGenerator
[info] - should match the immediate generator
[info] Vectors for tb_Decoder
[info] - should match the decoder (valid, invalid and random words)
[info] Vectors for tb_RegisterFile
[info] - should match the register file (reset values x1 = 10, x2 = 3, writes, x0)
[info] Vectors for tb_InstructionMemory
[info] - should match the instruction memory built from memory.hex
[info] Vectors for tb_RV32IRCore
[info] - should match the core on 1,000 random instructions (instruction memory in the testbench)
[info] Vectors for tb_RV32ITestWrapper
[info] - should match the wrapper running memory.hex
[info] InstructionMemorySpec:
[info] InstructionMemory
[info] - should load a hex file with comments and separators, and read 0 past its end
[info] - should read the project's memory.hex (lab1 program)
[info] ALUSpec:
[info] ALU
[info] - should pass the lab1 checks
[info] - should compute every operation on all edge-value pairs (overflow, shift by 31/32, sign)
[info] - should match the reference on random operands and give 0 for unused operation codes
[info] RV32IRCoreSpec:
[info] RV32ITestWrapper
[info] - should run the lab1 memory.hex program (10 R-type instructions, x1 = 10, x2 = 3)
[info] RV32IRCore
[info] - should execute all 10 R-type instructions on edge values
[info] - should execute all 9 I-type ALU instructions, including negative immediates and shift amounts 0 and 31
[info] - should load any 32-bit constant with ALU instructions only (li)
[info] - should use every result in the next instruction (single cycle: no hazards) and never write x0
[info] - should execute unsupported instructions as no-ops (no register change, pc + 4)
[info] - should start from preset registers (initialRegs) like lab1
[info] - should match the reference model on 3000 random ALU instructions
[info] ImmediateGeneratorSpec:
[info] ImmediateGenerator
```

```
[info] - should sign-extend every 12-bit I-type immediate (lab1 values and all 4096 values)
[info] Run completed in 18 seconds, 175 milliseconds.
[info] Total number of tests run: 32
[info] Suites: completed 9, aborted 0
[info] Tests: succeeded 32, failed 0, canceled 0, ignored 0, pending 0
[info] All tests passed.
[success] Total time: 26 s, completed 10 nov. 2026, 15:26:45
```

Now let us run **gtkwave**:

```
$ gtkwave RV32ITestWrapper.vcd core.gtkw
```

```
GTKWave Analyzer v3.3.116 (w)1999-2023 BSI

[0] start time.
[126] end time.
```

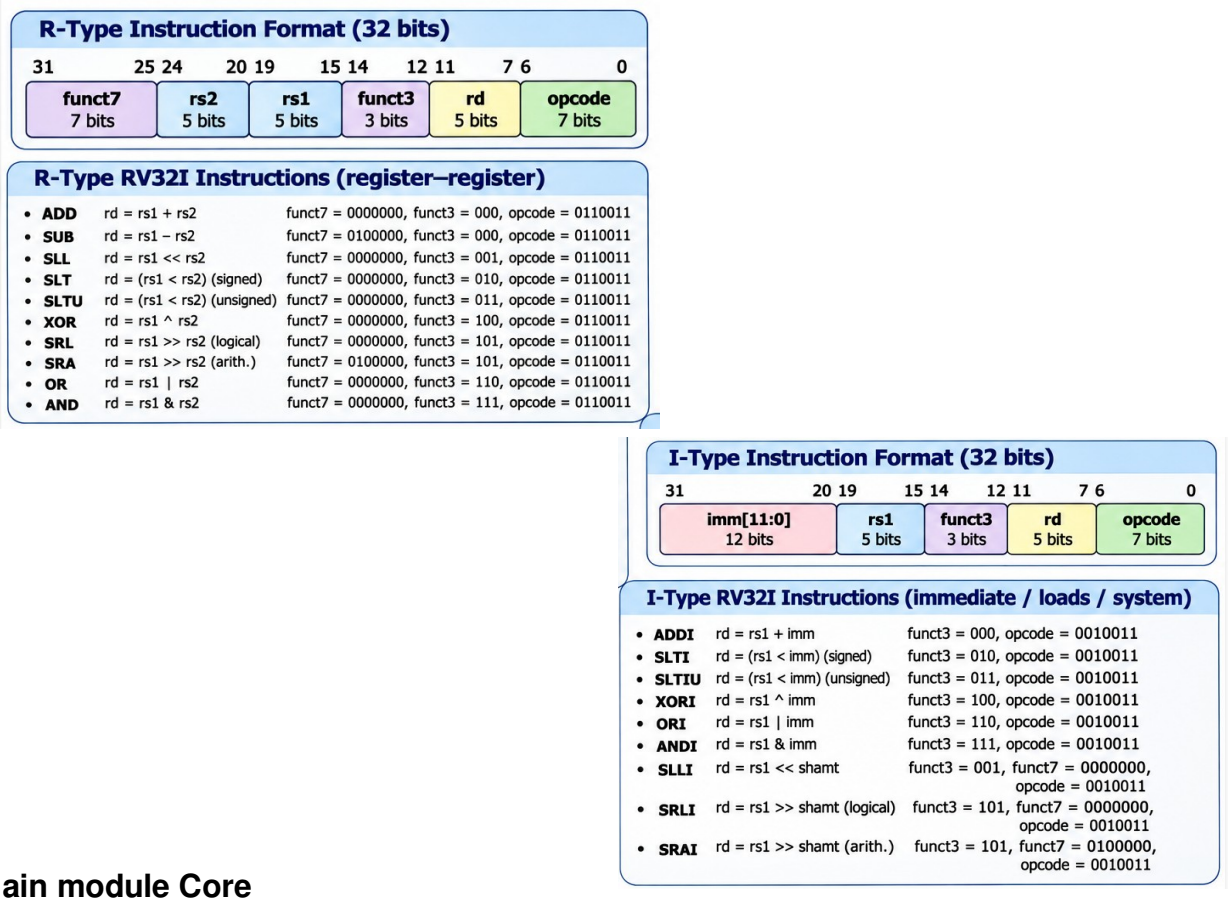



**Fig.1.2:** Gtkwave diagram (partial) for rv32i (R-type) instructions

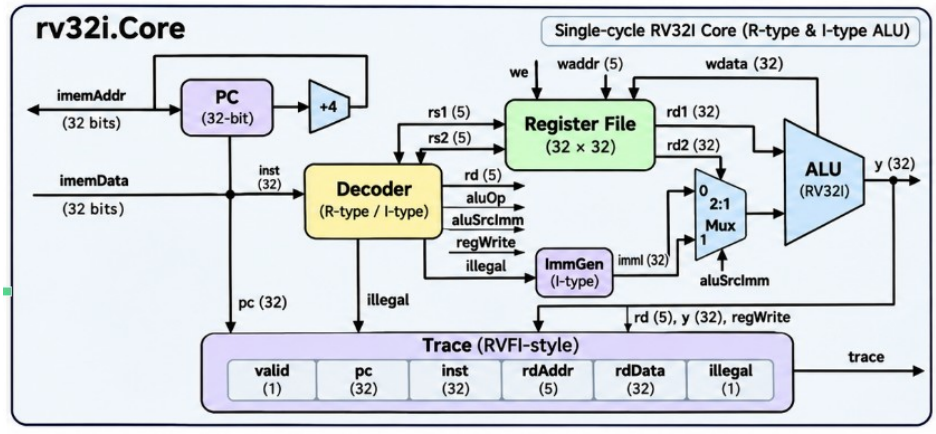
# GLab 2

## Building simple RV32I (R and I types) RISC-V architecture

Ther following figure illustartes the RV32I instruction formats relative to R and I type  instructions.




## Main module Core



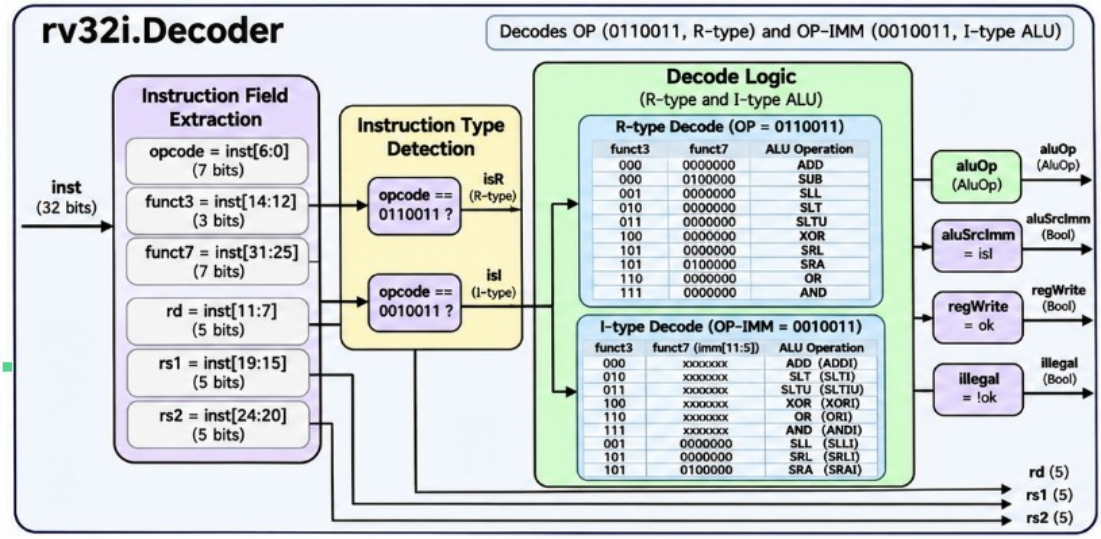

```
--------------------------------------------------------------------------------
package rv32i
import chisel3._
import chisel3.util._

/** Stage 2: single-cycle RV32I core for the R-type and I-type instructions, one instruction per clock cycle.
  *
  *  - R-type: ADD SUB SLL SLT SLTU XOR SRL SRA OR AND         (ALU b = rs2)
  *  - I-type: ADDI SLTI SLTIU XORI ORI ANDI SLLI SRLI SRAI    (ALU b = immediate)
  *  - I-type: LB LH LW LBU LHU                                (address rs1 + imm, data from the data memory)
  *  - I-type: JALR                                            (pc = (rs1 + imm) & ~1, rd = pc + 4)
  *
  * There are no exceptions in this stage. An instruction that cannot complete is executed as a no-op
```

```
  * (no register write, pc + 4, trace.valid = 0): an unsupported encoding, a misaligned load (LH/LHU at an odd
  * address, LW not at a multiple of 4), or a JALR whose target is not 4-byte aligned.
  * Memories are Harvard and combinational: imem for instructions, dmem (read-only here) for loads.
  */
class RV32IRICore(
  p: CoreParams = CoreParams(),
  initialRegs: Seq[BigInt] = Seq.empty,
  resetPc: BigInt = 0
) extends Module {
  require(p.xlen == 32, "RV32IRICore implements RV32I: xlen must be 32")
  require(p.numRegs == 32, "RV32IRICore implements RV32I: 32 registers")
  val io = IO(new Bundle {
    // Instruction-memory interface (combinational read).
    val imemAddr  = Output(UInt(p.xlen.W))
    val imemInst  = Input(UInt(32.W))
    // Data-memory interface (combinational read of the aligned word at dmemAddr & ~3).
    val dmemAddr  = Output(UInt(p.xlen.W))
    val dmemRe    = Output(Bool())
    val dmemRdata = Input(UInt(32.W))
    // Debug outputs for laboratory verification.
    val debugPC = Output(UInt(p.xlen.W))
    val debugX1 = Output(UInt(p.xlen.W))
    val debugX2 = Output(UInt(p.xlen.W))
    val debugX3 = Output(UInt(p.xlen.W))
    val debugX4 = Output(UInt(p.xlen.W))
    // What was executed this cycle.
    val trace = Output(new Trace(p))
  })
  // ---------------------------------------------------------------- program counter
  val pc = RegInit(resetPc.U(p.xlen.W))
  io.imemAddr := pc
  io.debugPC  := pc
  val pcPlus4 = pc + 4.U
  // ---------------------------------------------------------------- decode
  val decoder = Module(new Decoder)
  decoder.io.instr := io.imemInst
  val immGen = Module(new ImmediateGenerator(p))
  immGen.io.instr := io.imemInst
  // ---------------------------------------------------------------- register file
  val regFile = Module(new RegisterFile(p, initialRegs))
  regFile.io.rs1 := decoder.io.rs1
  regFile.io.rs2 := decoder.io.rs2
  // ---------------------------------------------------------------- ALU (also computes load addresses and JALR
targets)
  val alu = Module(new ALU(p))
  alu.io.a  := regFile.io.readData1
  alu.io.b  := Mux(decoder.io.aluSrc, immGen.io.imm, regFile.io.readData2)
  alu.io.op := decoder.io.aluOp
  val aluY = alu.io.y
  // ---------------------------------------------------------------- loads
  val size      = decoder.io.memSize
  val loadMis   = decoder.io.memRead && ((size === 1.U && aluY(0)) || (size === 2.U && aluY(1, 0) =/= 0.U))
  val doLoad    = decoder.io.memRead && !loadMis
  io.dmemAddr  := aluY
  io.dmemRe    := doLoad
  val rdata = io.dmemRdata
  val byte  = VecInit(Seq.tabulate(4)(k => rdata(8 * k + 7, 8 * k)))(aluY(1, 0))
  val half  = Mux(aluY(1), rdata(31, 16), rdata(15, 0))
  val loadData = WireDefault(rdata)
  switch(size) {
    is(0.U) { loadData := Mux(decoder.io.memUnsigned, Cat(0.U(24.W), byte), Cat(Fill(24, byte(7)), byte)) }
    is(1.U) { loadData := Mux(decoder.io.memUnsigned, Cat(0.U(16.W), half), Cat(Fill(16, half(15)), half)) }
  }
  // ---------------------------------------------------------------- JALR
  val jalrTarget = Cat(aluY(31, 1), 0.U(1.W))
  val jalrMis    = decoder.io.jalr && jalrTarget(1)          // target not 4-byte aligned
  val doJalr     = decoder.io.jalr && !jalrMis
  // ---------------------------------------------------------------- write-back
  val executed = decoder.io.valid && !loadMis && !jalrMis
  val wbData = MuxLookup(decoder.io.wbSel, aluY)(Seq(
    WbSel.ALU -> aluY,
    WbSel.MEM -> loadData,
    WbSel.PC4 -> pcPlus4
  ))
  regFile.io.rd          := decoder.io.rd
  regFile.io.writeData   := wbData
  regFile.io.writeEnable := decoder.io.regWrite && executed
  // ---------------------------------------------------------------- next pc
  val nextPc = Mux(doJalr, jalrTarget, pcPlus4)
  pc := nextPc
  // ---------------------------------------------------------------- debug and trace
  io.debugX1 := regFile.io.debugRead(1)
  io.debugX2 := regFile.io.debugRead(2)
  io.debugX3 := regFile.io.debugRead(3)
  io.debugX4 := regFile.io.debugRead(4)
  val writes = regFile.io.writeEnable && decoder.io.rd =/= 0.U
  io.trace.pc      := pc
  io.trace.pcNext  := nextPc
  io.trace.inst    := io.imemInst
  io.trace.valid   := executed
```

```
  io.trace.rdAddr  := Mux(writes, decoder.io.rd, 0.U)
  io.trace.rdData  := Mux(writes, wbData, 0.U)
  io.trace.memRe   := doLoad
  io.trace.memAddr := aluY
  assert(!(regFile.io.writeEnable && !executed), "register write by an instruction that was not executed")
  assert(pc(1, 0) === 0.U, "pc must stay 4-byte aligned")
}
```

---




---

```
package rv32i
import chisel3._
import chisel3.util._

/** Write-back source. */
object WbSel {
  val width = 2
  val ALU = 0.U(width.W)
  val MEM = 1.U(width.W)   // load data
  val PC4 = 2.U(width.W)   // return address (JALR)
}

/** Decoder for the Stage 2 instructions (R-type and I-type, 25 in total):
  *  - R-type   (0110011): ADD SUB SLL SLT SLTU XOR SRL SRA OR AND
  *  - I-type   (0010011): ADDI SLTI SLTIU XORI ORI ANDI SLLI SRLI SRAI
  *  - I-type   (0000011): LB LH LW LBU LHU                          (address = rs1 + imm)
  *  - I-type   (1100111): JALR                                       (target = (rs1 + imm) & ~1, rd = pc + 4)
  * `valid` is high only for these encodings; anything else gives valid = 0, aluOp = NOP and no side effect.
  */
class Decoder extends Module {
  val io = IO(new Bundle {
    val instr       = Input(UInt(32.W))
    val rs1         = Output(UInt(5.W))
    val rs2         = Output(UInt(5.W))
    val rd          = Output(UInt(5.W))
    val aluOp       = Output(UInt(ALUOp.width.W))
    val aluSrc      = Output(Bool())        // 0: ALU b = rs2, 1: ALU b = immediate
    val memRead     = Output(Bool())        // load
    val memSize     = Output(UInt(2.W))     // 0 byte, 1 half, 2 word
    val memUnsigned = Output(Bool())        // LBU, LHU
    val jalr        = Output(Bool())
    val wbSel       = Output(UInt(WbSel.width.W))
    val regWrite    = Output(Bool())
    val valid       = Output(Bool())
  })
  val opcode = io.instr(6, 0)
  val funct3 = io.instr(14, 12)
  val funct7 = io.instr(31, 25)
  io.rs1 := io.instr(19, 15)
  io.rs2 := io.instr(24, 20)
  io.rd  := io.instr(11, 7)
  io.aluOp       := ALUOp.NOP
  io.aluSrc      := false.B
  io.memRead     := false.B
  io.memSize     := funct3(1, 0)
  io.memUnsigned := funct3(2)
  io.jalr        := false.B
  io.wbSel       := WbSel.ALU
  io.valid       := false.B
  // R-type: funct7 ## funct3 selects the operation
  when(opcode === "b0110011".U) {
    io.aluOp := MuxLookup(Cat(funct7, funct3), ALUOp.NOP)(Seq(
      "b0000000000".U -> ALUOp.ADD,
      "b0100000000".U -> ALUOp.SUB,
      "b0000000001".U -> ALUOp.SLL,
      "b0000000010".U -> ALUOp.SLT,
```

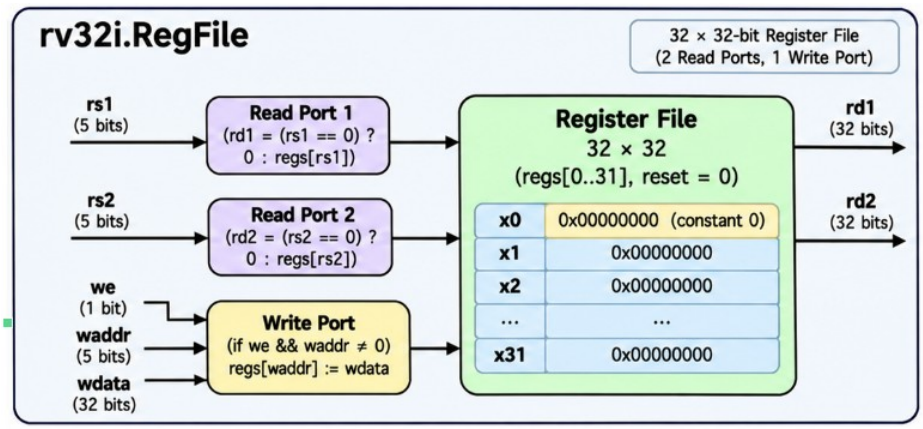
```
      "b0000000011".U -> ALUOp.SLTU,
      "b0000000100".U -> ALUOp.XOR,
      "b0000000101".U -> ALUOp.SRL,
      "b0100000101".U -> ALUOp.SRA,
      "b0000000110".U -> ALUOp.OR,
      "b0000000111".U -> ALUOp.AND
    ))
    io.valid := io.aluOp =/= ALUOp.NOP
  }
  // I-type ALU: the second operand is the immediate
  when(opcode === "b0010011".U) {
    io.aluSrc := true.B
    io.aluOp := MuxLookup(funct3, ALUOp.NOP)(Seq(
      "b000".U -> ALUOp.ADD,                                                    // ADDI
      "b010".U -> ALUOp.SLT,                                                    // SLTI
      "b011".U -> ALUOp.SLTU,                                                   // SLTIU
      "b100".U -> ALUOp.XOR,                                                    // XORI
      "b110".U -> ALUOp.OR,                                                     // ORI
      "b111".U -> ALUOp.AND,                                                    // ANDI
      "b001".U -> Mux(funct7 === "b0000000".U, ALUOp.SLL, ALUOp.NOP),           // SLLI
      "b101".U -> Mux(funct7 === "b0000000".U, ALUOp.SRL,                       // SRLI
                  Mux(funct7 === "b0100000".U, ALUOp.SRA, ALUOp.NOP))           // SRAI
    ))
    io.valid := io.aluOp =/= ALUOp.NOP
  }
  // I-type loads: the ALU computes the address rs1 + imm
  when(opcode === "b0000011".U) {
    val ok = funct3 === 0.U || funct3 === 1.U || funct3 === 2.U || funct3 === 4.U || funct3 === 5.U
    when(ok) {
      io.aluOp   := ALUOp.ADD
      io.aluSrc  := true.B
      io.memRead := true.B
      io.wbSel   := WbSel.MEM
      io.valid   := true.B
    }
  }
  // I-type JALR: the ALU computes rs1 + imm, rd gets pc + 4
  when(opcode === "b1100111".U && funct3 === 0.U) {
    io.aluOp  := ALUOp.ADD
    io.aluSrc := true.B
    io.jalr   := true.B
    io.wbSel  := WbSel.PC4
    io.valid  := true.B
  }
  io.regWrite := io.valid
}
```

---

---

```
package rv32i
import chisel3._
/** I-type immediate (instr[31:20]) sign-extended to xlen. For SLLI/SRLI/SRAI the ALU uses only its low bits. */
class ImmediateGenerator(p: CoreParams) extends Module {
  val io = IO(new Bundle {
    val instr = Input(UInt(32.W))
    val imm   = Output(UInt(p.xlen.W))
  })
  io.imm := io.instr(31, 20).asSInt.pad(p.xlen).asUInt
}
```

---

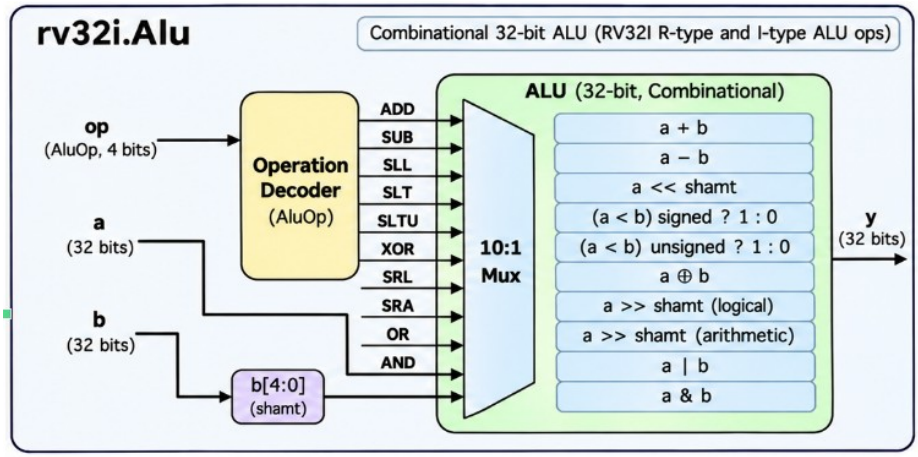



---

```
package rv32i
import chisel3._
import chisel3.util._
```

```
/** Register file: two combinational read ports, one write port, x0 hardwired to zero, optional reset values,
  * and every register visible on `debugRead` for laboratory verification.
  */
class RegisterFile(
  p: CoreParams,
  initialValues: Seq[BigInt] = Seq.empty
) extends Module {
  require(initialValues.isEmpty || initialValues.length == p.numRegs,
    "initialValues must be empty or contain exactly numRegs entries")
  val io = IO(new Bundle {
    val rs1         = Input(UInt(log2Ceil(p.numRegs).W))
    val rs2         = Input(UInt(log2Ceil(p.numRegs).W))
    val rd          = Input(UInt(log2Ceil(p.numRegs).W))
    val writeData   = Input(UInt(p.xlen.W))
    val writeEnable = Input(Bool())
    val readData1   = Output(UInt(p.xlen.W))
    val readData2   = Output(UInt(p.xlen.W))
    val debugRead   = Output(Vec(p.numRegs, UInt(p.xlen.W)))
  })
  val resetValues =
    if (initialValues.nonEmpty) initialValues.zipWithIndex.map { case (v, i) => (if (i == 0) BigInt(0) else
v).U(p.xlen.W) }
    else Seq.fill(p.numRegs)(0.U(p.xlen.W))
  val regs = RegInit(VecInit(resetValues))
  io.readData1 := Mux(io.rs1 === 0.U, 0.U, regs(io.rs1))
  io.readData2 := Mux(io.rs2 === 0.U, 0.U, regs(io.rs2))
  when(io.writeEnable && io.rd =/= 0.U) {
    regs(io.rd) := io.writeData
  }
  for (i <- 0 until p.numRegs) {
    io.debugRead(i) := (if (i == 0) 0.U else regs(i))
  }
}
```

---




---

```
$ cat ALU.scala
package rv32i

import chisel3._
import chisel3.util._

/** ALU operation codes (UInt constants, as in lab1). */
object ALUOp {
  val width = 4
  val ADD  = 0.U(width.W)
  val SUB  = 1.U(width.W)
  val SLL  = 2.U(width.W)
  val SLT  = 3.U(width.W)
  val SLTU = 4.U(width.W)
  val XOR  = 5.U(width.W)
  val SRL  = 6.U(width.W)
  val SRA  = 7.U(width.W)
  val OR   = 8.U(width.W)
  val AND  = 9.U(width.W)
  val NOP  = 15.U(width.W)   // result 0 (used for instructions the decoder does not recognise)
}

/** Combinational ALU. Shifts use the low log2(xlen) bits of `b`. */
class ALU(p: CoreParams) extends Module {
  val io = IO(new Bundle {
    val a  = Input(UInt(p.xlen.W))
    val b  = Input(UInt(p.xlen.W))
    val op = Input(UInt(ALUOp.width.W))
    val y  = Output(UInt(p.xlen.W))
  })

  val shamt = io.b(log2Ceil(p.xlen) - 1, 0)
```

```
  io.y := MuxLookup(io.op, 0.U(p.xlen.W))(Seq(
    ALUOp.ADD  -> (io.a + io.b),
    ALUOp.SUB  -> (io.a - io.b),
    ALUOp.SLL  -> (io.a << shamt)(p.xlen - 1, 0),
    ALUOp.SLT  -> (io.a.asSInt < io.b.asSInt).asUInt,
    ALUOp.SLTU -> (io.a < io.b).asUInt,
    ALUOp.XOR  -> (io.a ^ io.b),
    ALUOp.SRL  -> (io.a >> shamt),
    ALUOp.SRA  -> (io.a.asSInt >> shamt).asUInt,
    ALUOp.OR   -> (io.a | io.b),
    ALUOp.AND  -> (io.a & io.b)
  ))
}
```

---

All files in `main` branch

---

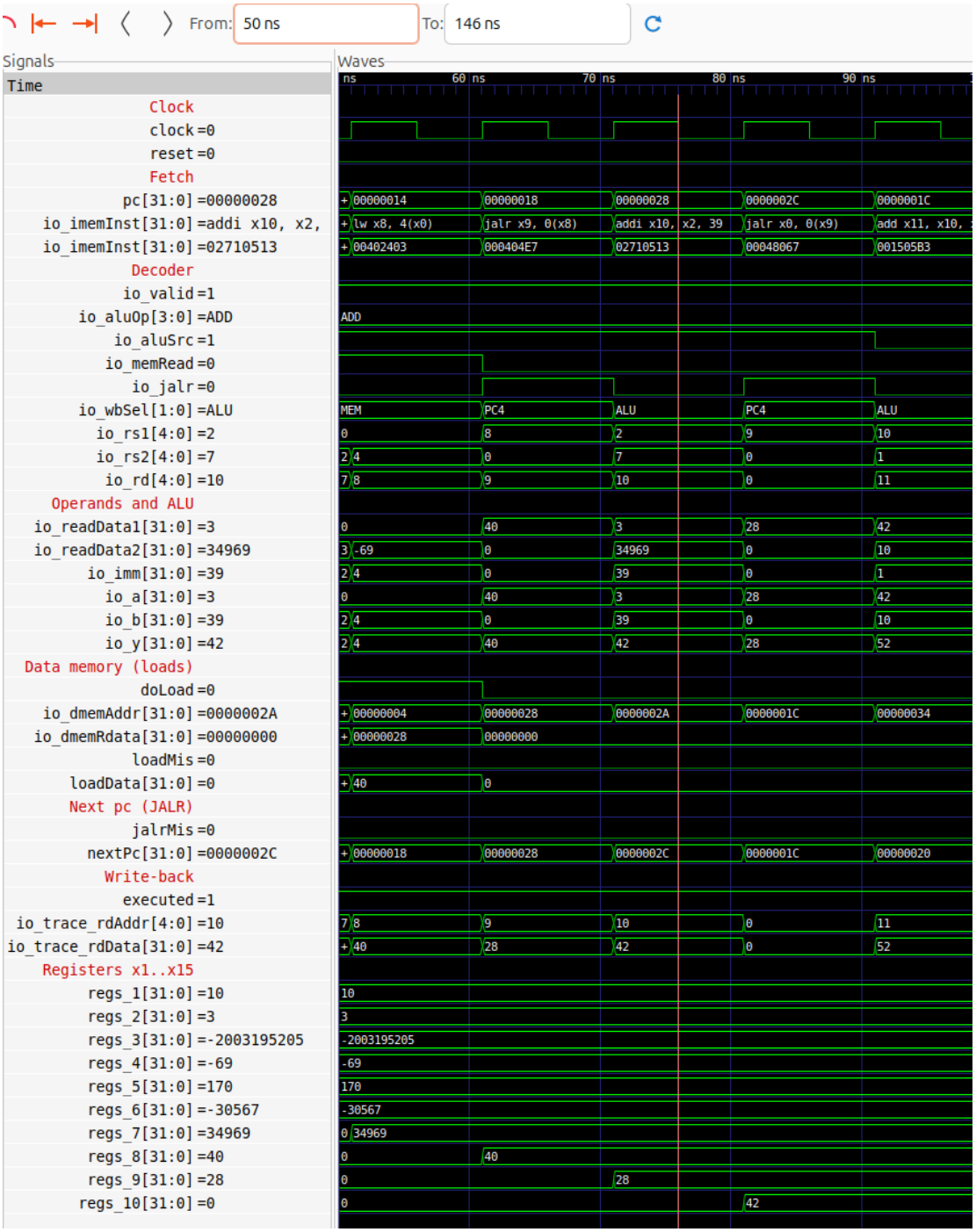
```
ALU.scala
DataMemory.scala
Emit.scala
InstructionMemory.scala
RegisterFile.scala
RV32IRICore.scala
Trace.scala
CoreParams.scala
Decoder.scala
ImmediateGenerator.scala
RefModel.scala
RV32IEncoding.scala
RV32ITestWrapper.scala
```

---

## $ sbt test

```
[info] welcome to sbt 1.10.2 (Ubuntu Java 17.0.20.1)
[info] loading project definition from /media/bako/0969-2ABE/books_2026/CL.Book.Chisel.2026/RV32.Steps/rv32i-
stage2/project
[info] loading settings for project root from build.sbt ...
[info] set current project to rv32i-stage2 (in build
file:/media/bako/0969-2ABE/books_2026/CL.Book.Chisel.2026/RV32.Steps/rv32i-stage2/)
[info] DecoderSpec:
[info] Decoder
[info] - should decode the 10 R-type instructions (lab1 cases)
[info] - should decode the 9 I-type ALU instructions (lab1 cases)
[info] - should decode the 5 loads and JALR (Stage 2)
[info] - should give the ALU instructions write-back from the ALU, no load and no jump
[info] - should reject every other encoding: valid = 0, aluOp = NOP, no register write, no load, no jump
[info] - should accept only the OP, OP-IMM, LOAD and JALR major opcodes among all 128
[info] WaveformSpec:
[info] Waveforms
[info] - should show the Stage 2 demo (program.hex + data.hex) in RV32ITestWrapper (waves/stage2_demo)
[info] - should show the lab1 R-type program (memory.hex) on the Stage 2 wrapper (waves/lab1_rtype)
[info] - should show every load size and sign, and a misaligned load (waves/loads)
[info] - should show JALR calls, returns and a jump (waves/jalr_call)
[info] TimingSpec:
[info] RV32IRICore
[info] - should execute one instruction per cycle: ALU, loads, JALR and unsupported words alike
[info] RegisterFileSpec:
[info] RegisterFile
[info] - should read, write and protect x0 (lab1 cases)
[info] - should start from the initial values, keep x0 at 0 even if a value is given, and show all registers on
debugRead
[info] - should write all 31 registers and return the old value during a write
[info] InstructionMemorySpec:
[info] InstructionMemory
[info] - should load a hex file with comments and separators, and read 0 past its end
[info] - should read the project's memory.hex (lab1 program)
[info] RV32IRICoreSpec:
[info] RV32ITestWrapper
[info] - should run the Stage 2 demo (program.hex + data.hex): loads of every size, a JALR call and return
[info] - should still run the lab1 memory.hex program (R-type, no data file)
[info] RV32IRICore
[info] - should execute all 10 R-type instructions on edge values
[info] - should execute all 9 I-type ALU instructions
[info] - should execute LB, LBU, LH, LHU and LW at every aligned offset, with sign and zero extension
[info] - should use a loaded value in the next instruction and as the next load's base address
[info] - should execute misaligned loads as no-ops (no exceptions in Stage 2)
[info] - should call and return with JALR, write the link register, clear bit 0, and skip over code
[info] - should execute a JALR to a misaligned target (bit 1 set) as a no-op: no link write, pc + 4
[info] - should execute unsupported instructions as no-ops (stores, branches, LUI, system, M extension, ...)
[info] - should start from preset registers (initialRegs) like lab1
[info] - should match the reference model on 3000 random instructions, including loads
[info] ALUSpec:
[info] ALU
[info] - should pass the lab1 checks
[info] - should compute every operation on all edge-value pairs (overflow, shift by 31/32, sign)
[info] - should match the reference on random operands and give 0 for unused operation codes
[info] DataMemorySpec:
[info] DataMemory
[info] - should return the aligned word for any byte address, and 0 past its end or with no file
[info] ImmediateGeneratorSpec:
[info] ImmediateGenerator
```

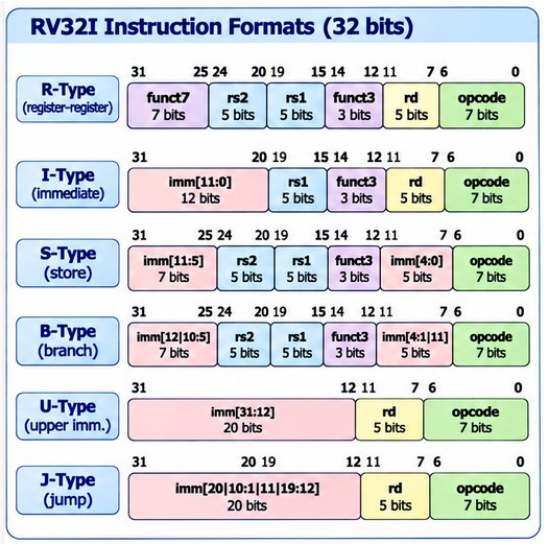
```
[info] - should sign-extend every 12-bit I-type immediate (lab1 values and all 4096 values)
[info] Run completed in 19 seconds, 798 milliseconds.
[info] Total number of tests run: 33
[info] Suites: completed 9, aborted 0
[info] Tests: succeeded 33, failed 0, canceled 0, ignored 0, pending 0
[info] All tests passed.
[success] Total time: 23 s, completed 10 nov. 2026, 15:59:54
------------------------------------------------------------------------------------------
```



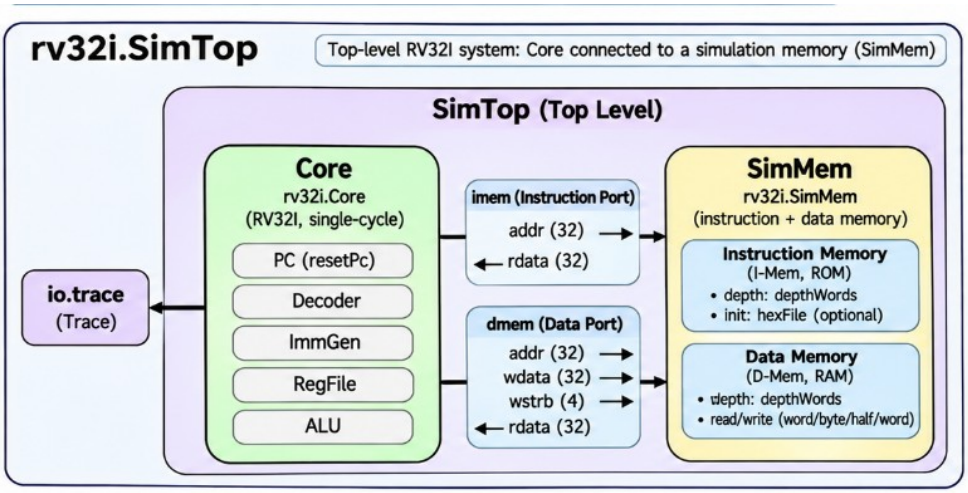
# GLab 3

## Building full RV32I  RISC-V architecture

In this GLab we propose complete RV32I architecture:



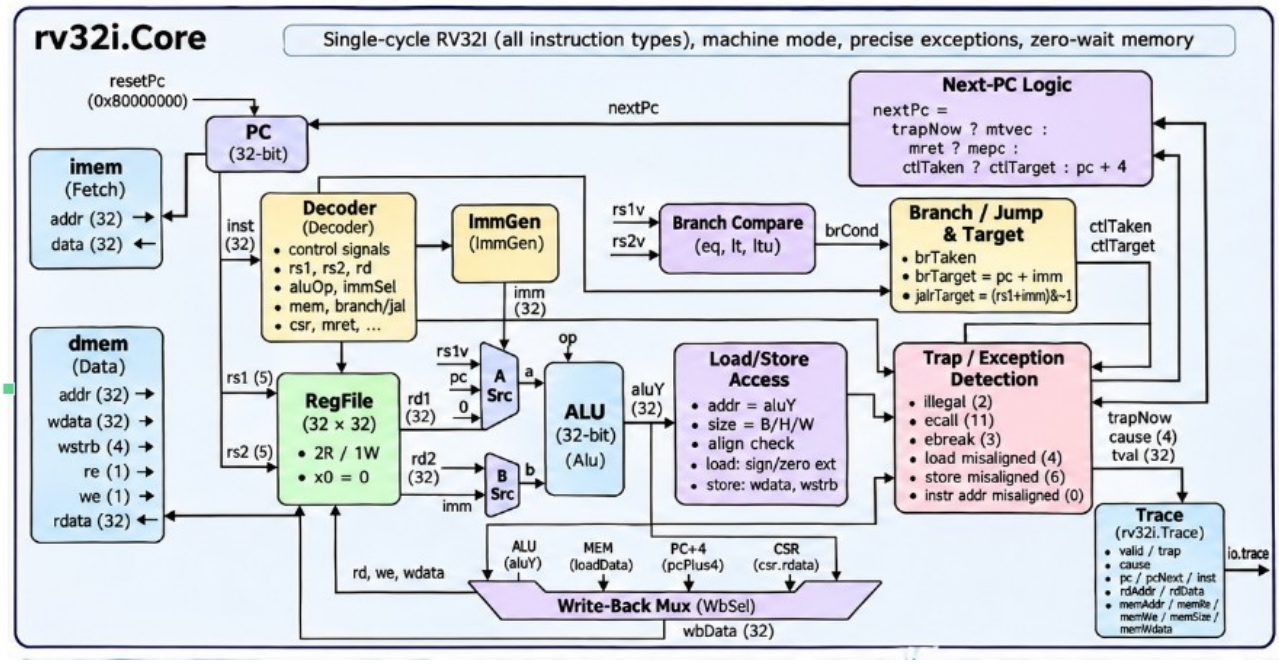

The following is the **`TopWrapper`** module of this architecture




```
package rv32i
import chisel3._

/** Stage 3 system: Core + InstructionMemory (program at 0x80000000) + DataMemory RAM (at 0x80010000). */
class RV32ITestWrapper(programFile: String, dataFile: String = "", dataDepth: Int = 256) extends Module {
  val io = IO(new Bundle {
    val trace = Output(new Trace)
  })
  val core = Module(new Core())
  val imem = Module(new InstructionMemory(programFile))
  val dmem = Module(new DataMemory(dataFile, dataDepth))
  imem.io.addr       := core.io.imem.addr
  core.io.imem.data  := imem.io.inst
  dmem.io.addr       := core.io.dmem.addr
  dmem.io.we         := core.io.dmem.we
  dmem.io.wdata      := core.io.dmem.wdata
  dmem.io.wstrb      := core.io.dmem.wstrb
  core.io.dmem.rdata := dmem.io.rdata
  io.trace := core.io.trace
}
```

---




---

---

```
$ cat Core.scala
package rv32i

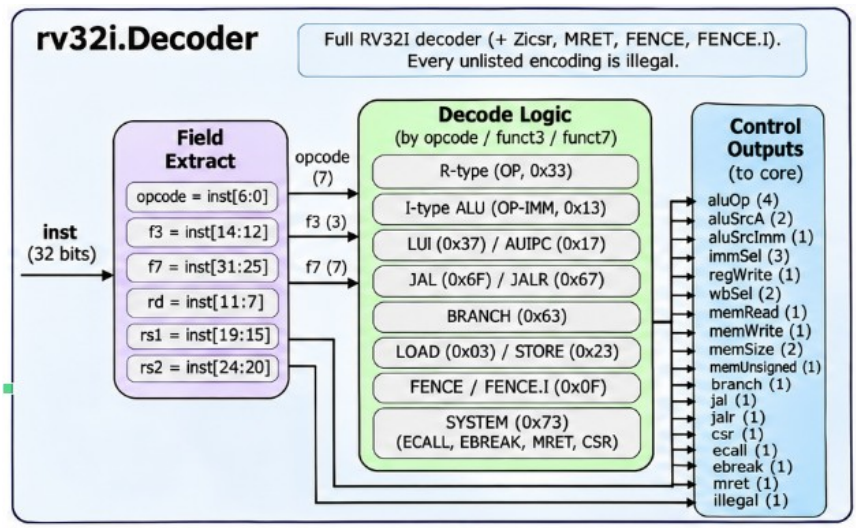
import chisel3._
import chisel3.util._

/** Stage 3: single-cycle RV32I core, the full base instruction set (40 instructions) + Zicsr, machine mode only,
  * no interrupts.
  *
  * Every cycle either retires one instruction (`trace.valid`) or takes one precise exception (`trace.trap`).
  * A trapping instruction changes no architectural state except the trap CSRs (mepc, mcause, mtval, mstatus)
  * and the pc, which becomes mtvec (direct mode).
  *
  * Instructions: LUI AUIPC JAL JALR, BEQ BNE BLT BGE BLTU BGEU, LB LH LW LBU LHU, SB SH SW,
  * ADDI SLTI SLTIU XORI ORI ANDI SLLI SRLI SRAI, ADD SUB SLL SLT SLTU XOR SRL SRA OR AND,
  * FENCE (no-op), ECALL, EBREAK, plus MRET, WFI (no-op) and CSRRW CSRRS CSRRC CSRRWI CSRRSI CSRRCI.
  * The M extension (funct7 = 0000001) is not implemented: those encodings are illegal instructions.
  *
  * Exceptions, in priority order: illegal instruction (2), ECALL (11), EBREAK (3), load misaligned (4),
  * store misaligned (6), instruction address misaligned (0: taken branch/JAL/JALR target with bit 1 set).
  * Misaligned data accesses trap (no hardware support). There are no access faults in this stage.
  *
  * Memory ports are zero-wait: the instruction and data read data must be combinational functions of the address.
  */
class Core(resetPc: Long = 0x80000000L, hartId: Int = 0) extends Module {
  val io = IO(new Bundle {
    val imem  = new InstrMemPort
    val dmem  = new DataMemPort
    val trace = Output(new Trace)
  })
  require(resetPc >= 0 && resetPc < (1L << 32) && resetPc % 4 == 0, "reset pc must be 4-byte aligned")
  val pc = RegInit(resetPc.U(32.W))
  io.imem.addr := pc
  val inst = io.imem.data
  val f3   = inst(14, 12)
  val dec  = Module(new Decoder)
  val immg = Module(new ImmGen)
  val rf   = Module(new RegFile)
  val alu  = Module(new Alu)
  val csr  = Module(new CsrFile(hartId))
  dec.io.inst  := inst
  immg.io.inst := inst
  immg.io.sel  := dec.io.immSel
  rf.io.rs1    := dec.io.rs1
  rf.io.rs2    := dec.io.rs2
  val rs1v = rf.io.rd1
  val rs2v = rf.io.rd2
  val imm  = immg.io.imm
  // ---------------- execute ----------------
  val aluA = WireDefault(rs1v)
  switch(dec.io.aluSrcA) {
```

```
  is(ASrc.RS1)  { aluA := rs1v }
  is(ASrc.PC)   { aluA := pc }
  is(ASrc.ZERO) { aluA := 0.U }
}
alu.io.op := dec.io.aluOp
alu.io.a  := aluA
alu.io.b  := Mux(dec.io.aluSrcImm, imm, rs2v)
val aluY = alu.io.y
// branch comparator and target adders
val eq  = rs1v === rs2v
val lt  = rs1v.asSInt < rs2v.asSInt
val ltu = rs1v < rs2v
val brCond = WireDefault(false.B)
switch(f3) {
  is("b000".U) { brCond := eq }
  is("b001".U) { brCond := !eq }
  is("b100".U) { brCond := lt }
  is("b101".U) { brCond := !lt }
  is("b110".U) { brCond := ltu }
  is("b111".U) { brCond := !ltu }
}
val brTaken    = dec.io.branch && brCond
val pcPlus4    = pc + 4.U
val brTarget   = pc + imm                          // branches and JAL
val jalrTarget = Cat(aluY(31, 1), 0.U(1.W))        // rs1 + imm, bit 0 cleared
val ctlTaken   = brTaken || dec.io.jal || dec.io.jalr
val ctlTarget  = Mux(dec.io.jalr, jalrTarget, brTarget)
// ---------------- data access ----------------
val size = dec.io.memSize
val misalignedAccess = (size === 2.U && aluY(1, 0) =/= 0.U) || (size === 1.U && aluY(0))
val loadMis  = dec.io.memRead && misalignedAccess
val storeMis = dec.io.memWrite && misalignedAccess
val fetchMis = ctlTaken && ctlTarget(1)
// ---------------- CSR / traps ----------------
val rs1Field = inst(19, 15)
csr.io.addr   := inst(31, 20)
csr.io.en     := dec.io.csr
csr.io.op     := f3(1, 0)
csr.io.src    := Mux(f3(2), Cat(0.U(27.W), rs1Field), rs1v)
csr.io.wrEn   := f3(1, 0) === 1.U || rs1Field =/= 0.U
val illegal = dec.io.illegal || csr.io.illegal
val trapNow = illegal || dec.io.ecall || dec.io.ebreak || loadMis || storeMis || fetchMis
val cause = PriorityMux(Seq(
  illegal         -> Cause.IllegalInstruction.U(4.W),
  dec.io.ecall    -> Cause.EcallFromM.U(4.W),
  dec.io.ebreak   -> Cause.Breakpoint.U(4.W),
  loadMis         -> Cause.LoadAddrMisaligned.U(4.W),
  storeMis        -> Cause.StoreAddrMisaligned.U(4.W),
  fetchMis        -> Cause.InstrAddrMisaligned.U(4.W)
))
val tval = PriorityMux(Seq(
  illegal                -> inst,
  dec.io.ecall           -> 0.U(32.W),
  dec.io.ebreak          -> pc,
  (loadMis || storeMis)  -> aluY,
  fetchMis               -> ctlTarget
))
val retire = !trapNow
csr.io.commit    := retire
csr.io.trapEnter := trapNow
csr.io.trapCause := Cat(0.U(28.W), cause)
csr.io.trapPc    := pc
csr.io.trapVal   := tval
csr.io.mretEnter := dec.io.mret && !trapNow
// ---------------- next pc ----------------
val nextPc = Mux(trapNow, csr.io.mtvecBase,
             Mux(dec.io.mret, csr.io.mepc,
             Mux(ctlTaken, ctlTarget, pcPlus4)))
pc := nextPc
// ---------------- load / store datapath ----------------
val rdata = io.dmem.rdata
val bytes = VecInit(Seq.tabulate(4)(k => rdata(8 * k + 7, 8 * k)))
val b     = bytes(aluY(1, 0))
val h     = Mux(aluY(1), rdata(31, 16), rdata(15, 0))
val uns   = dec.io.memUnsigned
val loadData = WireDefault(rdata)
switch(size) {
  is(0.U) { loadData := Mux(uns, Cat(0.U(24.W), b), Cat(Fill(24, b(7)), b)) }
  is(1.U) { loadData := Mux(uns, Cat(0.U(16.W), h), Cat(Fill(16, h(15)), h)) }
  is(2.U) { loadData := rdata }
}
val sbyte = rs2v(7, 0)
val shalf = rs2v(15, 0)
val wdata = WireDefault(rs2v)
val wstrb = WireDefault("b1111".U(4.W))
switch(size) {
  is(0.U) { wdata := Fill(4, sbyte); wstrb := (1.U(4.W) << aluY(1, 0))(3, 0) }
  is(1.U) { wdata := Fill(2, shalf); wstrb := Mux(aluY(1), "b1100".U(4.W), "b0011".U(4.W)) }
}
io.dmem.addr  := aluY
```

```
  io.dmem.re    := dec.io.memRead && !trapNow
  io.dmem.we    := dec.io.memWrite && !trapNow
  io.dmem.wdata := wdata
  io.dmem.wstrb := Mux(io.dmem.we, wstrb, 0.U)
  // ---------------- write-back ----------------
  val wbData = WireDefault(aluY)
  switch(dec.io.wbSel) {
    is(WbSel.ALU) { wbData := aluY }
    is(WbSel.MEM) { wbData := loadData }
    is(WbSel.PC4) { wbData := pcPlus4 }
    is(WbSel.CSR) { wbData := csr.io.rdata }
  }
  val rd = dec.io.rd
  rf.io.we    := dec.io.regWrite && retire
  rf.io.waddr := rd
  rf.io.wdata := wbData
  val writes = rf.io.we && rd =/= 0.U
  // ---------------- trace ----------------
  io.trace.valid    := retire
  io.trace.trap     := trapNow
  io.trace.cause    := cause
  io.trace.pc       := pc
  io.trace.pcNext   := nextPc
  io.trace.inst     := inst
  io.trace.rdAddr   := Mux(writes, rd, 0.U)
  io.trace.rdData   := Mux(writes, wbData, 0.U)
  io.trace.memAddr  := aluY
  io.trace.memRe    := io.dmem.re
  io.trace.memWe    := io.dmem.we
  io.trace.memSize  := size
  io.trace.memWdata := Mux(size === 0.U, Cat(0.U(24.W), sbyte), Mux(size === 1.U, Cat(0.U(16.W), shalf), rs2v))
  // ---------------- assertions (compiled out of synthesis builds) ----------------
  assert(pc(1, 0) === 0.U, "pc must stay 4-byte aligned")
  assert(!(io.trace.valid && io.trace.trap), "retire and trap in the same cycle")
  assert(!(io.dmem.we && io.dmem.re), "simultaneous load and store")
  assert(!(trapNow && (rf.io.we || io.dmem.we)), "trapping instruction modifies state")
}
```

---




---

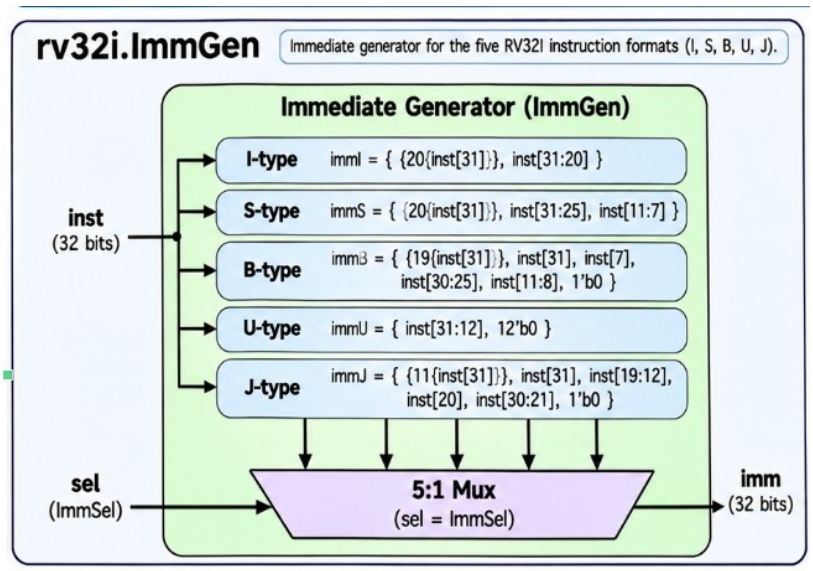
```
package rv32i
import chisel3._
import chisel3.util._

/** RV32I + Zicsr + M-mode SYSTEM decoder (combinational).
  *
  * Every control output is false/default unless the instruction is legal, so an illegal instruction has no
  * side effect besides the trap. The M extension (OP with funct7 = 0000001) is reported illegal here on
  * purpose: Stage 3 has no M extension, and in Stages 4 and 5 the core recognises those eight instructions
itself.
  *
  * FENCE and WFI are legal and execute as no-ops. FENCE.I (Zifencei) is not implemented and is illegal.
  */
class Decoder extends Module {
  val io = IO(new Bundle {
    val inst        = Input(UInt(32.W))
    val rs1         = Output(UInt(5.W))
    val rs2         = Output(UInt(5.W))
    val rd          = Output(UInt(5.W))
    val immSel      = Output(ImmSel())
    val aluSrcA     = Output(ASrc())
    val aluSrcImm   = Output(Bool())
    val aluOp       = Output(AluOp())
    val branch      = Output(Bool())
    val jal         = Output(Bool())
    val jalr        = Output(Bool())
```

```
  val memRead     = Output(Bool())
  val memWrite    = Output(Bool())
  val memSize     = Output(UInt(2.W))
  val memUnsigned = Output(Bool())
  val wbSel       = Output(WbSel())
  val regWrite    = Output(Bool())
  val csr         = Output(Bool())
  val ecall       = Output(Bool())
  val ebreak      = Output(Bool())
  val mret        = Output(Bool())
  val illegal     = Output(Bool())
})
val inst   = io.inst
val opcode = inst(6, 0)
val f3     = inst(14, 12)
val f7     = inst(31, 25)
io.rs1 := inst(19, 15)
io.rs2 := inst(24, 20)
io.rd  := inst(11, 7)
io.immSel      := ImmSel.I
io.aluSrcA     := ASrc.RS1
io.aluSrcImm   := false.B
io.aluOp       := AluOp.ADD
io.branch      := false.B
io.jal         := false.B
io.jalr        := false.B
io.memRead     := false.B
io.memWrite    := false.B
io.memSize     := f3(1, 0)
io.memUnsigned := f3(2)
io.wbSel       := WbSel.ALU
io.regWrite    := false.B
io.csr         := false.B
io.ecall       := false.B
io.ebreak      := false.B
io.mret        := false.B
io.illegal     := true.B
// funct3 -> ALU operation for OP and OP-IMM (funct7 bit 5 selects SUB / SRA)
private def aluFor(alt: Bool): AluOp.Type = {
  val op = WireDefault(AluOp.ADD)
  switch(f3) {
    is(0.U) { op := Mux(alt, AluOp.SUB, AluOp.ADD) }
    is(1.U) { op := AluOp.SLL }
    is(2.U) { op := AluOp.SLT }
    is(3.U) { op := AluOp.SLTU }
    is(4.U) { op := AluOp.XOR }
    is(5.U) { op := Mux(alt, AluOp.SRA, AluOp.SRL) }
    is(6.U) { op := AluOp.OR }
    is(7.U) { op := AluOp.AND }
  }
  op
}
switch(opcode) {
  is(Opcode.LUI.U) {
    io.illegal   := false.B
    io.regWrite  := true.B
    io.immSel    := ImmSel.U
    io.aluSrcA   := ASrc.ZERO
    io.aluSrcImm := true.B
  }
  is(Opcode.AUIPC.U) {
    io.illegal   := false.B
    io.regWrite  := true.B
    io.immSel    := ImmSel.U
    io.aluSrcA   := ASrc.PC
    io.aluSrcImm := true.B
  }
  is(Opcode.JAL.U) {
    io.illegal  := false.B
    io.jal      := true.B
    io.regWrite := true.B
    io.wbSel    := WbSel.PC4
    io.immSel   := ImmSel.J
  }
  is(Opcode.JALR.U) {
    when(f3 === 0.U) {
      io.illegal   := false.B
      io.jalr      := true.B
      io.regWrite  := true.B
      io.wbSel     := WbSel.PC4
      io.aluSrcImm := true.B
    }
  }
  is(Opcode.BRANCH.U) {
    when(f3 =/= 2.U && f3 =/= 3.U) {
      io.illegal := false.B
      io.branch  := true.B
      io.immSel  := ImmSel.B
    }
  }
```

```
    is(Opcode.LOAD.U) {
      when(f3 === 0.U || f3 === 1.U || f3 === 2.U || f3 === 4.U || f3 === 5.U) {
        io.illegal   := false.B
        io.memRead   := true.B
        io.regWrite  := true.B
        io.wbSel     := WbSel.MEM
        io.aluSrcImm := true.B
      }
    }
    is(Opcode.STORE.U) {
      when(f3 === 0.U || f3 === 1.U || f3 === 2.U) {
        io.illegal   := false.B
        io.memWrite  := true.B
        io.immSel    := ImmSel.S
        io.aluSrcImm := true.B
      }
    }
    is(Opcode.OP_IMM.U) {
      val shiftOk = Mux(f3 === 1.U, f7 === 0.U, Mux(f3 === 5.U, f7 === 0.U || f7 === "b0100000".U, true.B))
      when(shiftOk) {
        io.illegal   := false.B
        io.regWrite  := true.B
        io.aluSrcImm := true.B
        io.aluOp     := aluFor(f3 === 5.U && f7(5))
      }
    }
    is(Opcode.OP.U) {
      val altOk = f7 === "b0100000".U && (f3 === 0.U || f3 === 5.U)
      when(f7 === 0.U || altOk) {
        io.illegal  := false.B
        io.regWrite := true.B
        io.aluOp    := aluFor(f7(5))
      }
    }
    is(Opcode.MISC_MEM.U) {
      when(f3 === 0.U) { io.illegal := false.B } // FENCE: no-op on this single-hart, in-order core
    }
    is(Opcode.SYSTEM.U) {
      when(f3 === 0.U) {
        when(inst === "h00000073".U) { io.illegal := false.B; io.ecall := true.B }
        when(inst === "h00100073".U) { io.illegal := false.B; io.ebreak := true.B }
        when(inst === "h30200073".U) { io.illegal := false.B; io.mret := true.B }
        when(inst === "h10500073".U) { io.illegal := false.B } // WFI: no-op (no interrupts)
      }.elsewhen(f3 =/= 4.U) {
        io.illegal  := false.B
        io.csr      := true.B
        io.regWrite := true.B
        io.wbSel    := WbSel.CSR
      }
    }
  }
}
```

---




---

```
import chisel3._
import chisel3.util._

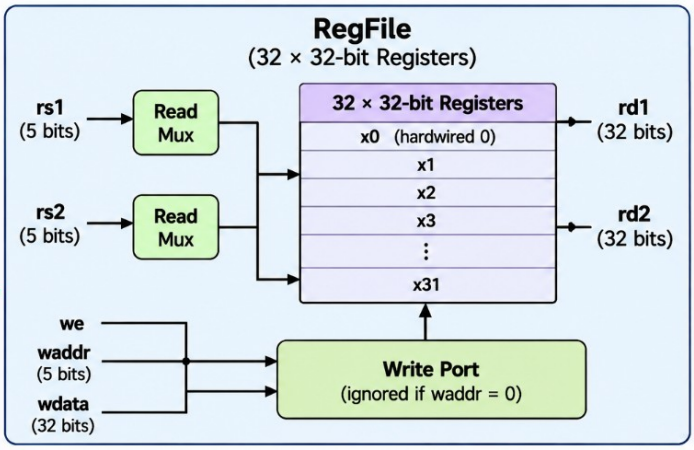
/** Immediate generator for the I, S, B, U and J formats (sign-extended to 32 bits). */
class ImmGen extends Module {
  val io = IO(new Bundle {
    val inst = Input(UInt(32.W))
    val sel  = Input(ImmSel())
```

```
    val imm  = Output(UInt(32.W))
  })

  val i    = io.inst
  val sign = i(31)
  val immI = Cat(Fill(21, sign), i(30, 20))
  val immS = Cat(Fill(21, sign), i(30, 25), i(11, 7))
  val immB = Cat(Fill(20, sign), i(7), i(30, 25), i(11, 8), 0.U(1.W))
  val immU = Cat(i(31, 12), 0.U(12.W))
  val immJ = Cat(Fill(12, sign), i(19, 12), i(20), i(30, 21), 0.U(1.W))

  io.imm := immI
  switch(io.sel) {
    is(ImmSel.I) { io.imm := immI }
    is(ImmSel.S) { io.imm := immS }
    is(ImmSel.B) { io.imm := immB }
    is(ImmSel.U) { io.imm := immU }
    is(ImmSel.J) { io.imm := immJ }
  }
}
```

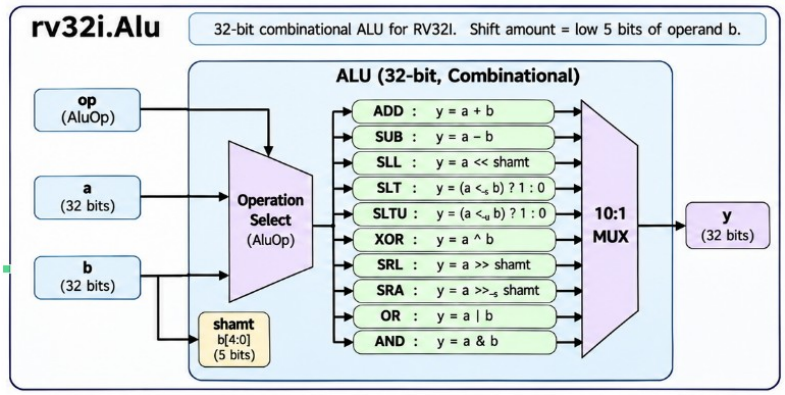
---




---

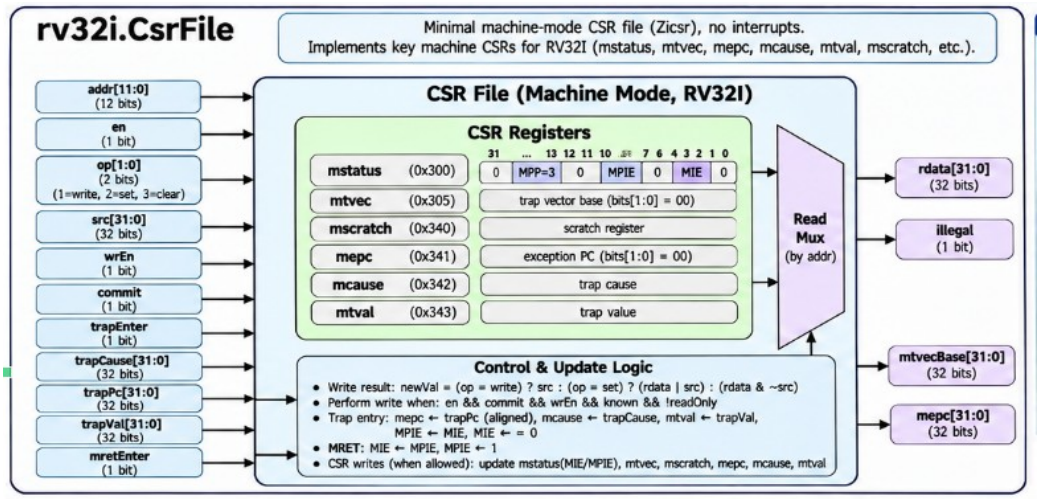
```
package rv32i
import chisel3._
/** 32 x 32-bit register file: two combinational read ports, one write port, x0 hardwired to zero.
  * All registers reset to 0. A read in the cycle of a write returns the old value (the write lands at the edge).
  */
class RegFile extends Module {
  val io = IO(new Bundle {
    val rs1   = Input(UInt(5.W))
    val rs2   = Input(UInt(5.W))
    val rd1   = Output(UInt(32.W))
    val rd2   = Output(UInt(32.W))
    val we    = Input(Bool())
    val waddr = Input(UInt(5.W))
    val wdata = Input(UInt(32.W))
  })
  val regs = RegInit(VecInit(Seq.fill(32)(0.U(32.W))))
  io.rd1 := Mux(io.rs1 === 0.U, 0.U, regs(io.rs1))
  io.rd2 := Mux(io.rs2 === 0.U, 0.U, regs(io.rs2))
  when(io.we && io.waddr =/= 0.U) {
    regs(io.waddr) := io.wdata
  }
}
```

---



```
--------------------------------------------------------------------------------------------
package rv32i
import chisel3._
import chisel3.util._

/** 32-bit ALU for RV32I. Shifts use the low 5 bits of `b`. */
class Alu extends Module {
  val io = IO(new Bundle {
    val op = Input(AluOp())
    val a  = Input(UInt(32.W))
    val b  = Input(UInt(32.W))
    val y  = Output(UInt(32.W))
  })
  val sh = io.b(4, 0)
  io.y := io.a + io.b
  switch(io.op) {
    is(AluOp.ADD)  { io.y := io.a + io.b }
    is(AluOp.SUB)  { io.y := io.a - io.b }
    is(AluOp.SLL)  { io.y := (io.a << sh)(31, 0) }
    is(AluOp.SLT)  { io.y := (io.a.asSInt < io.b.asSInt).asUInt }
    is(AluOp.SLTU) { io.y := (io.a < io.b).asUInt }
    is(AluOp.XOR)  { io.y := io.a ^ io.b }
    is(AluOp.SRL)  { io.y := io.a >> sh }
    is(AluOp.SRA)  { io.y := (io.a.asSInt >> sh).asUInt }
    is(AluOp.OR)   { io.y := io.a | io.b }
    is(AluOp.AND)  { io.y := io.a & io.b }
  }
}
--------------------------------------------------------------------------------------------
```




```
--------------------------------------------------------------------------------------------

package rv32i
import chisel3._
import chisel3.util._
/** Machine-mode CSRs for the RV32I core (no interrupts).
  *
  * CSR instruction interface (combinational read, write at the clock edge):
  *  - `en`: a CSR instruction is in execute; `addr`, `op` (funct3(1,0): 1 RW, 2 RS, 3 RC), `src` (rs1 value or
  *    zero-extended uimm) and `wrEn` (false for CSRRS/CSRRC with rs1/uimm = 0) describe it.
  *  - `rdata` is the old value; `illegal` is high for an unknown CSR or a write to a read-only CSR.
  *  - The write happens only when `commit` is high (the instruction retires) and the access is legal.
  *
  * Trap interface: `trapEnter` saves mepc/mcause/mtval and MIE -> MPIE, MIE := 0.
  * `mretEnter` restores MIE := MPIE, MPIE := 1. mstatus.MPP is hardwired to 3 (M-mode only).
  *
  * Counters: mcycle counts every clock cycle;
  * minstret counts retired instructions (`commit`). A CSR write to one half of a counter replaces that half
  * of the incremented value. cycle/instret(h) are read-only shadows. There is no `time` CSR.
  */
class CsrFile(hartId: Int = 0) extends Module {
  val io = IO(new Bundle {
    val addr      = Input(UInt(12.W))
    val en        = Input(Bool())
    val op        = Input(UInt(2.W))
    val src       = Input(UInt(32.W))
    val wrEn      = Input(Bool())
    val rdata     = Output(UInt(32.W))
    val illegal   = Output(Bool())
    val commit    = Input(Bool())
    val trapEnter = Input(Bool())
    val trapCause = Input(UInt(32.W))
    val trapPc    = Input(UInt(32.W))
    val trapVal   = Input(UInt(32.W))
    val mretEnter = Input(Bool())
    val mtvecBase = Output(UInt(32.W))
    val mepc      = Output(UInt(32.W))
```

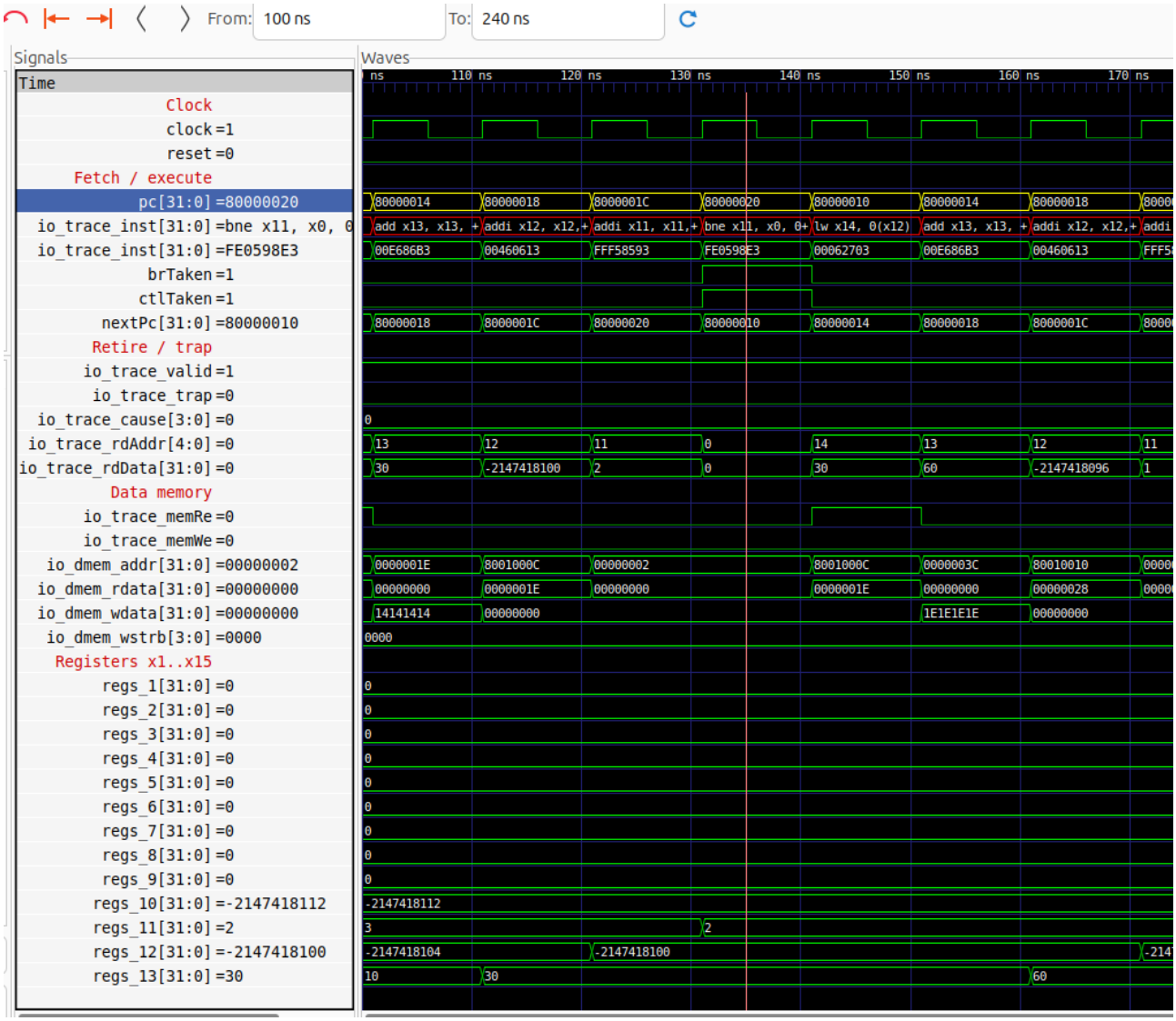
```
  })
  val mie_     = RegInit(false.B) // mstatus.MIE
  val mpie     = RegInit(false.B) // mstatus.MPIE
  val mieReg   = RegInit(0.U(32.W))
  val mtvec    = RegInit(0.U(32.W))
  val mscratch = RegInit(0.U(32.W))
  val mepc     = RegInit(0.U(32.W))
  val mcause   = RegInit(0.U(32.W))
  val mtval    = RegInit(0.U(32.W))
  val mcycle   = RegInit(0.U(64.W))
  val minstret = RegInit(0.U(64.W))
  val mstatus = Cat(0.U(19.W), "b11".U(2.W), 0.U(3.W), mpie, 0.U(3.W), mie_, 0.U(3.W)) // MPP = 11
  val A = CsrAddr
  val table: Seq[(Int, UInt)] = Seq(
    A.mstatus   -> mstatus,
    A.misa      -> A.MisaValue.U(32.W),
    A.mie       -> mieReg,
    A.mtvec     -> mtvec,
    A.mstatush  -> 0.U(32.W),
    A.mscratch  -> mscratch,
    A.mepc      -> mepc,
    A.mcause    -> mcause,
    A.mtval     -> mtval,
    A.mip       -> 0.U(32.W),
    A.mcycle    -> mcycle(31, 0),
    A.mcycleh   -> mcycle(63, 32),
    A.minstret  -> minstret(31, 0),
    A.minstreth -> minstret(63, 32),
    A.cycle     -> mcycle(31, 0),
    A.cycleh    -> mcycle(63, 32),
    A.instret   -> minstret(31, 0),
    A.instreth  -> minstret(63, 32),
    A.mvendorid -> 0.U(32.W),
    A.marchid   -> 0.U(32.W),
    A.mimpid    -> 0.U(32.W),
    A.mhartid   -> hartId.U(32.W)
  )
  val hit      = table.map { case (a, _) => io.addr === a.U }
  val known    = hit.reduce(_ || _)
  val old      = Mux1H(hit, table.map(_._2))
  val readOnly = io.addr(11, 10) === 3.U
  io.rdata   := Mux(known, old, 0.U)
  io.illegal := io.en && (!known || (readOnly && io.wrEn))
  val wval = MuxLookup(io.op, io.src)(Seq(
    1.U -> io.src,
    2.U -> (old | io.src),
    3.U -> (old & ~io.src)
  ))
  val doWrite = io.en && io.wrEn && io.commit && !io.illegal
  def wr(a: Int): Bool = doWrite && io.addr === a.U
  // ---- counters: increment, then a CSR write replaces the written half
  val cycleInc   = mcycle + 1.U
  val instretInc = minstret + io.commit.asUInt
  mcycle := MuxCase(cycleInc, Seq(
    wr(A.mcycle) -> Cat(cycleInc(63, 32), wval),
    wr(A.mcycleh)        -> Cat(wval, cycleInc(31, 0))
  ))
  minstret := MuxCase(instretInc, Seq(
    wr(A.minstret) -> Cat(instretInc(63, 32), wval),
    wr(A.minstreth)        -> Cat(wval, instretInc(31, 0))
  ))
  // ---- other writable CSRs (WARL masks)
  when(wr(A.mstatus)) { mie_ := wval(3); mpie := wval(7) }
  when(wr(A.mie))     { mieReg := wval & "h888".U }
  when(wr(A.mtvec))   { mtvec := Cat(wval(31, 2), 0.U(2.W)) }  // direct mode only
  when(wr(A.mscratch)) { mscratch := wval }
  when(wr(A.mepc))    { mepc := Cat(wval(31, 2), 0.U(2.W)) }   // IALIGN = 32
  when(wr(A.mcause))  { mcause := wval }
  when(wr(A.mtval))   { mtval := wval }
  // misa, mstatush, mip: writes ignored (WARL, read-only zero / fixed)
  // ---- trap entry and return
  when(io.trapEnter) {
    mepc   := io.trapPc
    mcause := io.trapCause
    mtval  := io.trapVal
    mpie   := mie_
    mie_   := false.B
  }.elsewhen(io.mretEnter) {
    mie_ := mpie
    mpie := true.B
  }
  io.mtvecBase := mtvec
  io.mepc      := mepc
}
```

---

**$ sbt test**

```
[info] welcome to sbt 1.10.2 (Ubuntu Java 17.0.20.1)
[info] loading project definition from /media/bako/0969-2ABE/books_2026/CL.Book.Chisel.2026/RV32.Steps/rv32i-
stage3/project
[info] loading settings for project root from build.sbt ...
```

```
[info] set current project to rv32i-stage3 (in build
file:/media/bako/0969-2ABE/books_2026/CL.Book.Chisel.2026/RV32.Steps/rv32i-stage3/)
[info] DecoderSpec:
[info] Decoder
[info] - should decode every RV32I, Zicsr and M-mode SYSTEM instruction
[info] - should flag illegal encodings and assert no other control signal for them
[info] - should classify every one of the 128 major opcodes
[info] AluSpec:
[info] Alu
[info] - should compute every operation on all edge-value pairs (incl. overflow, shift by 31/32, sign)
[info] - should match the reference on random operands
[info] WaveformSpec:
[info] Waveforms
[info] - should show the Stage 3 demo (program.hex + data.hex) on RV32ITestWrapper (waves/stage3_demo)
[info] - should show LUI, AUIPC, JAL and all six branches (waves/upper_jumps)
[info] - should show loads, stores, a loop and a call (waves/load_store_branch)
[info] - should show an ECALL, an illegal instruction (MUL), the handler and MRET (waves/trap)
[info] CsrFileSpec:
[info] CsrFile
[info] - should implement CSRRW / CSRRS / CSRRC semantics
[info] - should report misa = RV32I, mhartid and fixed values, and apply WARL masks
[info] - should flag unknown CSRs and writes to read-only CSRs as illegal, and not write them
[info] - should count cycles always and instructions on commit, and let a write replace one half
[info] - should save state on trap entry and restore it on mret
[info] ImmGenSpec:
[info] ImmGen
[info] - should build the I, S, B, U and J immediates from instructions the assembler encoded
[info] CoreTimingSpec:
[info] Core
[info] - should execute every RV32I and Zicsr instruction in one cycle, and take every trap in one cycle
[info] CoreProgramSpec:
[info] Core
[info] - should run arithmetic and logic instructions (OP, OP-IMM, LUI, AUIPC)
[info] - should run loads and stores of every size, signed and unsigned
[info] - should run every branch both ways, loops, JAL and JALR (incl. clearing bit 0)
[info] - should handle back-to-back dependencies, including a load result used immediately
[info] - should run loops written in RV32I only: gcd by subtraction, a sum, and multiplication by shift-and-add
[info] - should treat the eight M-extension encodings as illegal instructions (cause 2, mtval = the instruction)
[info] - should implement Zicsr and take every exception precisely (illegal, ECALL, EBREAK, misaligned
load/store/fetch)
[info] - should match the reference model on 3000 random instructions
[info] RegFileSpec:
[info] RegFile
[info] - should hardwire x0 to zero and store x1..x31
[info] - should ignore writes when we is low and return the old value during a write
[info] MemorySpec:
[info] InstructionMemory
[info] - should map the program at 0x80000000 and read 0 outside it
[info] - should accept comments, blank lines and separators in hex files
[info] DataMemory
[info] - should start from data.hex, write bytes, halves and words under byte strobes, and ignore outside writes
[info] RV32ITestWrapper
[info] - should run program.hex on data.hex: sum an array, store and reload it, call a function
[info] Run completed in 32 seconds, 770 milliseconds.
[info] Total number of tests run: 30
[info] Suites: completed 9, aborted 0
[info] Tests: succeeded 30, failed 0, canceled 0, ignored 0, pending 0
[info] All tests passed.
[success] Total time: 36 s, completed 10 nov. 2026, 17:28:00
```


From: 100 ns
To: 240 ns
Signals
Waves
Time
Clock
clock =1
reset =0
Fetch / execute
pc[31:0] =80000020
io_trace_inst[31:0] =bne x11, x0, 0
io_trace_inst[31:0] =FE0598E3
brTaken =1
ctlTaken =1
nextPc[31:0] =80000010
Retire / trap
io_trace_valid =1
io_trace_trap =0
io_trace_cause[3:0] =0
io_trace_rdAddr[4:0] =0
io_trace_rdData[31:0] =0
Data memory
io_trace_memRe =0
io_trace_memWe =0
io_dmem_addr[31:0] =00000002
io_dmem_rdata[31:0] =00000000
io_dmem_wdata[31:0] =00000000
io_dmem_wstrb[3:0] =0000
Registers x1..x15
regs_1[31:0] =0
regs_2[31:0] =0
regs_3[31:0] =0
regs_4[31:0] =0
regs_5[31:0] =0
regs_6[31:0] =0
regs_7[31:0] =0
regs_8[31:0] =0
regs_9[31:0] =0
regs_10[31:0] =-2147418112
regs_11[31:0] =2
regs_12[31:0] =-2147418100
regs_13[31:0] =30
ns
110 ns
120 ns
130 ns
140 ns
150 ns
160 ns
170 ns

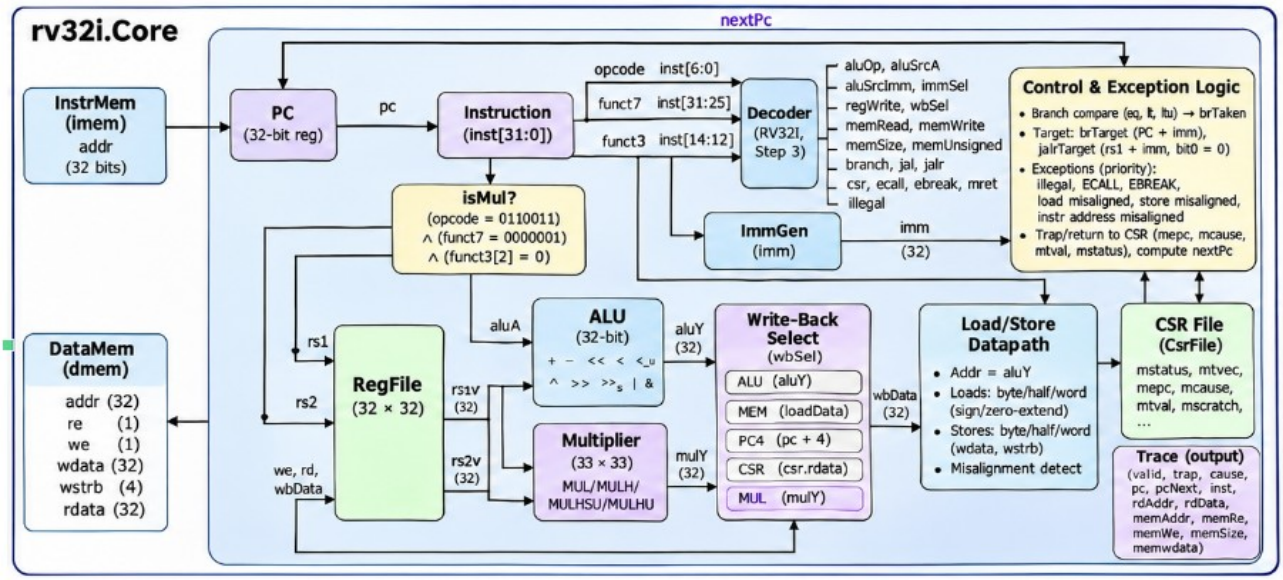
# GLab 4

## Building full RV32IM (MUL only) architecture

The following architecture is extended with M instructions including only multiplications.

```
--------------------------------------------------------------------------------
package rv32i
import chisel3._

/** Stage 3 system: Core + InstructionMemory (program at 0x80000000) + DataMemory RAM (at 0x80010000). */
class RV32ITestWrapper(programFile: String, dataFile: String = "", dataDepth: Int = 256) extends Module {
  val io = IO(new Bundle {
    val trace = Output(new Trace)
  })
  val core = Module(new Core())
  val imem = Module(new InstructionMemory(programFile))
  val dmem = Module(new DataMemory(dataFile, dataDepth))
  imem.io.addr       := core.io.imem.addr
  core.io.imem.data  := imem.io.inst
  dmem.io.addr       := core.io.dmem.addr
  dmem.io.we         := core.io.dmem.we
  dmem.io.wdata      := core.io.dmem.wdata
  dmem.io.wstrb      := core.io.dmem.wstrb
  core.io.dmem.rdata := dmem.io.rdata
  io.trace := core.io.trace
}
--------------------------------------------------------------------------------
```




```
--------------------------------------------------------------------------------
cat Core.scala
package rv32i

import chisel3._
import chisel3.util._

/** Stage 4: single-cycle RV32I core + the M-extension multiplications, Zicsr, machine mode only, no interrupts.
  *
  * Every cycle either retires one instruction (`trace.valid`) or takes one precise exception (`trace.trap`).
  * A trapping instruction changes no architectural state except the trap CSRs (mepc, mcause, mtval, mstatus)
  * and the pc, which becomes mtvec (direct mode).
  *
  * Instructions: LUI AUIPC JAL JALR, BEQ BNE BLT BGE BLTU BGEU, LB LH LW LBU LHU, SB SH SW,
  * ADDI SLTI SLTIU XORI ORI ANDI SLLI SRLI SRAI, ADD SUB SLL SLT SLTU XOR SRL SRA OR AND,
  * FENCE (no-op), ECALL, EBREAK, plus MRET, WFI (no-op) and CSRRW CSRRS CSRRC CSRRWI CSRRSI CSRRCI.
  *
  * Stage 4: MUL, MULH, MULHSU, MULHU execute in one cycle on the combinational (parallel) `Multiplier`. They are
  * decoded here (OP opcode, funct7 = 0000001, funct3 = 0xx), so the Stage 3 Decoder is unchanged and still
reports
  * funct7 = 0000001 as illegal on its own. DIV, DIVU, REM, REMU (funct3 = 1xx) remain illegal instructions in
this
  * stage (Stage 5 adds them), and misa stays RV32I: the M bit is only correct once division exists.
  *
  * Exceptions, in priority order: illegal instruction (2), ECALL (11), EBREAK (3), load misaligned (4),
  * store misaligned (6), instruction address misaligned (0: taken branch/JAL/JALR target with bit 1 set).
  * Misaligned data accesses trap (no hardware support). There are no access faults in this stage.
  *
  * Memory ports are zero-wait: the instruction and data read data must be combinational functions of the address.
  */
```

```
class Core(resetPc: Long = 0x80000000L, hartId: Int = 0) extends Module {
  val io = IO(new Bundle {
    val imem  = new InstrMemPort
    val dmem  = new DataMemPort
    val trace = Output(new Trace)
  })
  require(resetPc >= 0 && resetPc < (1L << 32) && resetPc % 4 == 0, "reset pc must be 4-byte aligned")
  val pc = RegInit(resetPc.U(32.W))
  io.imem.addr := pc
  val inst = io.imem.data
  val f3   = inst(14, 12)
  // ---------------- M extension (Stage 4): multiplications only ----------------
  val isMul = inst(6, 0) === "b0110011".U && inst(31, 25) === "b0000001".U && !f3(2)
  val dec  = Module(new Decoder)
  val immg = Module(new ImmGen)
  val rf   = Module(new RegFile)
  val alu  = Module(new Alu)
  val csr  = Module(new CsrFile(hartId))
  val mul  = Module(new Multiplier)

  dec.io.inst  := inst
  immg.io.inst := inst
  immg.io.sel  := dec.io.immSel
  // Register addresses straight from the instruction for a multiplication (the decoder treats it as illegal).
  rf.io.rs1    := Mux(isMul, inst(19, 15), dec.io.rs1)
  rf.io.rs2    := Mux(isMul, inst(24, 20), dec.io.rs2)
  val rs1v = rf.io.rd1
  val rs2v = rf.io.rd2
  val imm  = immg.io.imm

  mul.io.fn := f3(1, 0)
  mul.io.a  := rs1v
  mul.io.b  := rs2v

  // ---------------- execute ----------------
  val aluA = WireDefault(rs1v)
  switch(dec.io.aluSrcA) {
    is(ASrc.RS1)  { aluA := rs1v }
    is(ASrc.PC)   { aluA := pc }
    is(ASrc.ZERO) { aluA := 0.U }
  }
  alu.io.op := dec.io.aluOp
  alu.io.a  := aluA
  alu.io.b  := Mux(dec.io.aluSrcImm, imm, rs2v)
  val aluY = alu.io.y

  // branch comparator and target adders
  val eq  = rs1v === rs2v
  val lt  = rs1v.asSInt < rs2v.asSInt
  val ltu = rs1v < rs2v
  val brCond = WireDefault(false.B)
  switch(f3) {
    is("b000".U) { brCond := eq }
    is("b001".U) { brCond := !eq }
    is("b100".U) { brCond := lt }
    is("b101".U) { brCond := !lt }
    is("b110".U) { brCond := ltu }
    is("b111".U) { brCond := !ltu }
  }
  val brTaken    = dec.io.branch && brCond
  val pcPlus4    = pc + 4.U
  val brTarget   = pc + imm                          // branches and JAL
  val jalrTarget = Cat(aluY(31, 1), 0.U(1.W))        // rs1 + imm, bit 0 cleared
  val ctlTaken   = brTaken || dec.io.jal || dec.io.jalr
  val ctlTarget  = Mux(dec.io.jalr, jalrTarget, brTarget)

  // ---------------- data access ----------------
  val size = dec.io.memSize
  val misalignedAccess = (size === 2.U && aluY(1, 0) =/= 0.U) || (size === 1.U && aluY(0))
  val loadMis  = dec.io.memRead && misalignedAccess
  val storeMis = dec.io.memWrite && misalignedAccess
  val fetchMis = ctlTaken && ctlTarget(1)

  // ---------------- CSR / traps ----------------
  val rs1Field = inst(19, 15)
  csr.io.addr   := inst(31, 20)
  csr.io.en     := dec.io.csr
  csr.io.op     := f3(1, 0)
  csr.io.src    := Mux(f3(2), Cat(0.U(27.W), rs1Field), rs1v)
  csr.io.wrEn   := f3(1, 0) === 1.U || rs1Field =/= 0.U

  // A multiplication is legal even though the Stage 3 decoder flags funct7 = 0000001.
  val illegal = (dec.io.illegal && !isMul) || csr.io.illegal
  val trapNow = illegal || dec.io.ecall || dec.io.ebreak || loadMis || storeMis || fetchMis

  val cause = PriorityMux(Seq(
    illegal         -> Cause.IllegalInstruction.U(4.W),
    dec.io.ecall    -> Cause.EcallFromM.U(4.W),
    dec.io.ebreak   -> Cause.Breakpoint.U(4.W),
    loadMis         -> Cause.LoadAddrMisaligned.U(4.W),
```

```
    storeMis        -> Cause.StoreAddrMisaligned.U(4.W),
    fetchMis        -> Cause.InstrAddrMisaligned.U(4.W)
  ))
  val tval = PriorityMux(Seq(
    illegal                -> inst,
    dec.io.ecall           -> 0.U(32.W),
    dec.io.ebreak          -> pc,
    (loadMis || storeMis)  -> aluY,
    fetchMis               -> ctlTarget
  ))

  val retire = !trapNow

  csr.io.commit    := retire
  csr.io.trapEnter := trapNow
  csr.io.trapCause := Cat(0.U(28.W), cause)
  csr.io.trapPc    := pc
  csr.io.trapVal   := tval
  csr.io.mretEnter := dec.io.mret && !trapNow

  // ---------------- next pc ----------------
  val nextPc = Mux(trapNow, csr.io.mtvecBase,
               Mux(dec.io.mret, csr.io.mepc,
               Mux(ctlTaken, ctlTarget, pcPlus4)))
  pc := nextPc

  // ---------------- load / store datapath ----------------
  val rdata = io.dmem.rdata
  val bytes = VecInit(Seq.tabulate(4)(k => rdata(8 * k + 7, 8 * k)))
  val b     = bytes(aluY(1, 0))
  val h     = Mux(aluY(1), rdata(31, 16), rdata(15, 0))
  val uns   = dec.io.memUnsigned
  val loadData = WireDefault(rdata)
  switch(size) {
    is(0.U) { loadData := Mux(uns, Cat(0.U(24.W), b), Cat(Fill(24, b(7)), b)) }
    is(1.U) { loadData := Mux(uns, Cat(0.U(16.W), h), Cat(Fill(16, h(15)), h)) }
    is(2.U) { loadData := rdata }
  }

  val sbyte = rs2v(7, 0)
  val shalf = rs2v(15, 0)
  val wdata = WireDefault(rs2v)
  val wstrb = WireDefault("b1111".U(4.W))
  switch(size) {
    is(0.U) { wdata := Fill(4, sbyte); wstrb := (1.U(4.W) << aluY(1, 0))(3, 0) }
    is(1.U) { wdata := Fill(2, shalf); wstrb := Mux(aluY(1), "b1100".U(4.W), "b0011".U(4.W)) }
  }

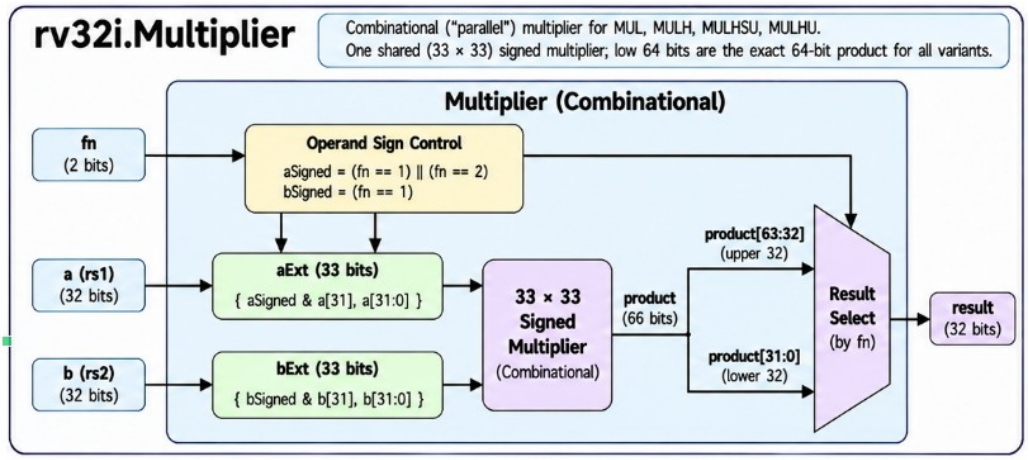
  io.dmem.addr  := aluY
  io.dmem.re    := dec.io.memRead && !trapNow
  io.dmem.we    := dec.io.memWrite && !trapNow
  io.dmem.wdata := wdata
  io.dmem.wstrb := Mux(io.dmem.we, wstrb, 0.U)

  // ---------------- write-back ----------------
  val wbData = WireDefault(aluY)
  switch(dec.io.wbSel) {
    is(WbSel.ALU) { wbData := aluY }
    is(WbSel.MEM) { wbData := loadData }
    is(WbSel.PC4) { wbData := pcPlus4 }
    is(WbSel.CSR) { wbData := csr.io.rdata }
  }
  when(isMul) { wbData := mul.io.result }            // last connect wins
  val rd = Mux(isMul, inst(11, 7), dec.io.rd)
  rf.io.we    := (dec.io.regWrite || isMul) && retire
  rf.io.waddr := rd
  rf.io.wdata := wbData
  val writes = rf.io.we && rd =/= 0.U

  // ---------------- trace ----------------
  io.trace.valid    := retire
  io.trace.trap     := trapNow
  io.trace.cause    := cause
  io.trace.pc       := pc
  io.trace.pcNext   := nextPc
  io.trace.inst     := inst
  io.trace.rdAddr   := Mux(writes, rd, 0.U)
  io.trace.rdData   := Mux(writes, wbData, 0.U)
  io.trace.memAddr  := aluY
  io.trace.memRe    := io.dmem.re
  io.trace.memWe    := io.dmem.we
  io.trace.memSize  := size
  io.trace.memWdata := Mux(size === 0.U, Cat(0.U(24.W), sbyte), Mux(size === 1.U, Cat(0.U(16.W), shalf), rs2v))

  // ---------------- assertions (compiled out of synthesis builds) ----------------
  assert(pc(1, 0) === 0.U, "pc must stay 4-byte aligned")
  assert(!(io.trace.valid && io.trace.trap), "retire and trap in the same cycle")
  assert(!(io.dmem.we && io.dmem.re), "simultaneous load and store")
  assert(!(trapNow && (rf.io.we || io.dmem.we)), "trapping instruction modifies state")
}
```

---




---

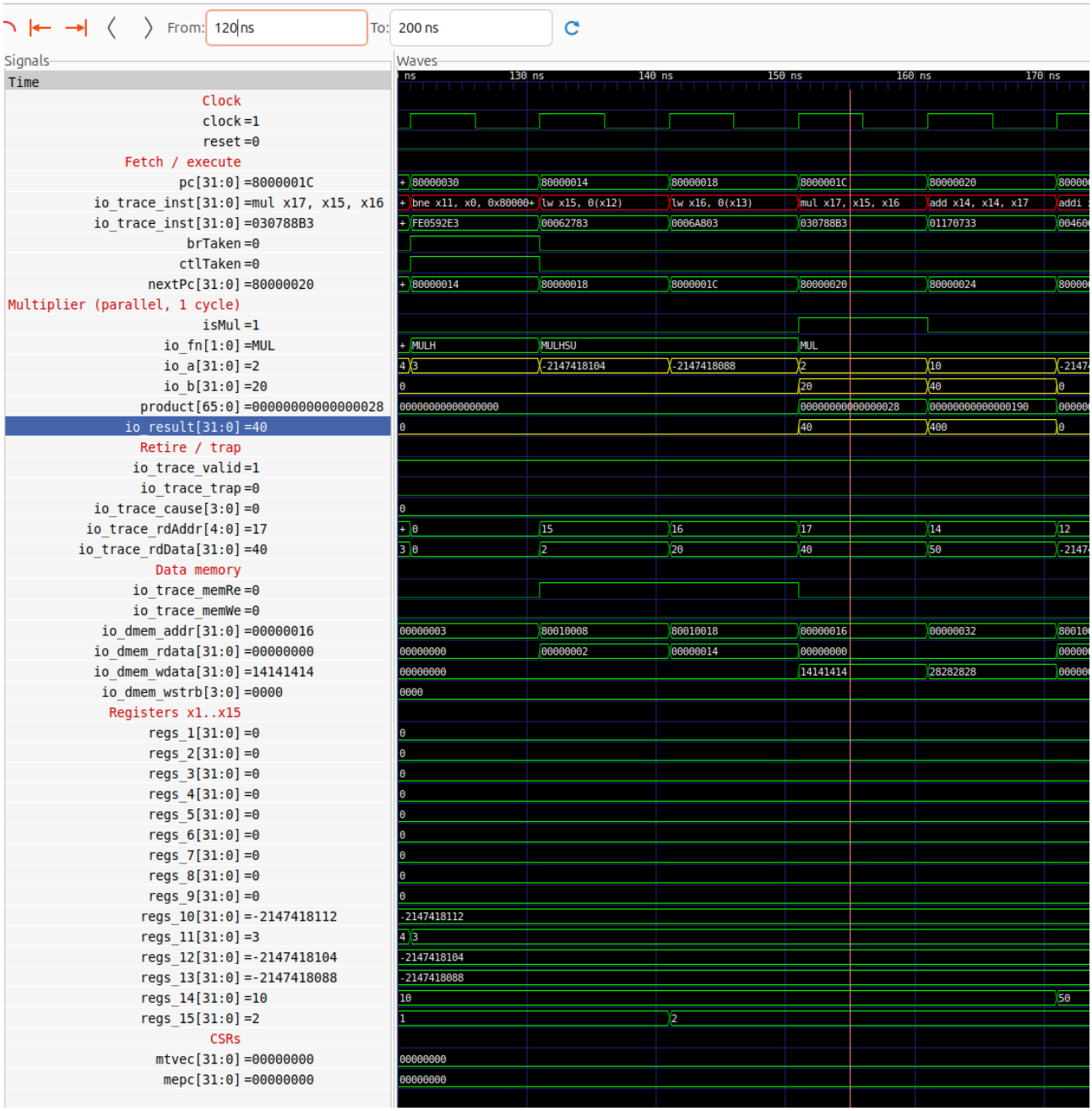
```
package rv32i
import chisel3._
import chisel3.util._
/** Step 4: combinational ("parallel") multiplier for MUL, MULH, MULHSU, MULHU (fn = funct3(1,0)).
  *
  * One shared (33 x 33) signed multiplier: each operand is extended by one bit, with its sign bit for
  * a signed operand (MULH: both, MULHSU: rs1) and with 0 otherwise. The low 64 bits of that product are
  * the exact 64-bit result of every variant, so one array serves all four instructions.
  *   fn 0 MUL    -> product[31:0]  (same bits for signed and unsigned)
  *   fn 1 MULH   -> product[63:32] signed   x signed
  *   fn 2 MULHSU -> product[63:32] signed   x unsigned
  *   fn 3 MULHU  -> product[63:32] unsigned x unsigned
  */
class Multiplier extends Module {
  val io = IO(new Bundle {
    val fn     = Input(UInt(2.W))
    val a      = Input(UInt(32.W))   // rs1
    val b      = Input(UInt(32.W))   // rs2
    val result = Output(UInt(32.W))
  })
  val aSigned = io.fn === 1.U || io.fn === 2.U        // MULH, MULHSU
  val bSigned = io.fn === 1.U                         // MULH
  val aExt    = Cat(aSigned && io.a(31), io.a).asSInt // 33 bits
  val bExt    = Cat(bSigned && io.b(31), io.b).asSInt
  val product = (aExt * bExt).asUInt                  // 66 bits, low 64 are exact
  io.result := Mux(io.fn === 0.U, product(31, 0), product(63, 32))
}
```

---

## $ sbt test

```
[info] welcome to sbt 1.10.2 (Ubuntu Java 17.0.20.1)
[info] loading project definition from /media/bako/0969-2ABE/books_2026/CL.Book.Chisel.2026/RV32.Steps/rv32im-
stage4/project
[info] loading settings for project root from build.sbt ...
[info] set current project to rv32im-stage4 (in build
file:/media/bako/0969-2ABE/books_2026/CL.Book.Chisel.2026/RV32.Steps/rv32im-stage4/)
[info] DecoderSpec:
[info] Decoder
[info] - should decode every RV32I, Zicsr and M-mode SYSTEM instruction
[info] - should flag illegal encodings and assert no other control signal for them
[info] - should classify every one of the 128 major opcodes
[info] AluSpec:
[info] Alu
[info] - should compute every operation on all edge-value pairs (incl. overflow, shift by 31/32, sign)
[info] - should match the reference on random operands
[info] WaveformSpec:
[info] Waveforms
[info] - should show the Stage 4 demo (program.hex + data.hex) on RV32ITestWrapper (waves/stage4_demo)
[info] - should show LUI, AUIPC, JAL and all six branches (waves/upper_jumps)
[info] - should show loads, stores, a loop and a call (waves/load_store_branch)
[info] - should show the four multiplications on corner values, each in one cycle (waves/mul)
[info] - should show an ECALL, an illegal instruction (DIV), the handler and MRET (waves/trap)
[info] CsrFileSpec:
[info] CsrFile
[info] - should implement CSRRW / CSRRS / CSRRC semantics
[info] - should report misa = RV32I, mhartid and fixed values, and apply WARL masks
[info] - should flag unknown CSRs and writes to read-only CSRs as illegal, and not write them
[info] - should count cycles always and instructions on commit, and let a write replace one half
[info] - should save state on trap entry and restore it on mret
[info] MultiplierSpec:
[info] Multiplier
[info] - should compute MUL, MULH, MULHSU and MULHU on edge values and random operands
[info] ImmGenSpec:
[info] ImmGen
```

```
[info] - should build the I, S, B, U and J immediates from instructions the assembler encoded
[info] CoreTimingSpec:
[info] Core
[info] - should execute every RV32I, Zicsr and multiplication instruction in one cycle, and every trap in one
cycle
[info] CoreProgramSpec:
[info] Core
[info] - should run arithmetic and logic instructions (OP, OP-IMM, LUI, AUIPC)
[info] - should run loads and stores of every size, signed and unsigned
[info] - should run every branch both ways, loops, JAL and JALR (incl. clearing bit 0)
[info] - should handle back-to-back dependencies, including a load result used immediately
[info] - should run loops written in RV32I only: gcd by subtraction, a sum, and multiplication by shift-and-add
[info] - should execute MUL, MULH, MULHSU and MULHU, including the corner cases, in one cycle
[info] - should compute 12! with MUL in a loop
[info] - should still treat DIV, DIVU, REM and REMU as illegal instructions (cause 2, mtval = the instruction)
[info] - should implement Zicsr and take every exception precisely (illegal, ECALL, EBREAK, misaligned
load/store/fetch)
[info] - should match the reference model on 3000 random instructions
[info] RegFileSpec:
[info] RegFile
[info] - should hardwire x0 to zero and store x1..x31
[info] - should ignore writes when we is low and return the old value during a write
[info] MemorySpec:
[info] InstructionMemory
[info] - should map the program at 0x80000000 and read 0 outside it
[info] - should accept comments, blank lines and separators in hex files
[info] DataMemory
[info] - should start from data.hex, write bytes, halves and words under byte strobes, and ignore outside writes
[info] RV32ITestWrapper
[info] - should run program.hex on data.hex: a dot product with MUL, then MULH/MULHU/MULHSU corner cases
[info] Run completed in 37 seconds, 677 milliseconds.
[info] Total number of tests run: 34
[info] Suites: completed 10, aborted 0
[info] Tests: succeeded 34, failed 0, canceled 0, ignored 0, pending 0
[info] All tests passed.
[success] Total time: 41 s, completed 10 nov. 2026, 17:50:19
--------------------------------------------------------------------------------
```


From: 120 ns
To: 200 ns
Signals
Waves
Time
130 ns
140 ns
150 ns
160 ns
170 ns
Clock
clock =1
reset =0
Fetch / execute
pc[31:0] =8000001C
io_trace_inst[31:0] =mul x17, x15, x16
io_trace_inst[31:0] =030788B3
brTaken =0
ctlTaken =0
nextPc[31:0] =80000020
Multiplier (parallel, 1 cycle)
isMul =1
io_fn[1:0] =MUL
io_a[31:0] =2
io_b[31:0] =20
product[65:0] =00000000000000028
io_result[31:0] =40
Retire / trap
io_trace_valid =1
io_trace_trap =0
io_trace_cause[3:0] =0
io_trace_rdAddr[4:0] =17
io_trace_rdData[31:0] =40
Data memory
io_trace_memRe =0
io_trace_memWe =0
io_dmem_addr[31:0] =00000016
io_dmem_rdata[31:0] =00000000
io_dmem_wdata[31:0] =14141414
io_dmem_wstrb[3:0] =0000
Registers x1..x15
regs_1[31:0] =0
regs_2[31:0] =0
regs_3[31:0] =0
regs_4[31:0] =0
regs_5[31:0] =0
regs_6[31:0] =0
regs_7[31:0] =0
regs_8[31:0] =0
regs_9[31:0] =0
regs_10[31:0] =-2147418112
regs_11[31:0] =3
regs_12[31:0] =-2147418104
regs_13[31:0] =-2147418088
regs_14[31:0] =10
regs_15[31:0] =2
CSRs
mtvec[31:0] =00000000
mepc[31:0] =00000000

# GLab 5

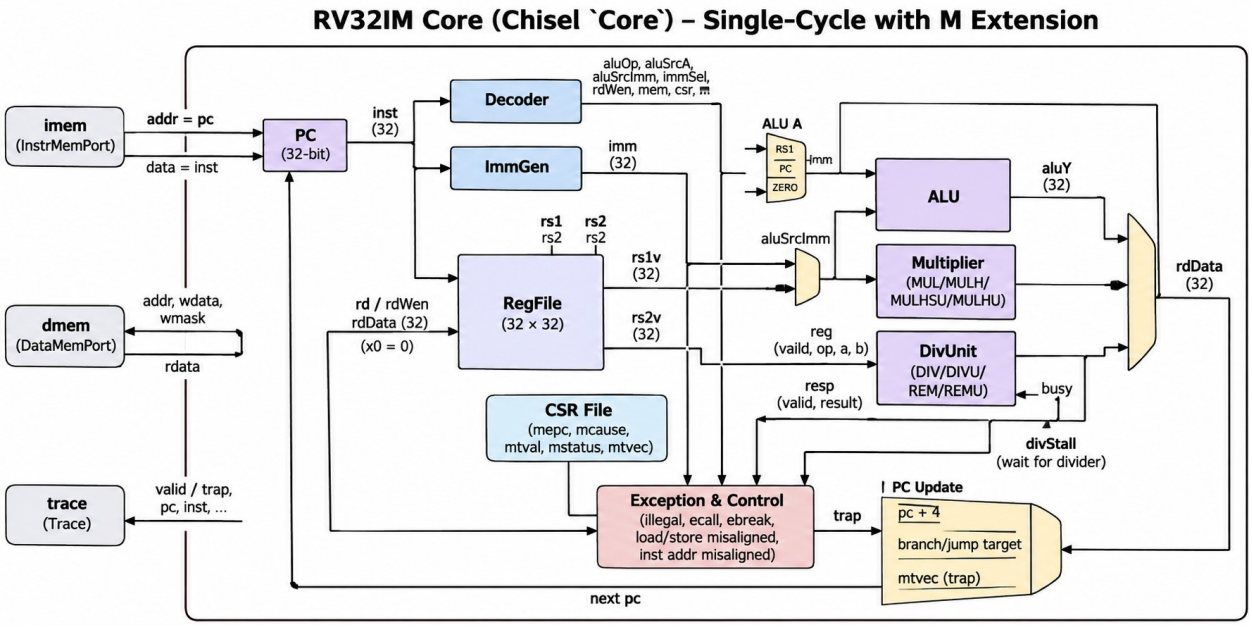
## Building full RV32IM architecture with sequential DIV/REM unit

The following architecture is extended with M instructions including combinational multiplier and sequential divider units. The following is the architectural wrapper containing the Core and data and instruction memories.

```
---------------------------------------------------------------------------------------------------
package rv32i
import chisel3._

/** Stage 3 system: Core + InstructionMemory (program at 0x80000000) + DataMemory RAM (at 0x80010000). */
class RV32ITestWrapper(programFile: String, dataFile: String = "", dataDepth: Int = 256) extends Module {
  val io = IO(new Bundle {
    val trace = Output(new Trace)
  })
  val core = Module(new Core())
  val imem = Module(new InstructionMemory(programFile))
  val dmem = Module(new DataMemory(dataFile, dataDepth))

  imem.io.addr       := core.io.imem.addr
  core.io.imem.data  := imem.io.inst
  dmem.io.addr       := core.io.dmem.addr
  dmem.io.we         := core.io.dmem.we
  dmem.io.wdata      := core.io.dmem.wdata
  dmem.io.wstrb      := core.io.dmem.wstrb
  core.io.dmem.rdata := dmem.io.rdata
  io.trace := core.io.trace
}
---------------------------------------------------------------------------------------------------
```




```
---------------------------------------------------------------------------------------------------
package rv32i

import chisel3._
import chisel3.util._

/** Stage 5: single-cycle RV32IM core: RV32I, the full M extension, Zicsr, machine mode only, no interrupts.
  *
  * Every cycle does exactly one of three things:
  *   - retires one instruction       (trace.valid)
  *   - takes one precise exception   (trace.trap)
  *   - waits for the divider         (neither: a DIV/DIVU/REM/REMU in progress; pc, registers, memory and
  *                                    CSRs do not change; the same instruction stays on the fetch port)
  * A trapping instruction changes no architectural state except the trap CSRs (mepc, mcause, mtval, mstatus)
  * and the pc, which becomes mtvec (direct mode).
  *
  * Stage 4: MUL, MULH, MULHSU, MULHU execute in one cycle on the combinational (parallel) `Multiplier`.
  * Stage 5: DIV, DIVU, REM, REMU execute on the sequential `DivUnit`: 1 extra cycle for divide-by-zero and
  *          signed overflow, 33 extra cycles otherwise; the instruction retires in the divider's response cycle.
  * All eight are decoded here (OP opcode, funct7 = 0000001), so the Stage 3 Decoder is unchanged and still
  * reports funct7 = 0000001 as illegal on its own.
```

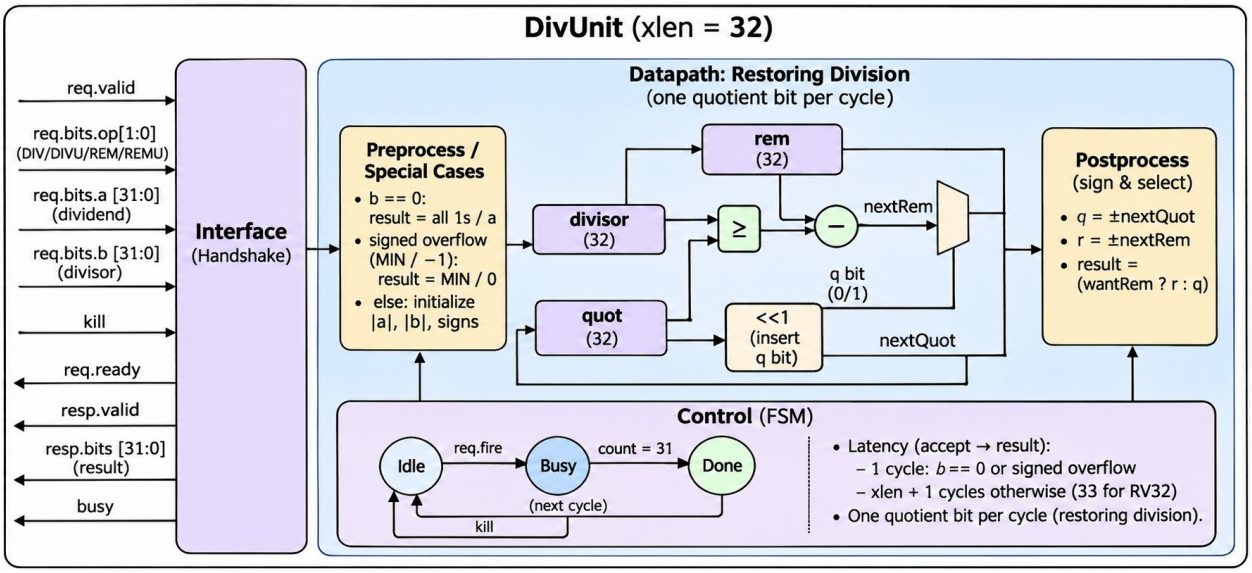
```
  *
  * Exceptions, in priority order: illegal instruction (2), ECALL (11), EBREAK (3), load misaligned (4),
  * store misaligned (6), instruction address misaligned (0: taken branch/JAL/JALR target with bit 1 set).
  * Misaligned data accesses trap (no hardware support). There are no access faults in this step.
  *
  * Memory ports are zero-wait: the instruction and data read data must be combinational functions of the
  * address. While the core waits for the divider it keeps `imem.addr` (the pc) unchanged.
  */
class Core(resetPc: Long = 0x80000000L, hartId: Int = 0) extends Module {
  val io = IO(new Bundle {
    val imem  = new InstrMemPort
    val dmem  = new DataMemPort
    val trace = Output(new Trace)
  })
  require(resetPc >= 0 && resetPc < (1L << 32) && resetPc % 4 == 0, "reset pc must be 4-byte aligned")
  val pc = RegInit(resetPc.U(32.W))
  io.imem.addr := pc
  val inst = io.imem.data
  val f3   = inst(14, 12)
  // ---------------- M extension: OP opcode with funct7 = 0000001 ----------------
  val isMulDiv = inst(6, 0) === "b0110011".U && inst(31, 25) === "b0000001".U
  val isMul    = isMulDiv && !f3(2)   // Stage 4: MUL, MULH, MULHSU, MULHU   (funct3 = 0xx)
  val isDiv    = isMulDiv && f3(2)    // Stage 5: DIV, DIVU, REM, REMU       (funct3 = 1xx)
  val dec  = Module(new Decoder)
  val immg = Module(new ImmGen)
  val rf   = Module(new RegFile)
  val alu  = Module(new Alu)
  val csr  = Module(new CsrFile(hartId))
  val mul  = Module(new Multiplier)
  val div  = Module(new DivUnit(32))
  dec.io.inst  := inst
  immg.io.inst := inst
  immg.io.sel  := dec.io.immSel
  // Register addresses straight from the instruction for M-extension ops (the decoder treats them as illegal).
  rf.io.rs1    := Mux(isMulDiv, inst(19, 15), dec.io.rs1)
  rf.io.rs2    := Mux(isMulDiv, inst(24, 20), dec.io.rs2)
  val rs1v = rf.io.rd1
  val rs2v = rf.io.rd2
  val imm  = immg.io.imm
  mul.io.fn := f3(1, 0)
  mul.io.a  := rs1v
  mul.io.b  := rs2v
  // Sequential divider. `busy` stays high through its response cycle, so holding the same instruction on the
  // fetch port never starts it twice. The divider latches rs1/rs2 when it accepts the request.
  div.io.req.valid   := isDiv && !div.io.busy
  div.io.req.bits.op := f3(1, 0)
  div.io.req.bits.a  := rs1v
  div.io.req.bits.b  := rs2v
  val divStall = isDiv && !div.io.resp.valid   // waiting: retire nothing this cycle
  // ---------------- execute ----------------
  val aluA = WireDefault(rs1v)
  switch(dec.io.aluSrcA) {
    is(ASrc.RS1)  { aluA := rs1v }
    is(ASrc.PC)   { aluA := pc }
    is(ASrc.ZERO) { aluA := 0.U }
  }
  alu.io.op := dec.io.aluOp
  alu.io.a  := aluA
  alu.io.b  := Mux(dec.io.aluSrcImm, imm, rs2v)
  val aluY = alu.io.y
  // branch comparator and target adders
  val eq  = rs1v === rs2v
  val lt  = rs1v.asSInt < rs2v.asSInt
  val ltu = rs1v < rs2v
  val brCond = WireDefault(false.B)
  switch(f3) {
    is("b000".U) { brCond := eq }
    is("b001".U) { brCond := !eq }
    is("b100".U) { brCond := lt }
    is("b101".U) { brCond := !lt }
    is("b110".U) { brCond := ltu }
    is("b111".U) { brCond := !ltu }
  }
  val brTaken    = dec.io.branch && brCond
  val pcPlus4    = pc + 4.U
  val brTarget   = pc + imm                          // branches and JAL
  val jalrTarget = Cat(aluY(31, 1), 0.U(1.W))        // rs1 + imm, bit 0 cleared
  val ctlTaken   = brTaken || dec.io.jal || dec.io.jalr
  val ctlTarget  = Mux(dec.io.jalr, jalrTarget, brTarget)
  // ---------------- data access ----------------
  val size = dec.io.memSize
  val misalignedAccess = (size === 2.U && aluY(1, 0) =/= 0.U) || (size === 1.U && aluY(0))
  val loadMis  = dec.io.memRead && misalignedAccess
  val storeMis = dec.io.memWrite && misalignedAccess
  val fetchMis = ctlTaken && ctlTarget(1)
  // ---------------- CSR / traps ----------------
  val rs1Field = inst(19, 15)
  csr.io.addr   := inst(31, 20)
  csr.io.en     := dec.io.csr
  csr.io.op     := f3(1, 0)
```

```
csr.io.src    := Mux(f3(2), Cat(0.U(27.W), rs1Field), rs1v)
csr.io.wrEn   := f3(1, 0) === 1.U || rs1Field =/= 0.U
// The eight M-extension instructions are legal even though the Stage 3 decoder flags funct7 = 0000001.
val illegal = (dec.io.illegal && !isMulDiv) || csr.io.illegal
val trapNow = illegal || dec.io.ecall || dec.io.ebreak || loadMis || storeMis || fetchMis
// A division never traps (it is legal and does not touch memory), but keep the divider consistent anyway.
div.io.kill := trapNow
val cause = PriorityMux(Seq(
  illegal         -> Cause.IllegalInstruction.U(4.W),
  dec.io.ecall    -> Cause.EcallFromM.U(4.W),
  dec.io.ebreak   -> Cause.Breakpoint.U(4.W),
  loadMis         -> Cause.LoadAddrMisaligned.U(4.W),
  storeMis        -> Cause.StoreAddrMisaligned.U(4.W),
  fetchMis        -> Cause.InstrAddrMisaligned.U(4.W)
))
val tval = PriorityMux(Seq(
  illegal                -> inst,
  dec.io.ecall           -> 0.U(32.W),
  dec.io.ebreak          -> pc,
  (loadMis || storeMis)  -> aluY,
  fetchMis               -> ctlTarget
))
val retire = !trapNow && !divStall
csr.io.commit    := retire           // no CSR side effects (e.g. minstret) while waiting for the divider
csr.io.trapEnter := trapNow
csr.io.trapCause := Cat(0.U(28.W), cause)
csr.io.trapPc    := pc
csr.io.trapVal   := tval
csr.io.mretEnter := dec.io.mret && !trapNow
// ---------------- next pc ----------------
val nextPc = Mux(trapNow, csr.io.mtvecBase,
             Mux(divStall, pc,
             Mux(dec.io.mret, csr.io.mepc,
             Mux(ctlTaken, ctlTarget, pcPlus4))))
pc := nextPc
// ---------------- load / store datapath ----------------
val rdata = io.dmem.rdata
val bytes = VecInit(Seq.tabulate(4)(k => rdata(8 * k + 7, 8 * k)))
val b     = bytes(aluY(1, 0))
val h     = Mux(aluY(1), rdata(31, 16), rdata(15, 0))
val uns   = dec.io.memUnsigned
val loadData = WireDefault(rdata)
switch(size) {
  is(0.U) { loadData := Mux(uns, Cat(0.U(24.W), b), Cat(Fill(24, b(7)), b)) }
  is(1.U) { loadData := Mux(uns, Cat(0.U(16.W), h), Cat(Fill(16, h(15)), h)) }
  is(2.U) { loadData := rdata }
}
val sbyte = rs2v(7, 0)
val shalf = rs2v(15, 0)
val wdata = WireDefault(rs2v)
val wstrb = WireDefault("b1111".U(4.W))
switch(size) {
  is(0.U) { wdata := Fill(4, sbyte); wstrb := (1.U(4.W) << aluY(1, 0))(3, 0) }
  is(1.U) { wdata := Fill(2, shalf); wstrb := Mux(aluY(1), "b1100".U(4.W), "b0011".U(4.W)) }
}
io.dmem.addr  := aluY
io.dmem.re    := dec.io.memRead && !trapNow
io.dmem.we    := dec.io.memWrite && !trapNow
io.dmem.wdata := wdata
io.dmem.wstrb := Mux(io.dmem.we, wstrb, 0.U)
// ---------------- write-back ----------------
val wbData = WireDefault(aluY)
switch(dec.io.wbSel) {
  is(WbSel.ALU) { wbData := aluY }
  is(WbSel.MEM) { wbData := loadData }
  is(WbSel.PC4) { wbData := pcPlus4 }
  is(WbSel.CSR) { wbData := csr.io.rdata }
}
when(isMul) { wbData := mul.io.result }            // last connect wins
when(isDiv) { wbData := div.io.resp.bits }
val rd = Mux(isMulDiv, inst(11, 7), dec.io.rd)
rf.io.we    := (dec.io.regWrite || isMulDiv) && retire
rf.io.waddr := rd
rf.io.wdata := wbData
val writes = rf.io.we && rd =/= 0.U
// ---------------- trace ----------------
io.trace.valid    := retire
io.trace.trap     := trapNow
io.trace.cause    := cause
io.trace.pc       := pc
io.trace.pcNext   := nextPc
io.trace.inst     := inst
io.trace.rdAddr   := Mux(writes, rd, 0.U)
io.trace.rdData   := Mux(writes, wbData, 0.U)
io.trace.memAddr  := aluY
io.trace.memRe    := io.dmem.re
io.trace.memWe    := io.dmem.we
io.trace.memSize  := size
io.trace.memWdata := Mux(size === 0.U, Cat(0.U(24.W), sbyte), Mux(size === 1.U, Cat(0.U(16.W), shalf), rs2v))
```

```
  // ---------------- assertions (compiled out of synthesis builds) ----------------
  assert(pc(1, 0) === 0.U, "pc must stay 4-byte aligned")
  assert(!(io.trace.valid && io.trace.trap), "retire and trap in the same cycle")
  assert(!(io.dmem.we && io.dmem.re), "simultaneous load and store")
  assert(!(trapNow && (rf.io.we || io.dmem.we)), "trapping instruction modifies state")
  assert(!(divStall && (rf.io.we || io.dmem.we || io.dmem.re || trapNow)), "state changed while waiting for the
divider")
  assert(!(divStall && nextPc =/= pc), "pc moved while waiting for the divider")
}
```

---

## DivUnit – Sequential Divider (RV32M)




---

```
package rv32i

import chisel3._
import chisel3.util._

/** One division request. `op` is funct3(1,0) of the instruction:
  * 0 = DIV, 1 = DIVU, 2 = REM, 3 = REMU (funct3 = 100, 101, 110, 111).
  */
class DivReq(xlen: Int) extends Bundle {
  val op = UInt(2.W)
  val a  = UInt(xlen.W) // rs1: dividend
  val b  = UInt(xlen.W) // rs2: divisor
}

/** Sequential divider for DIV, DIVU, REM and REMU (restoring division, one quotient bit per cycle).
  *
  * Handshake
  *  - `req` is accepted (req.fire) only while the unit is idle (`req.ready`).
  *  - `resp.valid` is high for exactly one cycle with the result; the unit is back to idle on the
  *    next cycle. `req.ready` is low during that response cycle, so a stalled core that keeps
  *    `req.valid` high on the same instruction cannot start it twice.
  *  - `kill` abandons a division in progress (for a trap or interrupt taken while the core is
  *    stalled). Division has no side effects, so the instruction can simply be re-executed later.
  *
  * Latency, counted in clock edges from the accepting edge to the response cycle
  *  - 1 for divide-by-zero and for signed overflow (most negative value / -1)
  *  - xlen + 1 otherwise (33 for RV32)
  *
  * Results follow the RISC-V spec (unprivileged ISA, M extension, division table):
  *  - x / 0: quotient = all ones, remainder = x
  *  - signed overflow MIN / -1: quotient = MIN, remainder = 0
  *  - quotient rounds toward zero; the remainder takes the sign of the dividend.
  */
class DivUnit(xlen: Int = 32) extends Module {
  require(xlen >= 2)

  val io = IO(new Bundle {
    val req  = Flipped(Decoupled(new DivReq(xlen)))
    val resp = Valid(UInt(xlen.W))
    val kill = Input(Bool())
    val busy = Output(Bool()) // high while busy or presenting the response
  })
  private val sIdle :: sBusy :: sDone :: Nil = Enum(3)
  private val state = RegInit(sIdle)
  private val minInt  = (BigInt(1) << (xlen - 1)).U(xlen.W)
  private val allOnes = ((BigInt(1) << xlen) - 1).U(xlen.W)
  private val quot    = Reg(UInt(xlen.W)) // dividend magnitude shifts out (MSB first) as quotient bits shift in
  private val rem     = Reg(UInt(xlen.W)) // partial remainder, always < divisor
  private val divisor = Reg(UInt(xlen.W)) // divisor magnitude
```

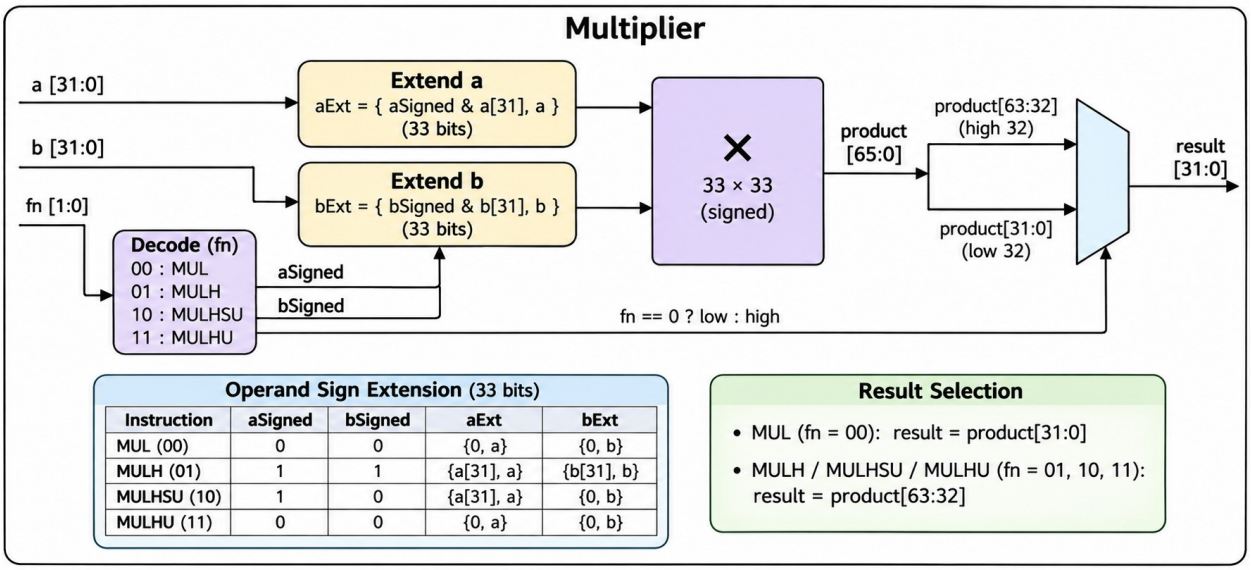
```
  private val count   = Reg(UInt(log2Ceil(xlen).W))
  private val negQ    = Reg(Bool())
  private val negR    = Reg(Bool())
  private val wantRem = Reg(Bool())
  private val result  = Reg(UInt(xlen.W))
  io.req.ready  := state === sIdle
  io.resp.valid := state === sDone
  io.resp.bits  := result
  io.busy       := state =/= sIdle
  // ------------------------------------------------------------------ accept a request
  when(io.req.fire) {
    val a      = io.req.bits.a
    val b      = io.req.bits.b
    val signed = !io.req.bits.op(0)
    val isRem  = io.req.bits.op(1)
    val negA   = signed && a(xlen - 1)
    val negB   = signed && b(xlen - 1)
    wantRem := isRem
    negQ    := negA =/= negB
    negR    := negA
    quot    := Mux(negA, 0.U(xlen.W) - a, a)
    divisor := Mux(negB, 0.U(xlen.W) - b, b)
    rem     := 0.U
    count   := 0.U
    when(b === 0.U) {
      result := Mux(isRem, a, allOnes)
      state  := sDone
    }.elsewhen(signed && a === minInt && b === allOnes) {
      result := Mux(isRem, 0.U(xlen.W), minInt)
      state  := sDone
    }.otherwise {
      state := sBusy
    }
  }
  // ------------------------------------------------------------------ one quotient bit per cycle
  when(state === sBusy) {
    val shifted  = Cat(rem, quot(xlen - 1))      // xlen + 1 bits
    val fits     = shifted >= divisor
    val nextRem  = Mux(fits, shifted - divisor, shifted)(xlen - 1, 0)
    val nextQuot = Cat(quot(xlen - 2, 0), fits)
    rem   := nextRem
    quot  := nextQuot
    count := count + 1.U
    when(count === (xlen - 1).U) {
      val q = Mux(negQ, 0.U(xlen.W) - nextQuot, nextQuot)
      val r = Mux(negR, 0.U(xlen.W) - nextRem, nextRem)
      result := Mux(wantRem, r, q)
      state  := sDone
    }
  }
  when(state === sDone) {
    state := sIdle
  }
  when(io.kill) {
    state := sIdle
  }
}

/** Software reference for the four division instructions, for tests and for RefModel.
  * Values are unsigned xlen-bit numbers held in BigInt; `op` is funct3(1,0) as in [[DivReq]].
  */
object DivRef {
  def apply(op: Int, a: BigInt, b: BigInt, xlen: Int = 32): BigInt = {
    val mask   = (BigInt(1) << xlen) - 1
    val min    = BigInt(1) << (xlen - 1)
    def toS(v: BigInt): BigInt = if (v >= min) v - (BigInt(1) << xlen) else v
    val signed = (op & 1) == 0
    val isRem  = (op & 2) != 0
    if (b == 0) {
      if (isRem) a else mask
    } else if (signed) {
      val sa = toS(a)
      val sb = toS(b)
      if (sa == -min && sb == -1) {
        if (isRem) BigInt(0) else min
      } else {
        // BigInt division truncates toward zero and the remainder takes the dividend's sign,
        // which is exactly the RISC-V rule.
        (if (isRem) sa % sb else sa / sb) & mask
      }
    } else {
      if (isRem) a % b else a / b
    }
  }
  /** RV32 convenience form for models that keep registers in Int. */
  def int32(op: Int, a: Int, b: Int): Int =
    apply(op, BigInt(a & 0xFFFFFFFFL), BigInt(b & 0xFFFFFFFFL)).toInt
}
```

---

---

## Multiplier – Combinational Multiplier (RV32M, Step 4)




---

```
package rv32i
import chisel3._
import chisel3.util._

/** Step 4: combinational ("parallel") multiplier for MUL, MULH, MULHSU, MULHU (fn = funct3(1,0)).
  *
  * One shared (33 x 33) signed multiplier: each operand is extended by one bit, with its sign bit for
  * a signed operand (MULH: both, MULHSU: rs1) and with 0 otherwise. The low 64 bits of that product are
  * the exact 64-bit result of every variant, so one array serves all four instructions.
  *   fn 0 MUL    -> product[31:0]  (same bits for signed and unsigned)
  *   fn 1 MULH   -> product[63:32] signed   x signed
  *   fn 2 MULHSU -> product[63:32] signed   x unsigned
  *   fn 3 MULHU  -> product[63:32] unsigned x unsigned
  */
class Multiplier extends Module {
  val io = IO(new Bundle {
    val fn     = Input(UInt(2.W))
    val a      = Input(UInt(32.W))   // rs1
    val b      = Input(UInt(32.W))   // rs2
    val result = Output(UInt(32.W))
  })

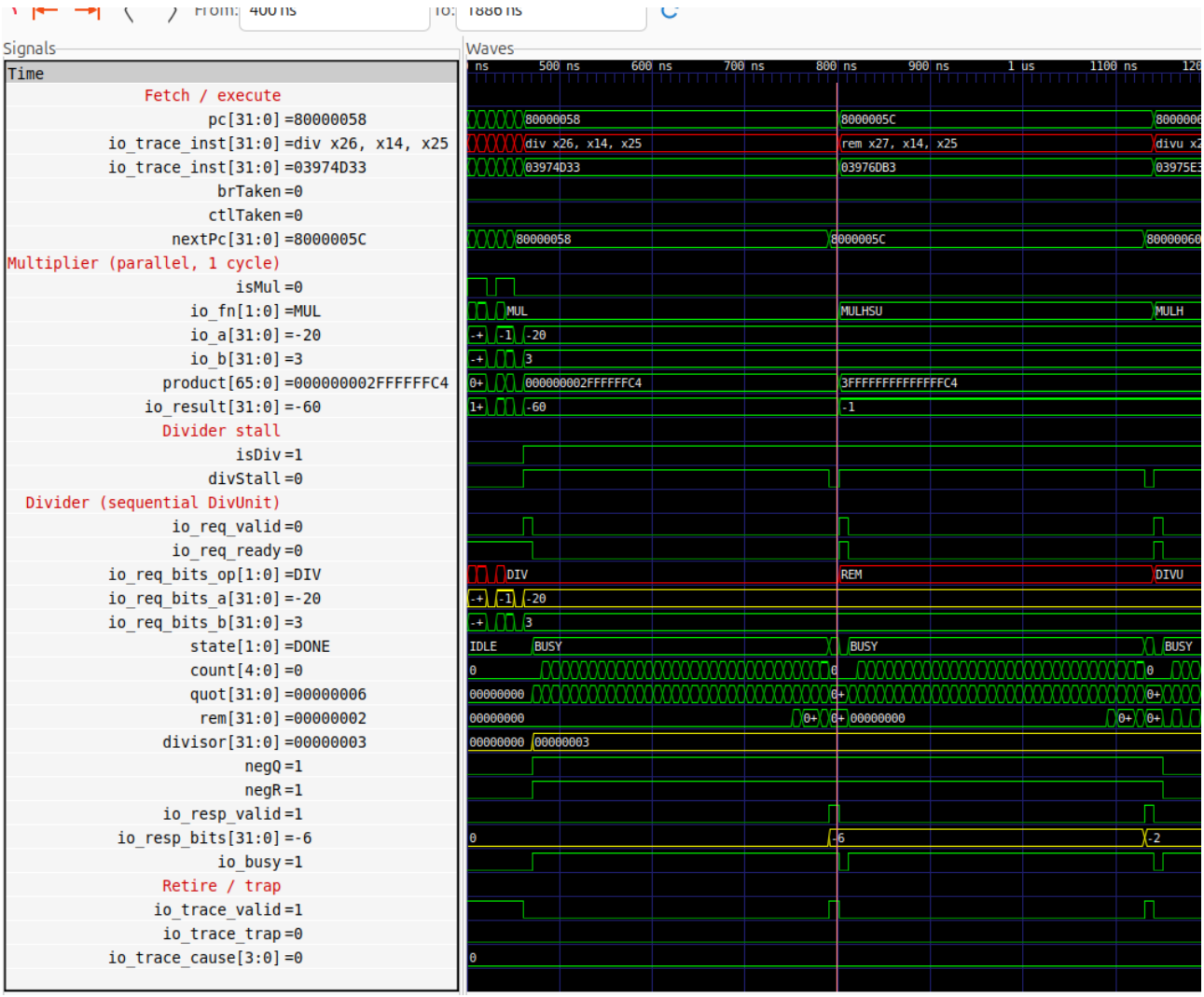
  val aSigned = io.fn === 1.U || io.fn === 2.U        // MULH, MULHSU
  val bSigned = io.fn === 1.U                         // MULH
  val aExt    = Cat(aSigned && io.a(31), io.a).asSInt // 33 bits
  val bExt    = Cat(bSigned && io.b(31), io.b).asSInt
  val product = (aExt * bExt).asUInt                  // 66 bits, low 64 are exact

  io.result := Mux(io.fn === 0.U, product(31, 0), product(63, 32))
}
```

---

```
$ sbt test
[info] welcome to sbt 1.10.2 (Ubuntu Java 17.0.20.1)
[info] loading project definition from /media/bako/0969-2ABE/books_2026/CL.Book.Chisel.2026/RV32.Steps/rv32im-
stage5/project
[info] loading settings for project root from build.sbt ...
[info] set current project to rv32im-stage5 (in build
file:/media/bako/0969-2ABE/books_2026/CL.Book.Chisel.2026/RV32.Steps/rv32im-stage5/)
[info] DecoderSpec:
[info] Decoder
[info] - should decode every RV32I, Zicsr and M-mode SYSTEM instruction
[info] - should flag illegal encodings and assert no other control signal for them
[info] - should classify every one of the 128 major opcodes
[info] AluSpec:
[info] Alu
[info] - should compute every operation on all edge-value pairs (incl. overflow, shift by 31/32, sign)
[info] - should match the reference on random operands
[info] WaveformSpec:
[info] Waveforms
[info] - should show the Stage 5 demo (program.hex + data.hex) on RV32ITestWrapper (waves/stage5_demo)
[info] - should show LUI, AUIPC, JAL and all six branches (waves/upper_jumps)
[info] - should show loads, stores, a loop and a call (waves/load_store_branch)
[info] - should show the four multiplications on corner values, each in one cycle (waves/mul)
[info] - should show DIV/REM waiting on the sequential divider and MUL in one cycle (waves/div_rem)
[info] - should show the divider on its own: request, 32 iterations, response (waves/div_unit)
```

```
[info] - should show an ECALL, an illegal instruction (FENCE.I), the handler and MRET (waves/trap)
[info] DivUnitSpec:
[info] DivUnit
[info] - should handle every edge-value pair for all four instructions
[info] - should match the reference on random operands
[info] - should abandon a division on kill and accept the next one
[info] - should run back-to-back requests from a core that holds valid high
[info] CsrFileSpec:
[info] CsrFile
[info] - should implement CSRRW / CSRRS / CSRRC semantics
[info] - should report misa = RV32IM, mhartid and fixed values, and apply WARL masks
[info] - should flag unknown CSRs and writes to read-only CSRs as illegal, and not write them
[info] - should count cycles always and instructions on commit, and let a write replace one half
[info] - should save state on trap entry and restore it on mret
[info] CoreDivSpec:
[info] Core
[info] - should execute DIV, DIVU, REM and REMU including the spec's corner cases
[info] - should use a division result in the next instruction and ignore rd = x0
[info] - should still execute the Step 4 multiplications in one cycle
[info] MultiplierSpec:
[info] Multiplier
[info] - should compute MUL, MULH, MULHSU and MULHU on edge values and random operands
[info] ImmGenSpec:
[info] ImmGen
[info] - should build the I, S, B, U and J immediates from instructions the assembler encoded
[info] CoreTimingSpec:
[info] Core
[info] - should execute every instruction in one cycle except DIV, DIVU, REM and REMU
[info] CoreProgramSpec:
[info] Core
[info] - should run arithmetic and logic instructions (OP, OP-IMM, LUI, AUIPC)
[info] - should run loads and stores of every size, signed and unsigned
[info] - should run every branch both ways, loops, JAL and JALR (incl. clearing bit 0)
[info] - should handle back-to-back dependencies, including a load result used immediately
[info] - should run loops written in RV32I only: gcd by subtraction, a sum, and multiplication by shift-and-add
[info] - should execute MUL, MULH, MULHSU and MULHU, including the corner cases, in one cycle
[info] - should compute 12! with MUL in a loop
[info] - should execute DIV, DIVU, REM and REMU (sequential divider), including the spec's corner cases
[info] - should compute gcd(1071, 462) with REM and use MUL and DIV results immediately
[info] - should implement Zicsr and take every exception precisely (illegal, ECALL, EBREAK, misaligned
load/store/fetch)
[info] - should match the reference model on 3000 random instructions
[info] DivRefSpec:
[info] DivRef
[info] - should follow the RISC-V division-by-zero and overflow table
[info] - should round toward zero and give the remainder the dividend's sign
[info] - should agree with the Int form
[info] RegFileSpec:
[info] RegFile
[info] - should hardwire x0 to zero and store x1..x31
[info] - should ignore writes when we is low and return the old value during a write
[info] MemorySpec:
[info] InstructionMemory
[info] - should map the program at 0x80000000 and read 0 outside it
[info] - should accept comments, blank lines and separators in hex files
[info] DataMemory
[info] - should start from data.hex, write bytes, halves and words under byte strobes, and ignore outside writes
[info] RV32ITestWrapper
[info] - should run program.hex on data.hex: dot product with MUL, MULH/MULHU/MULHSU, DIV/REM/DIVU/REMU
[info] Run completed in 1 minute, 22 seconds.
[info] Total number of tests run: 47
[info] Suites: completed 13, aborted 0
[info] Tests: succeeded 47, failed 0, canceled 0, ignored 0, pending 0
[info] All tests passed.
[success] Total time: 86 s (01:26), completed 11 nov. 2026, 09:56:18
------------------------------------------------------------------------------------------
```


From: 400 ns
To: 1886 ns
Signals
Waves
Time
Fetch / execute
pc[31:0] =80000058
io_trace_inst[31:0] =div x26, x14, x25
io_trace_inst[31:0] =03974D33
brTaken =0
ctlTaken =0
nextPc[31:0] =8000005C
Multiplier (parallel, 1 cycle)
isMul =0
io_fn[1:0] =MUL
io_a[31:0] =-20
io_b[31:0] =3
product[65:0] =000000002FFFFFC4
io_result[31:0] =-60
Divider stall
isDiv =1
divStall =0
Divider (sequential DivUnit)
io_req_valid =0
io_req_ready =0
io_req_bits_op[1:0] =DIV
io_req_bits_a[31:0] =-20
io_req_bits_b[31:0] =3
state[1:0] =DONE
count[4:0] =0
quot[31:0] =00000006
rem[31:0] =00000002
divisor[31:0] =00000003
negQ =1
negR =1
io_resp_valid =1
io_resp_bits[31:0] =-6
io_busy =1
Retire / trap
io_trace_valid =1
io_trace_trap =0
io_trace_cause[3:0] =0
500 ns
600 ns
700 ns
800 ns
900 ns
1 us
1100 ns
80000058
8000005C
div x26, x14, x25
rem x27, x14, x25
divu x2
03974D33
03976DB3
03975E3
80000058
8000005C
80000060
MUL
MULHSU
MULH
-1
-20
3
000000002FFFFFC4
3FFFFFFFFFFFFFC4
-60
-1
DIV
REM
DIVU
IDLE
BUSY
00000000
00000003
-6
-2

# GLab 6

## Building full RV32IM architecture with 2-stage pipeline

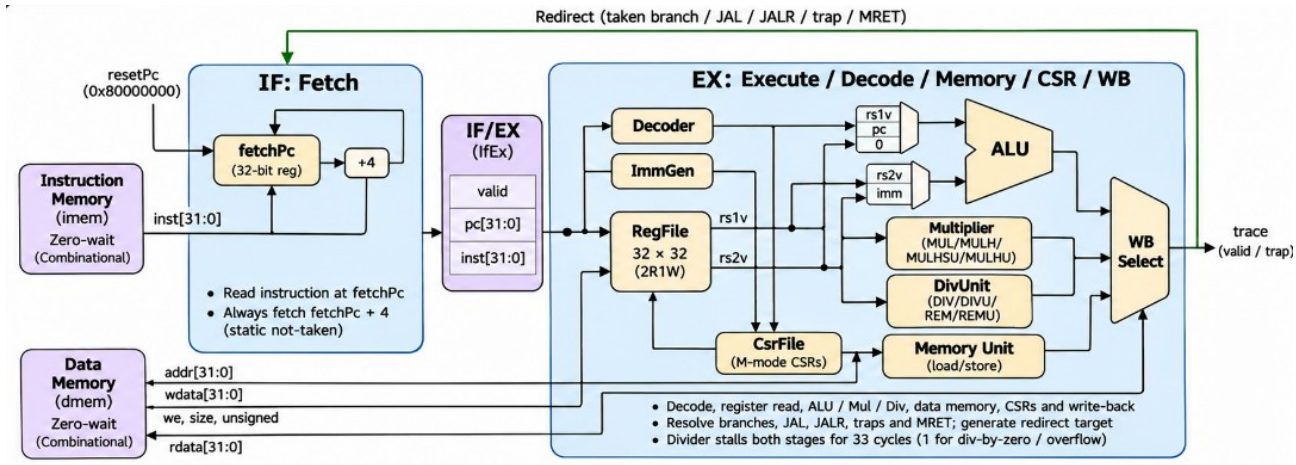
Here we have the 2-stage pipelined RV32IM project (fetch / execute),

**How the pipeline works**

- **IF (fetch):** a **`fetchPc`** register reads the instruction memory. The instruction and its PC are stored in the new `IfEx` pipeline register, which also has a **`valid`** bit.
- **EX (execute):** decode, register read, ALU, multiplier, divider, data memory, CSRs and write-back. This logic is the same as in the single-cycle core; it now works on the instruction held in **`IfEx`**.
- **Data hazards:** none. Registers are read and written in the same stage (EX), so the next instruction always reads the value just written. No forwarding is needed.
- **Branches:** predicted not taken; fetch always continues at PC+4. A taken branch, **`jal, jalr`**, a trap or **`mret`** is resolved in EX. The instruction already fetched behind it is discarded and fetch restarts at the target, costing 1 cycle. A not-taken branch costs nothing.
- **DIV/REM:** still sequential. Both stages freeze while the divider works.
- **Empty pipeline slots ("bubbles"):** one after reset and one after each redirect. A bubble changes nothing, and the core checks this with an assertion.

**What changed compared with the single-cycle project:** **`Core.scala`** (the two stages and the control logic) and **`Defs.scala`** (the **`IfEx Bundle`**). The decoder, ALU, register file, CSR file, multiplier and divider are unchanged.

**Cycles per instruction:** 1, plus 1 after a taken branch, jump, trap or `mret`, plus 33 for a regular division (1 for divide-by-zero or overflow).




```
--------------------------------------------------------------------------------------------------
package rv32i
import chisel3._
import chisel3.util._

/** 2-stage pipelined RV32IM core: IF (fetch) and EX (everything else), machine mode only, no interrupts.
  *
  * Pipeline
  *  - IF: `fetchPc` addresses the instruction memory; the word and its pc are captured in the [[IfEx]]
  *    pipeline register (with a valid bit) at the clock edge. Fetch always continues at fetchPc + 4
  *    (static not-taken prediction).
  *  - EX: decode, register read, ALU / multiplier / divider, data memory, CSRs and write-back, exactly as in the
  *    Step 5 single-cycle core, applied to the instruction held in IfEx.
  *
  * Hazards
  *  - Data: none. Registers are read and written in the same stage, so a result written at the end of one
  *    EX cycle is in the register file when the next instruction reads it. No forwarding, no interlock.
  *  - Control: a taken branch, JAL, JALR, a trap or MRET is resolved in EX. The instruction fetched behind it
  *    (wrong path) is discarded: IfEx becomes a bubble and fetch restarts at the target. Cost: 1 cycle.
  *  - Structural: DIV/DIVU/REM/REMU occupy the sequential divider for 33 extra cycles (1 for divide-by-zero and
  *    signed overflow). Both stages freeze meanwhile: fetchPc and IfEx keep their values.
  *
  * Every cycle EX does exactly one of: retire an instruction (trace.valid), take a trap (trace.trap), or nothing
  * (both low): a bubble after reset or after a redirect, or a cycle waiting for the divider. A bubble or a wait
  * changes no architectural state.
  *
  * Exceptions, in priority order: illegal instruction (2), ECALL (11), EBREAK (3), load misaligned (4),
  * store misaligned (6), instruction address misaligned (0: taken branch/JAL/JALR target with bit 1 set).
  *
  * Memory ports are zero-wait: instruction and data read data must be combinational functions of the address.
  */
class Core(resetPc: Long = 0x80000000L, hartId: Int = 0) extends Module {
```

```
val io = IO(new Bundle {
  val imem  = new InstrMemPort
  val dmem  = new DataMemPort
  val trace = Output(new Trace)
})

require(resetPc >= 0 && resetPc < (1L << 32) && resetPc % 4 == 0, "reset pc must be 4-byte aligned")

// ================= IF stage =================
val fetchPc = RegInit(resetPc.U(32.W))
io.imem.addr := fetchPc

/** IF/EX pipeline register: empty (bubble) after reset. */
val ifex = RegInit(0.U.asTypeOf(new IfEx))

// ================= EX stage =================
val exValid = ifex.valid
val pc      = ifex.pc
// A bubble executes as `addi x0, x0, 0`, which has no side effect; `retire` is also gated by exValid.
val inst    = Mux(exValid, ifex.inst, Core.Nop.U(32.W))
val f3      = inst(14, 12)

// ---------------- M extension: OP opcode with funct7 = 0000001 ----------------
val isMulDiv = inst(6, 0) === "b0110011".U && inst(31, 25) === "b0000001".U
val isMul    = isMulDiv && !f3(2)    // Step 4: MUL, MULH, MULHSU, MULHU   (funct3 = 0xx)
val isDiv    = isMulDiv && f3(2)     // Step 5: DIV, DIVU, REM, REMU       (funct3 = 1xx)

val dec  = Module(new Decoder)
val immg = Module(new ImmGen)
val rf   = Module(new RegFile)
val alu  = Module(new Alu)
val csr  = Module(new CsrFile(hartId))
val mul  = Module(new Multiplier)
val div  = Module(new DivUnit(32))

dec.io.inst  := inst
immg.io.inst := inst
immg.io.sel  := dec.io.immSel
// Register addresses straight from the instruction for M-extension ops (the decoder treats them as illegal).
rf.io.rs1    := Mux(isMulDiv, inst(19, 15), dec.io.rs1)
rf.io.rs2    := Mux(isMulDiv, inst(24, 20), dec.io.rs2)
val rs1v = rf.io.rd1
val rs2v = rf.io.rd2
val imm  = immg.io.imm

mul.io.fn := f3(1, 0)
mul.io.a  := rs1v
mul.io.b  := rs2v

// Sequential divider. `busy` stays high through its response cycle, so holding the same instruction on the
// fetch port never starts it twice. The divider latches rs1/rs2 when it accepts the request.
div.io.req.valid   := isDiv && !div.io.busy
div.io.req.bits.op := f3(1, 0)
div.io.req.bits.a  := rs1v
div.io.req.bits.b  := rs2v
val divStall = isDiv && !div.io.resp.valid   // waiting: retire nothing this cycle

// ---------------- execute ----------------
val aluA = WireDefault(rs1v)
switch(dec.io.aluSrcA) {
  is(ASrc.RS1)  { aluA := rs1v }
  is(ASrc.PC)   { aluA := pc }
  is(ASrc.ZERO) { aluA := 0.U }
}
alu.io.op := dec.io.aluOp
alu.io.a  := aluA
alu.io.b  := Mux(dec.io.aluSrcImm, imm, rs2v)
val aluY = alu.io.y

// branch comparator and target adders
val eq  = rs1v === rs2v
val lt  = rs1v.asSInt < rs2v.asSInt
val ltu = rs1v < rs2v
val brCond = WireDefault(false.B)
switch(f3) {
  is("b000".U) { brCond := eq }
  is("b001".U) { brCond := !eq }
  is("b100".U) { brCond := lt }
  is("b101".U) { brCond := !lt }
  is("b110".U) { brCond := ltu }
  is("b111".U) { brCond := !ltu }
}
val brTaken    = dec.io.branch && brCond
val pcPlus4    = pc + 4.U
val brTarget   = pc + imm                           // branches and JAL
val jalrTarget = Cat(aluY(31, 1), 0.U(1.W))         // rs1 + imm, bit 0 cleared
val ctlTaken   = brTaken || dec.io.jal || dec.io.jalr
val ctlTarget  = Mux(dec.io.jalr, jalrTarget, brTarget)

// ---------------- data access ----------------
```

```
val size = dec.io.memSize
val misalignedAccess = (size === 2.U && aluY(1, 0) =/= 0.U) || (size === 1.U && aluY(0))
val loadMis  = dec.io.memRead && misalignedAccess
val storeMis = dec.io.memWrite && misalignedAccess
val fetchMis = ctlTaken && ctlTarget(1)

// ---------------- CSR / traps ----------------
val rs1Field = inst(19, 15)
csr.io.addr   := inst(31, 20)
csr.io.en     := dec.io.csr
csr.io.op     := f3(1, 0)
csr.io.src    := Mux(f3(2), Cat(0.U(27.W), rs1Field), rs1v)
csr.io.wrEn   := f3(1, 0) === 1.U || rs1Field =/= 0.U

// The eight M-extension instructions are legal even though the Step 3 decoder flags funct7 = 0000001.
val illegal = (dec.io.illegal && !isMulDiv) || csr.io.illegal
val trapNow = illegal || dec.io.ecall || dec.io.ebreak || loadMis || storeMis || fetchMis

// A division never traps (it is legal and does not touch memory), but keep the divider consistent anyway.
div.io.kill := trapNow

val cause = PriorityMux(Seq(
  illegal         -> Cause.IllegalInstruction.U(4.W),
  dec.io.ecall    -> Cause.EcallFromM.U(4.W),
  dec.io.ebreak   -> Cause.Breakpoint.U(4.W),
  loadMis         -> Cause.LoadAddrMisaligned.U(4.W),
  storeMis        -> Cause.StoreAddrMisaligned.U(4.W),
  fetchMis        -> Cause.InstrAddrMisaligned.U(4.W)
))
val tval = PriorityMux(Seq(
  illegal                -> inst,
  dec.io.ecall           -> 0.U(32.W),
  dec.io.ebreak          -> pc,
  (loadMis || storeMis)  -> aluY,
  fetchMis               -> ctlTarget
))

val retire = exValid && !trapNow && !divStall

csr.io.commit    := retire           // no CSR side effects (e.g. minstret) while waiting for the divider
csr.io.trapEnter := trapNow
csr.io.trapCause := Cat(0.U(28.W), cause)
csr.io.trapPc    := pc
csr.io.trapVal   := tval
csr.io.mretEnter := dec.io.mret && !trapNow

// ---------------- next pc (architectural successor of the EX instruction) ----------------
val nextPc = Mux(trapNow, csr.io.mtvecBase,
             Mux(divStall, pc,
             Mux(dec.io.mret, csr.io.mepc,
             Mux(ctlTaken, ctlTarget, pcPlus4))))

// ---------------- pipeline control ----------------
// Redirect: the EX instruction changes the control flow, so the instruction in IF is on the wrong path.
val redirect = exValid && !divStall && (trapNow || dec.io.mret || ctlTaken)
when(divStall) {
  // freeze both stages: the divider is working on the EX instruction
}.elsewhen(redirect) {
  fetchPc    := nextPc
  ifex.valid := false.B                // flush the wrong-path instruction
}.otherwise {
  fetchPc    := fetchPc + 4.U
  ifex.valid := true.B
  ifex.pc    := fetchPc
  ifex.inst  := io.imem.data
}

// ---------------- load / store datapath ----------------
val rdata = io.dmem.rdata
val bytes = VecInit(Seq.tabulate(4)(k => rdata(8 * k + 7, 8 * k)))
val b     = bytes(aluY(1, 0))
val h     = Mux(aluY(1), rdata(31, 16), rdata(15, 0))
val uns   = dec.io.memUnsigned
val loadData = WireDefault(rdata)
switch(size) {
  is(0.U) { loadData := Mux(uns, Cat(0.U(24.W), b), Cat(Fill(24, b(7)), b)) }
  is(1.U) { loadData := Mux(uns, Cat(0.U(16.W), h), Cat(Fill(16, h(15)), h)) }
  is(2.U) { loadData := rdata }
}

val sbyte = rs2v(7, 0)
val shalf = rs2v(15, 0)
val wdata = WireDefault(rs2v)
val wstrb = WireDefault("b1111".U(4.W))
switch(size) {
  is(0.U) { wdata := Fill(4, sbyte); wstrb := (1.U(4.W) << aluY(1, 0))(3, 0) }
  is(1.U) { wdata := Fill(2, shalf); wstrb := Mux(aluY(1), "b1100".U(4.W), "b0011".U(4.W)) }
}

io.dmem.addr  := aluY
```

```
  io.dmem.re    := dec.io.memRead && !trapNow
  io.dmem.we    := dec.io.memWrite && !trapNow
  io.dmem.wdata := wdata
  io.dmem.wstrb := Mux(io.dmem.we, wstrb, 0.U)

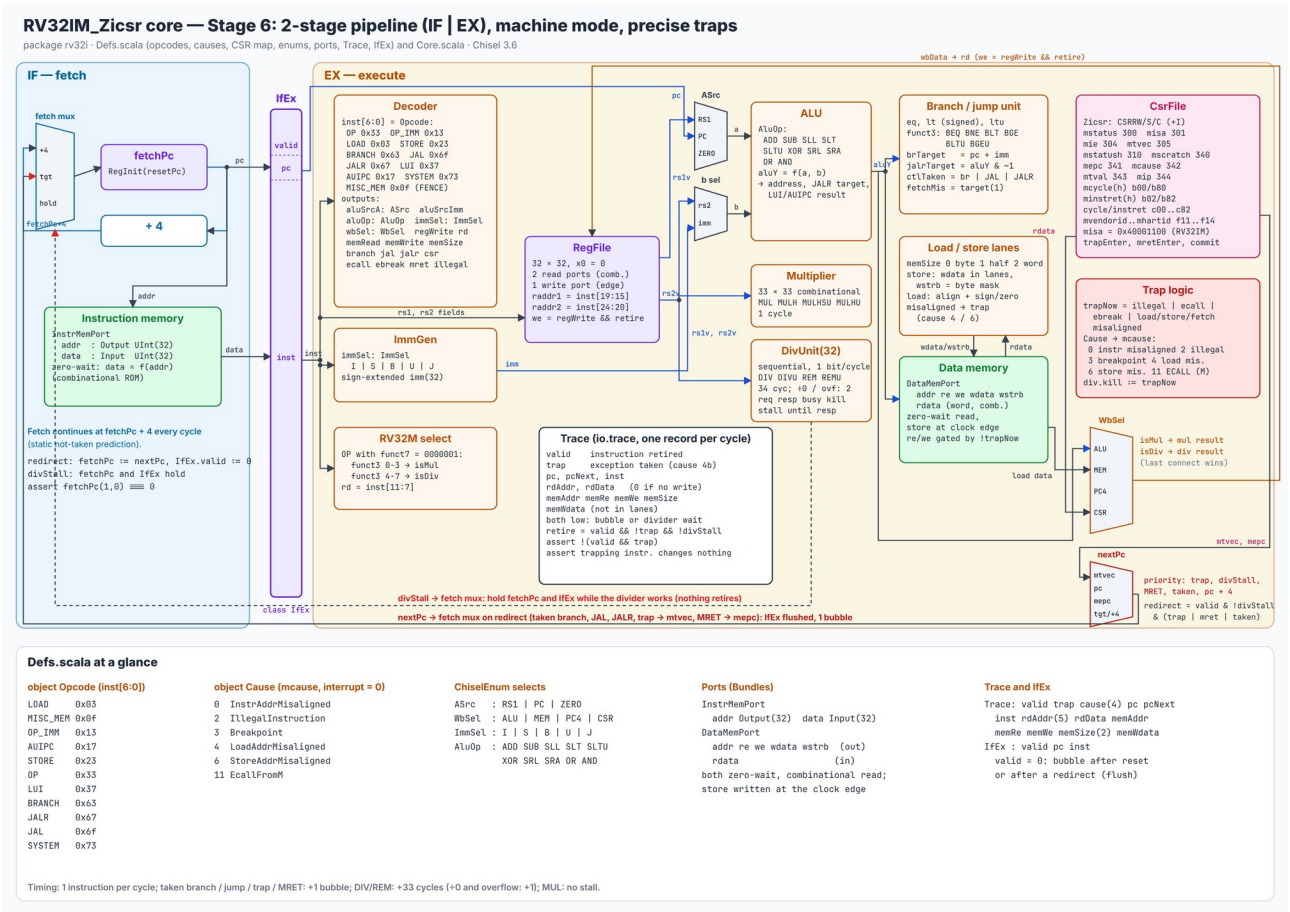
  // ---------------- write-back ----------------
  val wbData = WireDefault(aluY)
  switch(dec.io.wbSel) {
    is(WbSel.ALU) { wbData := aluY }
    is(WbSel.MEM) { wbData := loadData }
    is(WbSel.PC4) { wbData := pcPlus4 }
    is(WbSel.CSR) { wbData := csr.io.rdata }
  }
  when(isMul) { wbData := mul.io.result }            // last connect wins
  when(isDiv) { wbData := div.io.resp.bits }
  val rd = Mux(isMulDiv, inst(11, 7), dec.io.rd)
  rf.io.we    := (dec.io.regWrite || isMulDiv) && retire
  rf.io.waddr := rd
  rf.io.wdata := wbData
  val writes = rf.io.we && rd =/= 0.U

  // ---------------- trace ----------------
  io.trace.valid    := retire
  io.trace.trap     := trapNow
  io.trace.cause    := cause
  io.trace.pc       := pc
  io.trace.pcNext   := nextPc
  io.trace.inst     := inst
  io.trace.rdAddr   := Mux(writes, rd, 0.U)
  io.trace.rdData   := Mux(writes, wbData, 0.U)
  io.trace.memAddr  := aluY
  io.trace.memRe    := io.dmem.re
  io.trace.memWe    := io.dmem.we
  io.trace.memSize  := size
  io.trace.memWdata := Mux(size === 0.U, Cat(0.U(24.W), sbyte), Mux(size === 1.U, Cat(0.U(16.W), shalf), rs2v))

  // ---------------- assertions (compiled out of synthesis builds) ----------------
  assert(fetchPc(1, 0) === 0.U, "fetch pc must stay 4-byte aligned")
  assert(!exValid || pc(1, 0) === 0.U, "EX pc must be 4-byte aligned")
  assert(exValid || !(rf.io.we || io.dmem.we || io.dmem.re || trapNow || csr.io.commit), "bubble changed state")
  assert(!(io.trace.valid && io.trace.trap), "retire and trap in the same cycle")
  assert(!(io.dmem.we && io.dmem.re), "simultaneous load and store")
  assert(!(trapNow && (rf.io.we || io.dmem.we)), "trapping instruction modifies state")
  assert(!(divStall && (rf.io.we || io.dmem.we || io.dmem.re || trapNow)), "state changed while waiting for the
divider")
  assert(!(divStall && nextPc =/= pc), "pc moved while waiting for the divider")
}

object Core {
  /** `addi x0, x0, 0`: what EX executes when IfEx holds a bubble. */
  val Nop = 0x00000013L
}
```

---




---

```
package rv32i
import chisel3._

/** RISC-V major opcodes (inst[6:0]) used by this core. */
object Opcode {
  val LOAD     = 0x03
  val MISC_MEM = 0x0f
  val OP_IMM   = 0x13
  val AUIPC    = 0x17
  val STORE    = 0x23
  val OP       = 0x33
  val LUI      = 0x37
  val BRANCH   = 0x63
  val JALR     = 0x67
  val JAL      = 0x6f
  val SYSTEM   = 0x73
}

/** mcause exception codes (interrupt bit = 0). */
object Cause {
  val InstrAddrMisaligned = 0
  val IllegalInstruction  = 2
  val Breakpoint          = 3
  val LoadAddrMisaligned  = 4
  val StoreAddrMisaligned = 6
  val EcallFromM          = 11
}

/** CSR addresses implemented by [[CsrFile]]. */
object CsrAddr {
  val mstatus   = 0x300
  val misa      = 0x301
  val mie       = 0x304
  val mtvec     = 0x305
  val mstatush  = 0x310
  val mscratch  = 0x340
  val mepc      = 0x341
  val mcause    = 0x342
  val mtval     = 0x343
  val mip       = 0x344
  val mcycle    = 0xb00
  val minstret  = 0xb02
  val mcycleh   = 0xb80
  val minstreth = 0xb82
  val cycle     = 0xc00
```

```
  val instret   = 0xc02
  val cycleh    = 0xc80
  val instreth  = 0xc82
  val mvendorid = 0xf11
  val marchid   = 0xf12
  val mimpid    = 0xf13
  val mhartid   = 0xf14

  /** misa for RV32IM: MXL = 1 (bits 31:30), I = bit 8, M = bit 12. */
  val MisaValue = 0x40001100L
}

/** First ALU operand. */
object ASrc extends ChiselEnum {
  val RS1, PC, ZERO = Value
}

/** Write-back source. */
object WbSel extends ChiselEnum {
  val ALU, MEM, PC4, CSR = Value
}

/** Immediate format. */
object ImmSel extends ChiselEnum {
  val I, S, B, U, J = Value
}

/** ALU operation. */
object AluOp extends ChiselEnum {
  val ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND = Value
}

/** Instruction port: zero-wait, `data` is a combinational function of `addr`. */
class InstrMemPort extends Bundle {
  val addr = Output(UInt(32.W))
  val data = Input(UInt(32.W))
}

/** Data port: zero-wait. `rdata` is the aligned word at `addr & ~3`, a combinational function of `addr`.
  * A store writes the bytes selected by `wstrb` of `wdata` (already shifted into byte lanes) at the clock edge.
  */
class DataMemPort extends Bundle {
  val addr  = Output(UInt(32.W))
  val re    = Output(Bool())
  val we    = Output(Bool())
  val wdata = Output(UInt(32.W))
  val wstrb = Output(UInt(4.W))
  val rdata = Input(UInt(32.W))
}

/** Per-cycle retirement trace (EX stage). `valid`: an instruction retired. `trap`: an exception was taken.
  * Both low: a pipeline bubble or a cycle waiting for the divider (nothing changed in that cycle).
  */
class Trace extends Bundle {
  val valid    = Bool()
  val trap     = Bool()
  val cause    = UInt(4.W)
  val pc       = UInt(32.W)
  val pcNext   = UInt(32.W)
  val inst     = UInt(32.W)
  val rdAddr   = UInt(5.W)   // 0 when no register is written
  val rdData   = UInt(32.W)  // 0 when no register is written
  val memAddr  = UInt(32.W)
  val memRe    = Bool()
  val memWe    = Bool()
  val memSize  = UInt(2.W)   // 0 byte, 1 half, 2 word
  val memWdata = UInt(32.W)  // store data, not shifted into lanes
}

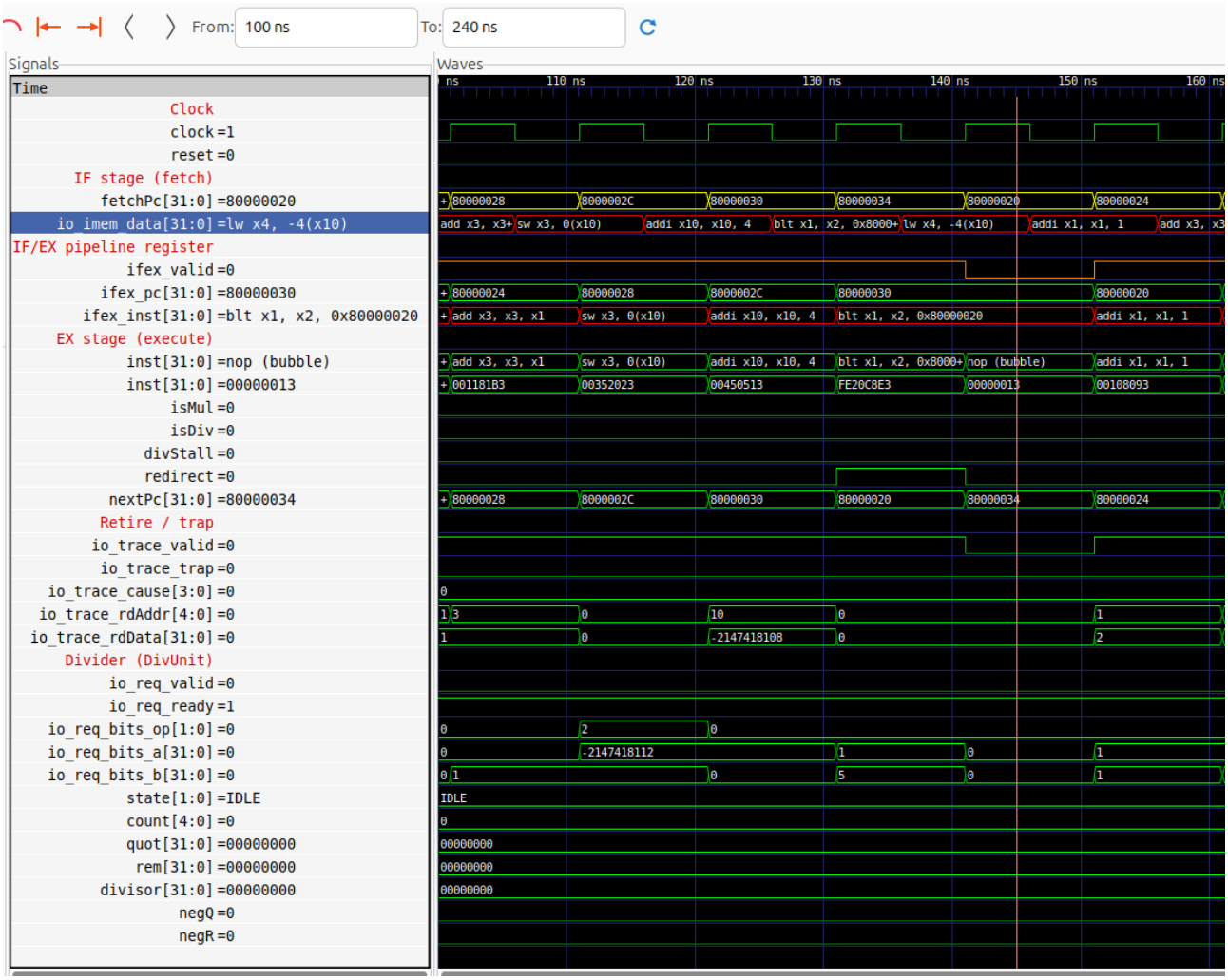
/** IF/EX pipeline register of the 2-stage core. */
class IfEx extends Bundle {
  val valid = Bool()       // false: bubble (after reset, or flushed by a redirect)
  val pc    = UInt(32.W)
  val inst  = UInt(32.W)
}
```

---

## $ sbt test

```
[info] welcome to sbt 1.10.2 (Ubuntu Java 17.0.20.1)
[info] loading project definition from /media/bako/0969-2ABE/books_2026/CL.Book.Chisel.2026/RV32.Steps/rv32im-
pipe2/project
[info] loading settings for project root from build.sbt ...
[info] set current project to rv32im-pipe2 (in build
file:/media/bako/0969-2ABE/books_2026/CL.Book.Chisel.2026/RV32.Steps/rv32im-pipe2/)
[info] DecoderSpec:
[info] Decoder
[info] - should decode every RV32I, Zicsr and M-mode SYSTEM instruction
[info] - should flag illegal encodings and assert no other control signal for them
[info] - should classify every one of the 128 major opcodes
[info] AluSpec:
[info] Alu
[info] - should compute every operation on all edge-value pairs (incl. overflow, shift by 31/32, sign)
[info] - should match the reference on random operands
[info] WaveformSpec:
[info] Waveforms
[info] - should show DIV/REM stalls and MUL in one cycle (waves/div_rem)
[info] - should show loads, stores, a loop and a call (waves/load_store_branch)
[info] - should show an ECALL trap, the handler and MRET (waves/trap)
[info] - should show the divider on its own: request, 32 iterations, response (waves/div_unit)
[info] DivUnitSpec:
[info] DivUnit
[info] - should handle every edge-value pair for all four instructions
[info] - should match the reference on random operands
[info] - should abandon a division on kill and accept the next one
[info] - should run back-to-back requests from a core that holds valid high
[info] CsrFileSpec:
[info] CsrFile
[info] - should implement CSRRW / CSRRS / CSRRC semantics
[info] - should report misa = RV32IM, mhartid and fixed values, and apply WARL masks
[info] - should flag unknown CSRs and writes to read-only CSRs as illegal, and not write them
[info] - should count cycles always and instructions on commit, and let a write replace one half
[info] - should save state on trap entry and restore it on mret
[info] CoreDivSpec:
[info] Core
[info] - should execute DIV, DIVU, REM and REMU including the spec's corner cases
[info] - should use a division result in the next instruction and ignore rd = x0
[info] - should still execute the Step 4 multiplications in one cycle
[info] MultiplierSpec:
[info] Multiplier
[info] - should compute MUL, MULH, MULHSU and MULHU on edge values and random operands
[info] ImmGenSpec:
[info] ImmGen
[info] - should build the I, S, B, U and J immediates from instructions the assembler encoded
[info] CoreTimingSpec:
[info] Core
[info] - should execute every instruction in one cycle except DIV/DIVU/REM/REMU, plus one bubble per redirect
[info] - should discard the wrong-path instruction behind a taken branch, a jump, a trap and MRET
[info] CoreProgramSpec:
[info] Core
[info] - should run arithmetic and logic instructions (OP, OP-IMM, LUI, AUIPC)
[info] - should run loads and stores of every size, signed and unsigned
[info] - should run every branch both ways, loops, JAL and JALR (incl. clearing bit 0)
[info] - should handle back-to-back dependencies, including load-use and M results used immediately
[info] - should execute the M extension: corner cases, a gcd loop with REM and a factorial with MUL
[info] - should implement Zicsr and take every exception precisely (illegal, ECALL, EBREAK, misaligned
load/store/fetch)
[info] - should match the reference model on 3000 random instructions
[info] DivRefSpec:
[info] DivRef
[info] - should follow the RISC-V division-by-zero and overflow table
[info] - should round toward zero and give the remainder the dividend's sign
[info] - should agree with the Int form
[info] RegFileSpec:
[info] RegFile
[info] - should hardwire x0 to zero and store x1..x31
[info] - should ignore writes when we is low and return the old value during a write
[info] Run completed in 1 minute, 24 seconds.
[info] Total number of tests run: 37
[info] Suites: completed 12, aborted 0
[info] Tests: succeeded 37, failed 0, canceled 0, ignored 0, pending 0
[info] All tests passed.
[success] Total time: 88 s (01:28), completed 11 nov. 2026, 10:17:44
```

---


From: 100 ns
To: 240 ns
Signals
Waves
Time
Clock
clock =1
reset =0
IF stage (fetch)
fetchPc[31:0] =80000020
io_imem_data[31:0] =lw x4, -4(x10)
IF/EX pipeline register
ifex_valid =0
ifex_pc[31:0] =80000030
ifex_inst[31:0] =blt x1, x2, 0x80000020
EX stage (execute)
inst[31:0] =nop (bubble)
inst[31:0] =00000013
isMul =0
isDiv =0
divStall =0
redirect =0
nextPc[31:0] =80000034
Retire / trap
io_trace_valid =0
io_trace_trap =0
io_trace_cause[3:0] =0
io_trace_rdAddr[4:0] =0
io_trace_rdData[31:0] =0
Divider (DivUnit)
io_req_valid =0
io_req_ready =1
io_req_bits_op[1:0] =0
io_req_bits_a[31:0] =0
io_req_bits_b[31:0] =0
state[1:0] =IDLE
count[4:0] =0
quot[31:0] =00000000
rem[31:0] =00000000
divisor[31:0] =00000000
negQ =0
negR =0
110 ns
120 ns
130 ns
140 ns
150 ns
160 ns
80000028
8000002C
80000030
80000034
80000020
80000024
add x3, x3+
sw x3, 0(x10)
addi x10, x10, 4
blt x1, x2, 0x8000+
lw x4, -4(x10)
addi x1, x1, 1
add x3, x3
80000024
80000028
8000002C
80000030
80000020
add x3, x3, x1
sw x3, 0(x10)
addi x10, x10, 4
blt x1, x2, 0x80000020
addi x1, x1, 1
add x3, x3, x1
sw x3, 0(x10)
addi x10, x10, 4
blt x1, x2, 0x8000+
nop (bubble)
addi x1, x1, 1
00181B3
00352023
00450513
FE20C8E3
00000013
00108093
80000028
8000002C
80000030
80000020
80000034
80000024
0
1 3
0
10
0
1
1
0
-2147418108
0
2
0
2
0
0
-2147418112
1
0
1
0 1
0
5
0
1
IDLE
0
00000000
00000000
00000000

# GLab 7

## Building full RV32IM architecture with 5-stage pipeline

This is the biggest step so far, so I'll de-risk it the same way as before. First a cycle-accurate Python model of the hazard, forwarding, flush and precise-trap logic, checked against an ISA model. Then the Chisel, translated line by line from that model.
The 5-stage pipeline (your roadmap's Step 7) is ready as **`CorePipe5`**.
**The Step 4, 5 and 6 cores are still selectable with `SocConfig(stages = 1 | 2 | 5)` and serve as references.**
**Same caveat as every step so far: none of this has been compiled or simulated.** Maven is still blocked here, so your first `sbt test` is also the first compile. On the riscv-formal side, nothing here could run riscv-formal or Spike either.
What I could check passed:

- **Design model:** a Python cycle model of all five stages matched an ISA model on 3,000 random programs. The programs mix traps, CSRs, M ops, loads/stores and branches, and every gap between retirements also matched the timing rule below.
- **The checker itself:** I deliberately broke the model seven ways and every one was caught: missing WB→EX or MEM→EX forwarding, missing WB→ID bypass, no load-use stall, a store not squashed by an older trap, the M unit not killed, and a one-instruction flush. The store case was only caught after I made the test trap handler skip the instruction behind the trap. Before that, a leaked store just replayed with the same value.

**How the pipeline behaves:**

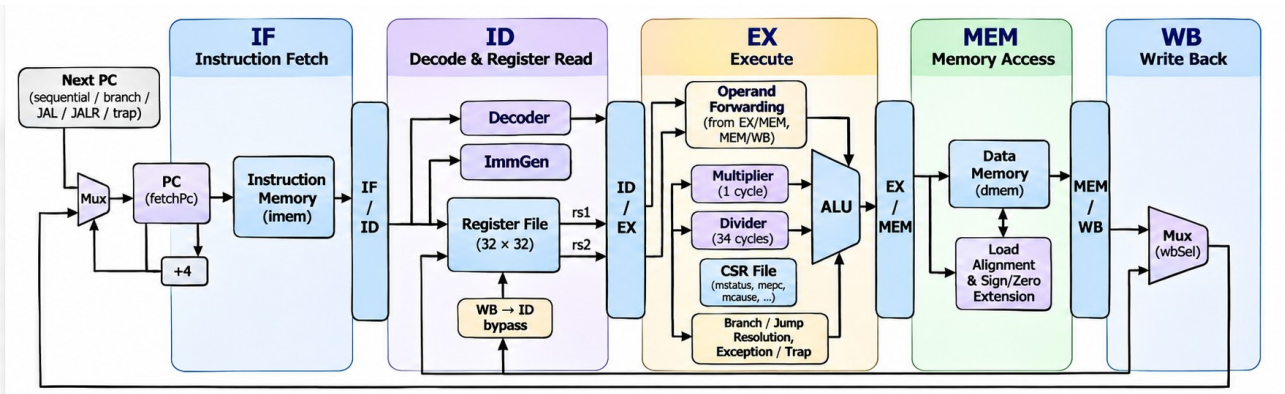
- **Forwarding:** EX takes operands from MEM first, then WB, into the ALU, branch compare, store data, JALR base, CSR operand and M operands. The register file also passes WB's write straight through to ID, so WB→ID has no hazard.
- **Load-use:** load data arrives in MEM but is forwarded only from WB. An instruction that uses a load result right after the load stalls one cycle; CSR reads behave the same way.
- **Branches and jumps:** they resolve in EX. A redirect flushes two instructions (static not-taken), and a taken branch to pc+4 costs nothing.
- **M unit:** while it works, IF/ID/EX freeze and bubbles go into MEM. A running operation is killed if an older instruction traps.
- **Precise traps:** exception flags travel down from ID and EX; CSR-illegal accesses are detected in MEM. Traps and MRET take effect in MEM, where CSRs are also read and written in program order. In that same cycle the store write and M-unit start are blocked, so older instructions finish and younger ones never change state.

**Timing rule:** the gap between consecutive retirements is 1 cycle (MUL 18, DIV 34). Add 3 after a trap or MRET, 2 after a taken redirect, and 1 for a load-use or CSR-use pair.
**Tests:**

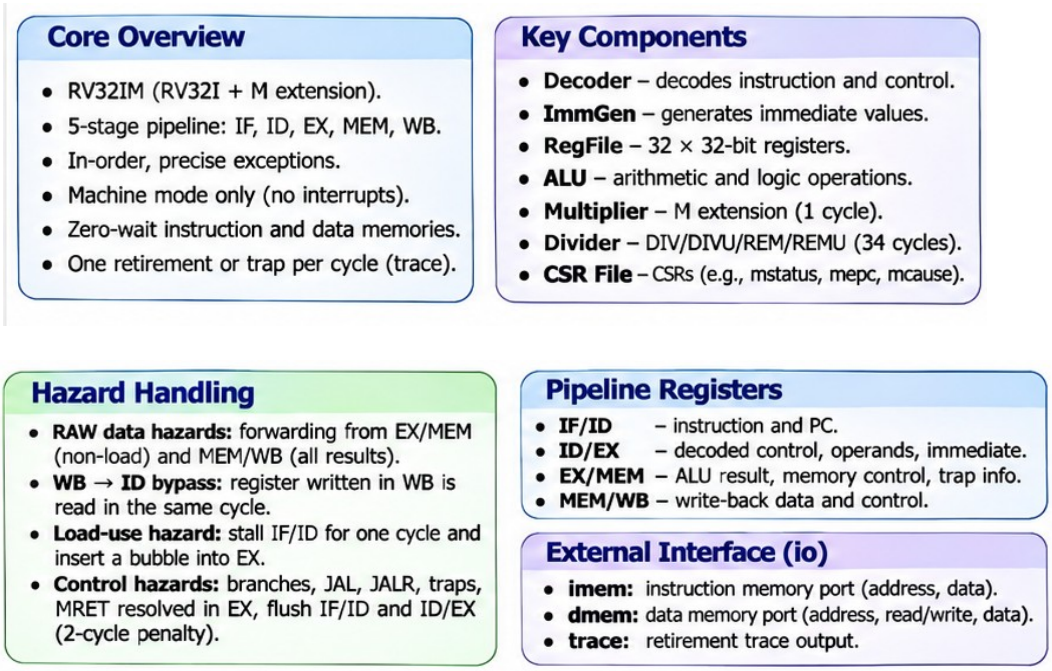
- `Pipe5Spec` is all directed:
  - every forwarding path, including both operands forwarded at once and MEM winning over WB;
  - load-use into ALU, store data, a pointer-chasing load address, a branch, M, and a CSR read;
  - back-to-back loads and stores, including store then load of the same word;
  - a branch on a just-loaded register, taken and not taken;
  - traps from each stage: ID (illegal, ECALL, EBREAK), EX (misaligned load, store, jump target) and MEM (unknown or read-only CSR). Each checks that the older store is visible and the younger store or M op is squashed.

  Every test also checks Step 4 trace equality and the exact gap between each pair of retirements.
- `RandomProgSpec` generates random RV32IM + CSR programs: bounded loops, jumps, `fence.i`, and every trap kind, each followed by an instruction the handler skips. Step 7 must match Step 4 on each.
- `scripts/random-spike.sh` assembles the same random programs and compares the commit logs against Spike.
- **riscv-formal:** `RvfiPipe5` exposes the RVFI ports, with `wrapper.sv` and `checks.cfg` under `formal/riscv-formal/`. `scripts/riscv-formal.sh` runs the insn, reg, pc_fwd/pc_bwd, causal, unique, liveness and cover checks.

---



---




---

```
package rv32i

import chisel3._
import chisel3.util._

/** 5-stage pipelined RV32IM core: IF, ID, EX, MEM, WB. Machine mode only, no interrupts.
  *
  * Stages
  *  - IF : `fetchPc` addresses the instruction memory; the word goes into [[IfId]]. Fetch continues at pc + 4
  *         (static not-taken prediction).
  *  - ID : decode, immediate, register read (with a WB -> ID bypass), load-use hazard detection -> [[IdEx]].
  *  - EX : operand forwarding (from MEM and WB), ALU, multiplier (1 cycle), sequential divider, branch/jump
  *         resolution, CSR read/write, exception detection and trap entry -> [[ExMem]].
  *  - MEM: data memory access and load alignment -> [[MemWb]].
  *  - WB : register write and the retirement trace.
  *
  * Hazards
  *  - RAW data hazards: forwarding into EX from EX/MEM (non-load results) and MEM/WB (all results); a register
  *    written in WB is bypassed to the ID read in the same cycle. A load followed by an instruction that uses its
  *    result (load-use) stalls IF/ID for one cycle and sends a bubble into EX.
  *  - Control: taken branches, JAL, JALR, traps and MRET are resolved in EX; the two younger instructions
  *    (in IF/ID and ID/EX) are flushed and fetch restarts at the target (2-cycle penalty).
  *  - Divider: DIV/DIVU/REM/REMU stay in EX for 34 cycles (2 for divide-by-zero and signed overflow); IF, ID and
  *    EX freeze, MEM and WB keep draining (bubbles).
  *
  * Precise exceptions: every exception is known in EX (illegal, ECALL, EBREAK from ID; misaligned addresses and
  * targets from EX). The trap is taken there: older instructions in MEM/WB complete, younger ones are flushed,
the
  * trapping instruction does not access memory or write a register. It then travels down the pipe so the trace
  * reports it in WB, in program order. CSRs (including minstret) are read and written in EX, in program order.
  *
  * The trace is produced in WB: one retirement or trap per cycle, or neither (bubble / stall).
  * Memory ports are zero-wait: read data is a combinational function of the address.
  */
class Core(resetPc: Long = 0x80000000L, hartId: Int = 0) extends Module {
  val io = IO(new Bundle {
    val imem  = new InstrMemPort
    val dmem  = new DataMemPort
    val trace = Output(new Trace)
  })

  require(resetPc >= 0 && resetPc < (1L << 32) && resetPc % 4 == 0, "reset pc must be 4-byte aligned")

  val dec  = Module(new Decoder)
  val immg = Module(new ImmGen)
  val rf   = Module(new RegFile)
  val alu  = Module(new Alu)
  val csr  = Module(new CsrFile(hartId))
  val mul  = Module(new Multiplier)
  val div  = Module(new DivUnit(32))

  // ---------------- pipeline registers (all empty after reset) ----------------
  val ifid  = RegInit(0.U.asTypeOf(new IfId))
  val idex  = RegInit(0.U.asTypeOf(new IdEx))
```

```
  val exmem = RegInit(0.U.asTypeOf(new ExMem))
  val memwb = RegInit(0.U.asTypeOf(new MemWb))

  // ================================================================================
  // IF
  // ================================================================================
  val fetchPc = RegInit(resetPc.U(32.W))
  io.imem.addr := fetchPc

  // ================================================================================
  // ID
  // ================================================================================
  val idInst   = Mux(ifid.valid, ifid.inst, Core.Nop.U(32.W))
  val idOpcode = idInst(6, 0)
  val idF3     = idInst(14, 12)
  val idMulDiv = idOpcode === Opcode.OP.U && idInst(31, 25) === "b0000001".U
  val rs1      = idInst(19, 15)
  val rs2      = idInst(24, 20)

  dec.io.inst  := idInst
  immg.io.inst := idInst
  immg.io.sel  := dec.io.immSel
  rf.io.rs1    := rs1
  rf.io.rs2    := rs2

  // WB -> ID bypass: the register written in WB this cycle is read with its new value.
  val wbWrites = memwb.valid && !memwb.trap && memwb.regWrite && memwb.rd =/= 0.U
  val rs1Val   = Mux(wbWrites && memwb.rd === rs1, memwb.wdata, rf.io.rd1)
  val rs2Val   = Mux(wbWrites && memwb.rd === rs2, memwb.wdata, rf.io.rd2)

  val idCtrl = Wire(new Ctrl)
  idCtrl.aluSrcA     := dec.io.aluSrcA
  idCtrl.aluSrcImm   := dec.io.aluSrcImm
  idCtrl.aluOp       := dec.io.aluOp
  idCtrl.branch      := dec.io.branch
  idCtrl.jal         := dec.io.jal
  idCtrl.jalr        := dec.io.jalr
  idCtrl.memRead     := dec.io.memRead
  idCtrl.memWrite    := dec.io.memWrite
  idCtrl.memSize     := dec.io.memSize
  idCtrl.memUnsigned := dec.io.memUnsigned
  idCtrl.wbSel       := dec.io.wbSel
  idCtrl.regWrite    := dec.io.regWrite || idMulDiv
  idCtrl.csr         := dec.io.csr
  idCtrl.ecall       := dec.io.ecall
  idCtrl.ebreak      := dec.io.ebreak
  idCtrl.mret        := dec.io.mret
  // The eight M-extension instructions are legal even though the RV32I decoder flags funct7 = 0000001.
  idCtrl.illegal     := dec.io.illegal && !idMulDiv
  idCtrl.isMul       := idMulDiv && !idF3(2)
  idCtrl.isDiv       := idMulDiv && idF3(2)
  // Which source registers the instruction really reads (no false hazards for LUI, JAL, CSR-immediate, ...).
  idCtrl.usesRs1     := !idCtrl.illegal && (dec.io.jalr || dec.io.branch || dec.io.memRead || dec.io.memWrite ||
                        idOpcode === Opcode.OP_IMM.U || idOpcode === Opcode.OP.U || (dec.io.csr && !idF3(2)))
  idCtrl.usesRs2     := !idCtrl.illegal && (dec.io.branch || dec.io.memWrite || idOpcode === Opcode.OP.U)

  // Load-use hazard: EX holds a load whose destination is a source of the ID instruction.
  val loadUse = ifid.valid && idex.valid && idex.ctrl.memRead && idex.rd =/= 0.U &&
    ((idCtrl.usesRs1 && rs1 === idex.rd) || (idCtrl.usesRs2 && rs2 === idex.rd))

  // ================================================================================
  // EX
  // ================================================================================
  val exValid = idex.valid
  val c       = idex.ctrl
  val exPc    = idex.pc
  val exInst  = idex.inst
  val exF3    = exInst(14, 12)

  // Forwarding: EX/MEM (not for loads: their data is not there yet) has priority over MEM/WB.
  private def forward(idx: UInt, idVal: UInt): UInt = {
    val fromMem = exmem.valid && !exmem.trap && exmem.regWrite && !exmem.memRead && exmem.rd === idx && idx =/=
0.U
    val fromWb  = memwb.valid && !memwb.trap && memwb.regWrite && memwb.rd === idx && idx =/= 0.U
    Mux(fromMem, exmem.result, Mux(fromWb, memwb.wdata, idVal))
  }
  val a = forward(idex.rs1, idex.rs1Val)
  val b = forward(idex.rs2, idex.rs2Val)

  // ALU
  val aluA = WireDefault(a)
  switch(c.aluSrcA) {
    is(ASrc.RS1)  { aluA := a }
    is(ASrc.PC)   { aluA := exPc }
    is(ASrc.ZERO) { aluA := 0.U }
  }
  alu.io.op := c.aluOp
  alu.io.a  := aluA
  alu.io.b  := Mux(c.aluSrcImm, idex.imm, b)
  val aluY = alu.io.y
```

```
// multiplier (combinational) and divider (sequential)
mul.io.fn := exF3(1, 0)
mul.io.a  := a
mul.io.b  := b
div.io.req.valid   := exValid && c.isDiv && !div.io.busy
div.io.req.bits.op := exF3(1, 0)
div.io.req.bits.a  := a
div.io.req.bits.b  := b
val divStall = exValid && c.isDiv && !div.io.resp.valid

// branches and jumps
val eq  = a === b
val lt  = a.asSInt < b.asSInt
val ltu = a < b
val brCond = WireDefault(false.B)
switch(exF3) {
  is("b000".U) { brCond := eq }
  is("b001".U) { brCond := !eq }
  is("b100".U) { brCond := lt }
  is("b101".U) { brCond := !lt }
  is("b110".U) { brCond := ltu }
  is("b111".U) { brCond := !ltu }
}
val pcPlus4    = exPc + 4.U
val brTarget   = exPc + idex.imm
val jalrTarget = Cat(aluY(31, 1), 0.U(1.W))
val ctlTaken   = (c.branch && brCond) || c.jal || c.jalr
val ctlTarget  = Mux(c.jalr, jalrTarget, brTarget)

// address checks
val misalignedAccess = (c.memSize === 2.U && aluY(1, 0) =/= 0.U) || (c.memSize === 1.U && aluY(0))
val loadMis  = c.memRead && misalignedAccess
val storeMis = c.memWrite && misalignedAccess
val fetchMis = ctlTaken && ctlTarget(1)

// CSRs
val rs1Field = exInst(19, 15)
csr.io.addr := exInst(31, 20)
csr.io.en   := exValid && c.csr
csr.io.op   := exF3(1, 0)
csr.io.src  := Mux(exF3(2), Cat(0.U(27.W), rs1Field), a)
csr.io.wrEn := exF3(1, 0) === 1.U || rs1Field =/= 0.U

// exceptions (all known here) and commit
val illegal = c.illegal || csr.io.illegal
val trapNow = exValid && (illegal || c.ecall || c.ebreak || loadMis || storeMis || fetchMis)
div.io.kill := trapNow

val cause = PriorityMux(Seq(
  illegal    -> Cause.IllegalInstruction.U(4.W),
  c.ecall    -> Cause.EcallFromM.U(4.W),
  c.ebreak   -> Cause.Breakpoint.U(4.W),
  loadMis    -> Cause.LoadAddrMisaligned.U(4.W),
  storeMis   -> Cause.StoreAddrMisaligned.U(4.W),
  fetchMis   -> Cause.InstrAddrMisaligned.U(4.W)
))
val tval = PriorityMux(Seq(
  illegal               -> exInst,
  c.ecall               -> 0.U(32.W),
  c.ebreak              -> exPc,
  (loadMis || storeMis) -> aluY,
  fetchMis              -> ctlTarget
))

// The EX instruction is committed here: nothing after EX can fail.
val exCommit = exValid && !trapNow && !divStall
csr.io.commit    := exCommit
csr.io.trapEnter := trapNow
csr.io.trapCause := Cat(0.U(28.W), cause)
csr.io.trapPc    := exPc
csr.io.trapVal   := tval
csr.io.mretEnter := exValid && c.mret && !trapNow

val nextPc = Mux(trapNow, csr.io.mtvecBase,
             Mux(c.mret, csr.io.mepc,
             Mux(ctlTaken, ctlTarget, pcPlus4)))
val redirect = exValid && !divStall && (trapNow || c.mret || ctlTaken)

val exResult = WireDefault(aluY)
switch(c.wbSel) {
  is(WbSel.ALU) { exResult := aluY }
  is(WbSel.PC4) { exResult := pcPlus4 }
  is(WbSel.CSR) { exResult := csr.io.rdata }
  is(WbSel.MEM) { exResult := aluY }        // replaced by the load data in MEM
}
when(c.isMul) { exResult := mul.io.result }
when(c.isDiv) { exResult := div.io.resp.bits }

// ================================================================================
```

```
// pipeline control: IF, ID and EX
// ============================================================================
when(divStall) {
  // freeze IF, IF/ID and ID/EX while the divider works on the EX instruction
}.elsewhen(redirect) {
  fetchPc    := nextPc
  ifid.valid := false.B                      // flush the two wrong-path instructions
  idex.valid := false.B
}.elsewhen(loadUse) {
  idex.valid := false.B                      // bubble into EX; IF and IF/ID hold
}.otherwise {
  fetchPc     := fetchPc + 4.U
  ifid.valid  := true.B
  ifid.pc     := fetchPc
  ifid.inst   := io.imem.data
  idex.valid  := ifid.valid
  idex.pc     := ifid.pc
  idex.inst   := idInst
  idex.rs1    := rs1
  idex.rs2    := rs2
  idex.rd     := idInst(11, 7)
  idex.rs1Val := rs1Val
  idex.rs2Val := rs2Val
  idex.imm    := immg.io.imm
  idex.ctrl   := idCtrl
}

// EX -> MEM: a bubble while the divider works, otherwise the executed instruction (or its trap)
exmem.valid := exValid && !divStall
when(!divStall) {
  exmem.trap        := trapNow
  exmem.cause       := cause
  exmem.pc          := exPc
  exmem.pcNext      := nextPc
  exmem.inst        := exInst
  exmem.rd          := idex.rd
  exmem.regWrite    := c.regWrite && !trapNow
  exmem.result      := exResult
  exmem.memRead     := c.memRead && !trapNow
  exmem.memWrite    := c.memWrite && !trapNow
  exmem.memSize     := c.memSize
  exmem.memUnsigned := c.memUnsigned
  exmem.addr        := aluY
  exmem.storeData   := b
}

// ============================================================================
// MEM
// ============================================================================
val mAddr = exmem.addr
val size  = exmem.memSize
io.dmem.addr := mAddr
io.dmem.re   := exmem.valid && exmem.memRead
io.dmem.we   := exmem.valid && exmem.memWrite

val sbyte = exmem.storeData(7, 0)
val shalf = exmem.storeData(15, 0)
val wdata = WireDefault(exmem.storeData)
val wstrb = WireDefault("b1111".U(4.W))
switch(size) {
  is(0.U) { wdata := Fill(4, sbyte); wstrb := (1.U(4.W) << mAddr(1, 0))(3, 0) }
  is(1.U) { wdata := Fill(2, shalf); wstrb := Mux(mAddr(1), "b1100".U(4.W), "b0011".U(4.W)) }
}
io.dmem.wdata := wdata
io.dmem.wstrb := Mux(io.dmem.we, wstrb, 0.U)

val rdata = io.dmem.rdata
val bytes = VecInit(Seq.tabulate(4)(k => rdata(8 * k + 7, 8 * k)))
val lb    = bytes(mAddr(1, 0))
val lh    = Mux(mAddr(1), rdata(31, 16), rdata(15, 0))
val uns   = exmem.memUnsigned
val loadData = WireDefault(rdata)
switch(size) {
  is(0.U) { loadData := Mux(uns, Cat(0.U(24.W), lb), Cat(Fill(24, lb(7)), lb)) }
  is(1.U) { loadData := Mux(uns, Cat(0.U(16.W), lh), Cat(Fill(16, lh(15)), lh)) }
}

memwb.valid    := exmem.valid
memwb.trap     := exmem.trap
memwb.cause    := exmem.cause
memwb.pc       := exmem.pc
memwb.pcNext   := exmem.pcNext
memwb.inst     := exmem.inst
memwb.rd       := exmem.rd
memwb.regWrite := exmem.regWrite
memwb.wdata    := Mux(exmem.memRead, loadData, exmem.result)
memwb.memRe    := io.dmem.re
memwb.memWe    := io.dmem.we
memwb.memAddr  := mAddr
memwb.memSize  := size
```

```
  memwb.memWdata := Mux(size === 0.U, Cat(0.U(24.W), sbyte), Mux(size === 1.U, Cat(0.U(16.W), shalf),
exmem.storeData))

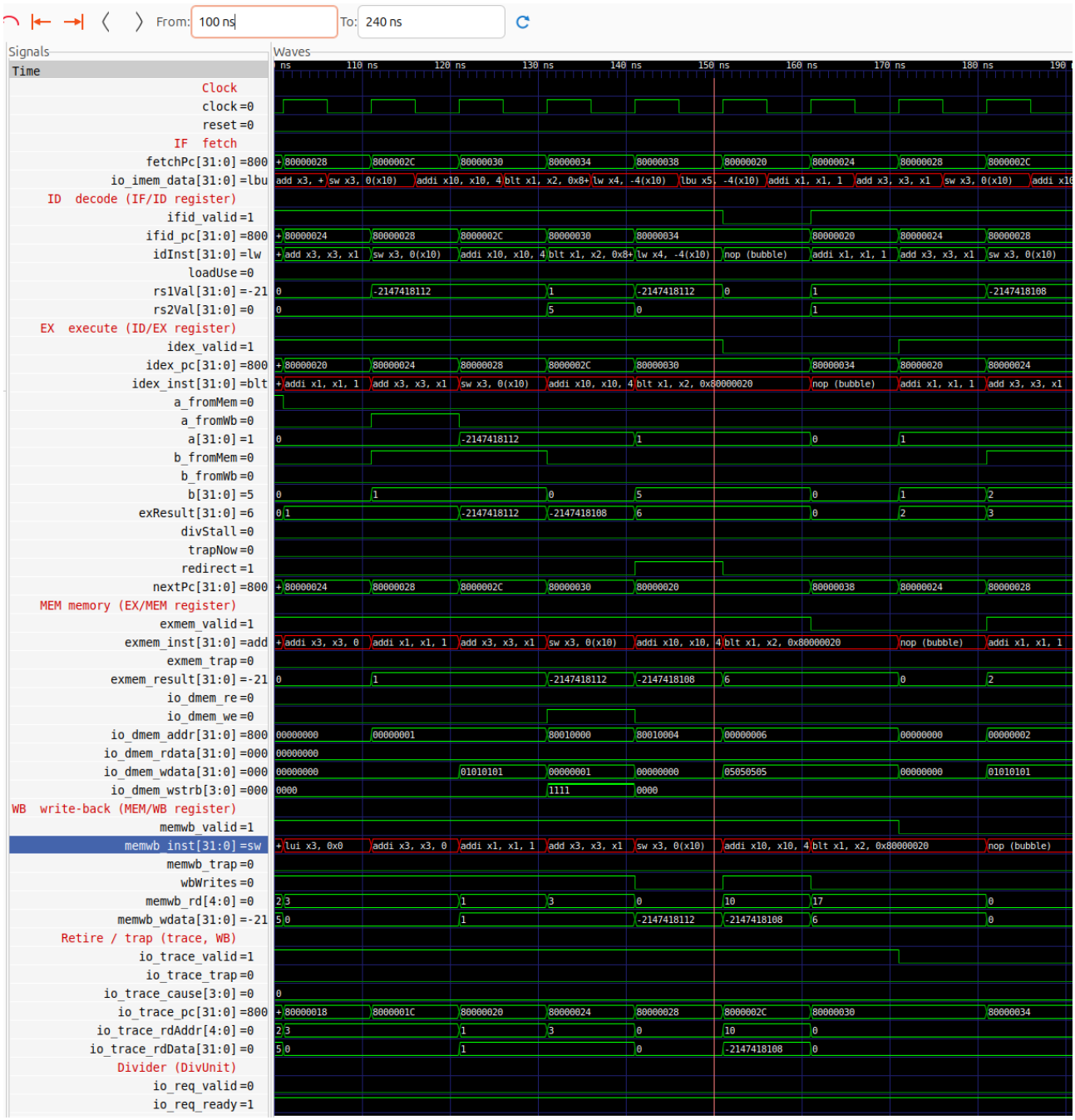
  // ================================================================================
  // WB
  // ================================================================================
  rf.io.we    := memwb.valid && !memwb.trap && memwb.regWrite
  rf.io.waddr := memwb.rd
  rf.io.wdata := memwb.wdata

  io.trace.valid    := memwb.valid && !memwb.trap
  io.trace.trap     := memwb.valid && memwb.trap
  io.trace.cause    := memwb.cause
  io.trace.pc       := memwb.pc
  io.trace.pcNext   := memwb.pcNext
  io.trace.inst     := memwb.inst
  io.trace.rdAddr   := Mux(wbWrites, memwb.rd, 0.U)
  io.trace.rdData   := Mux(wbWrites, memwb.wdata, 0.U)
  io.trace.memAddr  := memwb.memAddr
  io.trace.memRe    := memwb.valid && memwb.memRe
  io.trace.memWe    := memwb.valid && memwb.memWe
  io.trace.memSize  := memwb.memSize
  io.trace.memWdata := memwb.memWdata

  // ================================================================================
  // assertions (compiled out of synthesis builds)
  // ================================================================================
  assert(fetchPc(1, 0) === 0.U, "fetch pc must stay 4-byte aligned")
  assert(!(io.dmem.we && io.dmem.re), "simultaneous load and store")
  assert(!(exmem.valid && exmem.trap && (exmem.memRead || exmem.memWrite || exmem.regWrite)),
    "trapping instruction accesses memory or writes a register")
  assert(!(exValid && exmem.valid && exmem.memRead && exmem.rd =/= 0.U &&
    ((c.usesRs1 && idex.rs1 === exmem.rd) || (c.usesRs2 && idex.rs2 === exmem.rd))),
    "load-use hazard reached EX (missing stall)")
  assert(!(divStall && trapNow), "a division cannot trap")
  assert(!(csr.io.commit && !exValid), "bubble committed")
}

object Core {
  /** `addi x0, x0, 0`: what ID decodes when IF/ID holds a bubble. */
  val Nop = 0x00000013L
  /** Cycles from EX to WB: an instruction executed in EX in cycle t is reported in the trace in cycle t + 2. */
  val ExToWb = 2
  /** Empty cycles before the first instruction reaches WB after reset. */
  val FillCycles = 4
}
-----------------------------------------------------------------------------------------------
```

```
$ sbt test
[info] welcome to sbt 1.10.2 (Ubuntu Java 17.0.21.0.3)
[info] loading project definition from /media/bako/0969-2ABE/books_2026/CL.Book.Chisel.2026/RV32.Steps/rv32im-
pipe5/project
[info] loading settings for project root from build.sbt ...
[info] set current project to rv32im-pipe5 (in build
file:/media/bako/0969-2ABE/books_2026/CL.Book.Chisel.2026/RV32.Steps/rv32im-pipe5/)
[info] DecoderSpec:
[info] Decoder
[info] - should decode every RV32I, Zicsr and M-mode SYSTEM instruction
[info] - should flag illegal encodings and assert no other control signal for them
[info] - should classify every one of the 128 major opcodes
[info] AluSpec:
[info] Alu
[info] - should compute every operation on all edge-value pairs (incl. overflow, shift by 31/32, sign)
[info] - should match the reference on random operands
[info] WaveformSpec:
[info] Waveforms
[info] - should show DIV/REM stalls and MUL in one cycle (waves/div_rem)
[info] - should show loads, stores, a loop and a call (waves/load_store_branch)
[info] - should show an ECALL trap, the handler and MRET (waves/trap)
[info] - should show the divider on its own: request, 32 iterations, response (waves/div_unit)
[info] DivUnitSpec:
[info] DivUnit
[info] - should handle every edge-value pair for all four instructions
[info] - should match the reference on random operands
[info] - should abandon a division on kill and accept the next one
[info] - should run back-to-back requests from a core that holds valid high
[info] CsrFileSpec:
[info] CsrFile
[info] - should implement CSRRW / CSRRS / CSRRC semantics
[info] - should report misa = RV32IM, mhartid and fixed values, and apply WARL masks
[info] - should flag unknown CSRs and writes to read-only CSRs as illegal, and not write them
[info] - should count cycles always and instructions on commit, and let a write replace one half
[info] - should save state on trap entry and restore it on mret
[info] CoreDivSpec:
[info] Core
[info] - should execute DIV, DIVU, REM and REMU including the spec's corner cases
[info] - should use a division result in the next instruction and ignore rd = x0
[info] - should still execute the Step 4 multiplications in one cycle
[info] MultiplierSpec:
[info] Multiplier
[info] - should compute MUL, MULH, MULHSU and MULHU on edge values and random operands
[info] ImmGenSpec:
[info] ImmGen
[info] - should build the I, S, B, U and J immediates from instructions the assembler encoded
[info] CoreTimingSpec:
[info] Core
[info] - should retire one instruction per cycle except DIV/DIVU/REM/REMU, redirect bubbles and load-use stalls
[info] - should discard the wrong-path instruction behind a taken branch, a jump, a trap and MRET
[info] CoreProgramSpec:
[info] Core
[info] - should run arithmetic and logic instructions (OP, OP-IMM, LUI, AUIPC)
[info] - should run loads and stores of every size, signed and unsigned
[info] - should run every branch both ways, loops, JAL and JALR (incl. clearing bit 0)
[info] - should handle back-to-back dependencies, including load-use and M results used immediately
[info] - should execute the M extension: corner cases, a gcd loop with REM and a factorial with MUL
[info] - should implement Zicsr and take every exception precisely (illegal, ECALL, EBREAK, misaligned
load/store/fetch)
[info] - should match the reference model on 3000 random instructions
[info] DivRefSpec:
[info] DivRef
[info] - should follow the RISC-V division-by-zero and overflow table
[info] - should round toward zero and give the remainder the dividend's sign
[info] - should agree with the Int form
[info] RegFileSpec:
[info] RegFile
[info] - should hardwire x0 to zero and store x1..x31
[info] - should ignore writes when we is low and return the old value during a write
[info] Run completed in 2 minutes.
[info] Total number of tests run: 37
[info] Suites: completed 12, aborted 0
[info] Tests: succeeded 37, failed 0, canceled 0, ignored 0, pending 0
[info] All tests passed.
[success] Total time: 124 s (02:04), completed 11 nov. 2026, 10:40:45
-----------------------------------------------------------------------------------------------
```


From: 100 ns
To: 240 ns
Signals
Waves
Time
Clock
clock =0
reset =0
IF fetch
fetchPc[31:0] =800
io_imem_data[31:0] =lbu
ID decode (IF/ID register)
ifid_valid =1
ifid_pc[31:0] =800
idInst[31:0] =lw
loadUse =0
rs1Val[31:0] =-21
rs2Val[31:0] =0
EX execute (ID/EX register)
idex_valid =1
idex_pc[31:0] =800
idex_inst[31:0] =blt
a_fromMem =0
a_fromWb =0
a[31:0] =1
b_fromMem =0
b_fromWb =0
b[31:0] =5
exResult[31:0] =6
divStall =0
trapNow =0
redirect =1
nextPc[31:0] =800
MEM memory (EX/MEM register)
exmem_valid =1
exmem_inst[31:0] =add
exmem_trap =0
exmem_result[31:0] =-21
io_dmem_re =0
io_dmem_we =0
io_dmem_addr[31:0] =800
io_dmem_rdata[31:0] =000
io_dmem_wdata[31:0] =000
io_dmem_wstrb[3:0] =000
WB write-back (MEM/WB register)
memwb_valid =1
memwb_inst[31:0] =sw
memwb_trap =0
wbWrites =0
memwb_rd[4:0] =0
memwb_wdata[31:0] =-21
Retire / trap (trace, WB)
io_trace_valid =1
io_trace_trap =0
io_trace_cause[3:0] =0
io_trace_pc[31:0] =800
io_trace_rdAddr[4:0] =0
io_trace_rdData[31:0] =0
Divider (DivUnit)
io_req_valid =0
io_req_ready =1

# GLab 8

## 5-stage pipeline with I-cache and D-cache

This is the largest step, so the same approach as before: a cycle-accurate Python model of the whole cached system first. It covers the core, I$, D$ (write-through and write-back), arbiter, random-latency memory and MMIO, checked against an ISA model with mutation tests. Then the Chisel, translated from it.
Key design decisions before modelling:

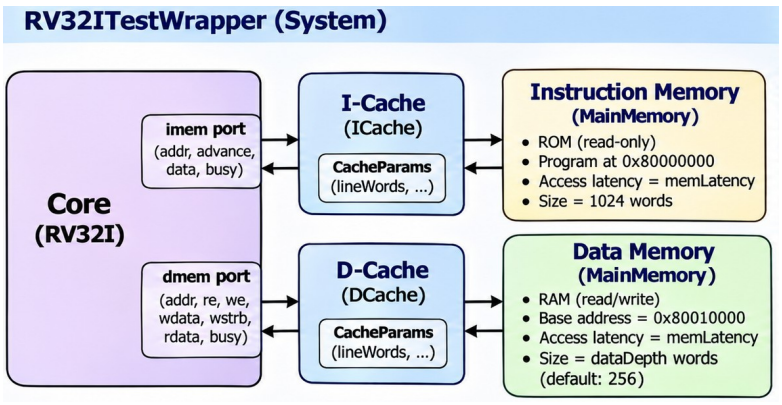
- **Stores move to MEM.** The tag check needs the synchronous tag read, so stores (and loads) complete in MEM, which is also the commit point, so it stays precise.
- **D$ lookup address.** The D$ reads its arrays with `stall ? memAddr : exAddr`, so its output always belongs to the instruction in MEM. The same trick as the I$/IF.
- `fence.i` **executes in MEM.** It flushes the D$, then invalidates the I$, then redirects to pc+4.
- **EX can now be frozen by a MEM stall.** EX must keep refreshing its operand registers from forwarding while frozen. Otherwise a value forwarded from WB is lost when that producer leaves.

The generator was at fault: emitting the patch + `fence.i` pair as two entries shifted later branch targets backwards, producing loops. I'm switching targets to be relative to the real position.
Step 8 is ready: `CoreCached` is the Step 7 pipeline with an I-cache and a D-cache, both D$ versions, the line-burst bus with its arbiter, a memory model with random latency, and an uncached MMIO region. Steps 4–7 stay selectable, and Step 7 is the reference the cached core's trace must match.
**As with every step, none of this has been compiled or simulated.** Maven is still blocked here; your first `sbt test` is also the first compile.
What I could check passed:

- **System model:** a Python cycle model of core + I$ + D$ (both versions) + arbiter + random-latency memory + MMIO matched an ISA model on hundreds of random programs. Those programs include self-modifying code, and the model treats a block-RAM read of a word written in the same cycle as undefined.
- **The checker itself:** seven seeded bugs were all caught. The other two are performance-only or can't be observed with this bus, so those guards are kept as defensive measures.

**The model found a real design bug before any Chisel existed.** Once a D$ miss can freeze EX, an instruction waiting there can lose a value it was being forwarded from WB, because that producer moves on. The fix: while EX is frozen it keeps re-capturing its operands. A model run without that fix fails.
**What's in place:**

- **Bus:** request / write-data / response channels, one transaction at a time, which maps onto AXI4 later. The arbiter gives the D$ priority and holds the grant until the transaction ends.
- **I$:** direct-mapped, 4 KB with 16-byte lines, tag and data in block RAM, IDLE → REQ → REFILL. A miss stalls IF while the rest of the pipeline drains.
- **D$:** direct-mapped, 4 KB, write-through/no-allocate (`writeBack = false`) or write-back/allocate (the default) with a write-back-then-refill eviction sequence. A miss stalls MEM.
    - The cache never relies on reading a block-RAM word in the cycle it's written: unsafe lookups are simply redone.
    - A store followed immediately by a load of the same word gets its data bypassed.
- **Stores now complete in MEM, not EX**, because the tag check needs the synchronous tag read. MEM is the commit point, so stores and MMIO side effects remain precise.
- **MMIO:** a UART at 0x1000_0000 and a test-done register at 0x1000_0004. riscv-tests' `tohost` block is uncached too.
- `fence.i`**:** handled in MEM. It writes back dirty D$ lines, invalidates the I$, then refetches from pc+4.
- **Performance counters** on `io.perf`: cycles, retired instructions, I/D stall cycles, I$ hits/misses, D$ load/store hits/misses, write-backs and uncached accesses.



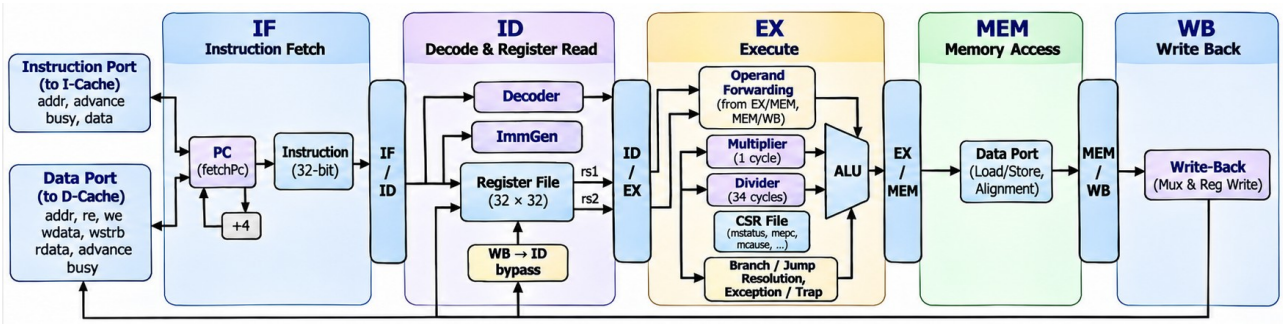
```
--------------------------------------------------------------------------------------------------------------
import chisel3._

/** Stage 8 system: Core + I-cache + D-cache, each in front of its own main memory (Harvard):
  * instruction memory (ROM, program at 0x80000000) and data memory (RAM at 0x80010000), both with `memLatency`.
  */
class RV32ITestWrapper(programFile: String, dataFile: String = "", memLatency: Int = 4,
                       icacheParams: CacheParams = CacheParams(), dcacheParams: CacheParams = CacheParams(),
                       dataDepth: Int = 256) extends Module {
  val io = IO(new Bundle {
    val trace  = Output(new Trace)
    val icache = Output(new CacheStats)
    val dcache = Output(new CacheStats)
  })
  val core   = Module(new Core())
  val icache = Module(new ICache(icacheParams))
  val dcache = Module(new DCache(dcacheParams))
  val imem   = Module(new MainMemory(programFile, 1024, 0x80000000L, memLatency, icacheParams.lineWords, writable
= false))
  val dmem   = Module(new MainMemory(dataFile, dataDepth, 0x80010000L, memLatency, dcacheParams.lineWords))

  icache.io.addr     := core.io.imem.addr
  core.io.imem.data  := icache.io.data
  core.io.imem.busy  := icache.io.busy
  icache.io.advance  := core.io.imem.advance
  icache.io.mem <> imem.io.port

  dcache.io.addr     := core.io.dmem.addr
  dcache.io.re       := core.io.dmem.re
  dcache.io.we       := core.io.dmem.we
  dcache.io.wdata    := core.io.dmem.wdata
  dcache.io.wstrb    := core.io.dmem.wstrb
  core.io.dmem.rdata := dcache.io.rdata
  core.io.dmem.busy  := dcache.io.busy
  dcache.io.advance  := core.io.dmem.advance
  dcache.io.mem <> dmem.io.port

  io.trace  := core.io.trace
  io.icache := icache.io.stats
  io.dcache := dcache.io.stats
}
--------------------------------------------------------------------------------------------------------------
```




```
--------------------------------------------------------------------------------------------------------------
```

**Core Overview**

- RV32IM (RV32I + M extension).
- 5-stage pipeline: IF, ID, EX, MEM, WB.
- For use with I-Cache and D-Cache (Stage 8).
- Machine mode only, no interrupts.
- Precise exceptions, in-order retirement.
- Trace output in WB.

**Key Components**

- **Decoder** – generates control signals.
- **ImmGen** – generates immediate values.
- **RegFile** – 32 × 32-bit registers.
- **ALU** – arithmetic and logic operations.
- **Multiplier** – M extension (1 cycle).
- **Divider** – DIV/DIVU/REM/REMU (34 cycles).
- **CSR File** – CSRs (e.g., mstatus, mepc, mcause).

```
--------------------------------------------------------------------------------------------------------------
```

**Hazard Handling**

- **RAW data hazards**: forwarding from EX/MEM (non-load) and MEM/WB; WB → ID bypass.
- **Load-use hazard**: stall IF/ID for one cycle.
- **Control hazards**: taken branches, JAL, JALR, traps, MRET resolved in EX; flush two younger instructions.
- **Divider**: DIV/DIVU/REM/REMU stay in EX for 34 cycles (2 for divide-by-zero/overflow); IF, ID and EX hold.

**Cache Interaction & Pipeline Freeze**

- Each memory port has a **busy** input from the cache.
- **freeze** = imem.busy || dmem.busy.
- While **freeze** is true: the whole pipeline is frozen (no pipeline register update, no commit, no divider start, no CSR write, no trace); memory requests are held.
- A divider result arriving during a freeze is held (divHave) until EX can accept it.

```
--------------------------------------------------------------------------------------------------------------
package rv32i
import chisel3._
import chisel3.util._
```

```
/** Stage 8: 5-stage pipelined RV32IM core (IF, ID, EX, MEM, WB) for use with an I-cache and a D-cache.
  * Machine mode only, no interrupts.
  *
  * Stages
  *  - IF : `fetchPc` addresses the instruction port; the word goes into [[IfId]]. Fetch continues at pc + 4
  *         (static not-taken prediction).
  *  - ID : decode, immediate, register read (with a WB -> ID bypass), load-use hazard detection -> [[IdEx]].
  *  - EX : operand forwarding (from MEM and WB), ALU, multiplier (1 cycle), sequential divider, branch/jump
  *         resolution, CSR read/write, exception detection and trap entry -> [[ExMem]].
  *  - MEM: data port access and load alignment -> [[MemWb]].
  *  - WB : register write and the retirement trace.
  *
  * Hazards (as in the Stage 7 core without caches)
  *  - RAW: forwarding into EX from EX/MEM (non-load results) and MEM/WB; WB -> ID bypass; a load followed by a
user of
  *    its result stalls IF/ID one cycle (load-use).
  *  - Control: taken branches, JAL, JALR, traps and MRET resolve in EX and flush the two younger instructions.
  *  - Divider: DIV/DIVU/REM/REMU stay in EX for 34 cycles (2 for divide-by-zero and overflow); IF, ID and EX
hold.
  *
  * Caches (Stage 8): the instruction and data ports have a `busy` input. While either is high the whole pipeline
is
  * frozen for that cycle (`freeze`): no pipeline register changes, nothing commits (CSRs, traps, divider start,
  * register write, trace), and the memory requests stay unchanged so the caches can complete them. A divider
result
  * that arrives during a freeze is held (`divHave`) until the division leaves EX.
  *
  * Precise exceptions: every exception is known in EX and taken there; the trap record travels to WB, so the
trace
  * reports it in program order. CSRs (including minstret) are read and written in EX, in program order.
  * The trace is produced in WB; `exCycle` tells when the instruction left EX (for mcycle comparisons).
  */
class Core(resetPc: Long = 0x80000000L, hartId: Int = 0) extends Module {
  val io = IO(new Bundle {
    val imem  = new InstrMemPort
    val dmem  = new DataMemPort
    val trace = Output(new Trace)
  })

  require(resetPc >= 0 && resetPc < (1L << 32) && resetPc % 4 == 0, "reset pc must be 4-byte aligned")

  val dec  = Module(new Decoder)
  val immg = Module(new ImmGen)
  val rf   = Module(new RegFile)
  val alu  = Module(new Alu)
  val csr  = Module(new CsrFile(hartId))
  val mul  = Module(new Multiplier)
  val div  = Module(new DivUnit(32))

  // ---------------- pipeline registers (all empty after reset) ----------------
  val ifid  = RegInit(0.U.asTypeOf(new IfId))
  val idex  = RegInit(0.U.asTypeOf(new IdEx))
  val exmem = RegInit(0.U.asTypeOf(new ExMem))
  val memwb = RegInit(0.U.asTypeOf(new MemWb))

  // ================================================================================
  // IF
  // ================================================================================
  val fetchPc = RegInit(resetPc.U(32.W))
  io.imem.addr := fetchPc

  // ---------------- cache waits freeze the whole pipeline ----------------
  val freeze = io.imem.busy || io.dmem.busy
  io.imem.advance := !freeze
  io.dmem.advance := !freeze
  /** Cycles since reset; each instruction carries the value of the cycle in which it left EX. */
  val cycleCount = RegInit(0.U(32.W))
  cycleCount := cycleCount + 1.U

  // ================================================================================
  // ID
  // ================================================================================
  val idInst   = Mux(ifid.valid, ifid.inst, Core.Nop.U(32.W))
  val idOpcode = idInst(6, 0)
  val idF3     = idInst(14, 12)
  val idMulDiv = idOpcode === Opcode.OP.U && idInst(31, 25) === "b0000001".U
  val rs1      = idInst(19, 15)
  val rs2      = idInst(24, 20)

  dec.io.inst  := idInst
  immg.io.inst := idInst
  immg.io.sel  := dec.io.immSel
  rf.io.rs1    := rs1
  rf.io.rs2    := rs2

  // WB -> ID bypass: the register written in WB this cycle is read with its new value.
  val wbWrites = memwb.valid && !memwb.trap && memwb.regWrite && memwb.rd =/= 0.U && !freeze
  val rs1Val   = Mux(wbWrites && memwb.rd === rs1, memwb.wdata, rf.io.rd1)
  val rs2Val   = Mux(wbWrites && memwb.rd === rs2, memwb.wdata, rf.io.rd2)
```

```
  val idCtrl = Wire(new Ctrl)
  idCtrl.aluSrcA     := dec.io.aluSrcA
  idCtrl.aluSrcImm   := dec.io.aluSrcImm
  idCtrl.aluOp       := dec.io.aluOp
  idCtrl.branch      := dec.io.branch
  idCtrl.jal         := dec.io.jal
  idCtrl.jalr        := dec.io.jalr
  idCtrl.memRead     := dec.io.memRead
  idCtrl.memWrite    := dec.io.memWrite
  idCtrl.memSize     := dec.io.memSize
  idCtrl.memUnsigned := dec.io.memUnsigned
  idCtrl.wbSel       := dec.io.wbSel
  idCtrl.regWrite    := dec.io.regWrite || idMulDiv
  idCtrl.csr         := dec.io.csr
  idCtrl.ecall       := dec.io.ecall
  idCtrl.ebreak      := dec.io.ebreak
  idCtrl.mret        := dec.io.mret
  // The eight M-extension instructions are legal even though the RV32I decoder flags funct7 = 0000001.
  idCtrl.illegal     := dec.io.illegal && !idMulDiv
  idCtrl.isMul       := idMulDiv && !idF3(2)
  idCtrl.isDiv       := idMulDiv && idF3(2)
  // Which source registers the instruction really reads (no false hazards for LUI, JAL, CSR-immediate, ...).
  idCtrl.usesRs1     := !idCtrl.illegal && (dec.io.jalr || dec.io.branch || dec.io.memRead || dec.io.memWrite ||
                        idOpcode === Opcode.OP_IMM.U || idOpcode === Opcode.OP.U || (dec.io.csr && !idF3(2)))
  idCtrl.usesRs2     := !idCtrl.illegal && (dec.io.branch || dec.io.memWrite || idOpcode === Opcode.OP.U)

  // Load-use hazard: EX holds a load whose destination is a source of the ID instruction.
  val loadUse = ifid.valid && idex.valid && idex.ctrl.memRead && idex.rd =/= 0.U &&
    ((idCtrl.usesRs1 && rs1 === idex.rd) || (idCtrl.usesRs2 && rs2 === idex.rd))

  // ================================================================================
  // EX
  // ================================================================================
  val exValid = idex.valid
  val c       = idex.ctrl
  val exPc    = idex.pc
  val exInst  = idex.inst
  val exF3    = exInst(14, 12)

  // Forwarding: EX/MEM (not for loads: their data is not there yet) has priority over MEM/WB.
  private def forward(idx: UInt, idVal: UInt): UInt = {
    val fromMem = exmem.valid && !exmem.trap && exmem.regWrite && !exmem.memRead && exmem.rd === idx && idx =/=
0.U
    val fromWb  = memwb.valid && !memwb.trap && memwb.regWrite && memwb.rd === idx && idx =/= 0.U
    Mux(fromMem, exmem.result, Mux(fromWb, memwb.wdata, idVal))
  }
  val a = forward(idex.rs1, idex.rs1Val)
  val b = forward(idex.rs2, idex.rs2Val)

  // ALU
  val aluA = WireDefault(a)
  switch(c.aluSrcA) {
    is(ASrc.RS1)  { aluA := a }
    is(ASrc.PC)   { aluA := exPc }
    is(ASrc.ZERO) { aluA := 0.U }
  }
  alu.io.op := c.aluOp
  alu.io.a  := aluA
  alu.io.b  := Mux(c.aluSrcImm, idex.imm, b)
  val aluY = alu.io.y

  // multiplier (combinational) and divider (sequential)
  mul.io.fn := exF3(1, 0)
  mul.io.a  := a
  mul.io.b  := b
  // The result is kept in divHeld if it arrives while the pipeline is frozen, until the division leaves EX.
  val divHave   = RegInit(false.B)
  val divHeld   = Reg(UInt(32.W))
  val divReady  = div.io.resp.valid || divHave
  val divResult = Mux(div.io.resp.valid, div.io.resp.bits, divHeld)
  div.io.req.valid   := exValid && c.isDiv && !div.io.busy && !divHave && !freeze
  div.io.req.bits.op := exF3(1, 0)
  div.io.req.bits.a  := a
  div.io.req.bits.b  := b
  val divStall = exValid && c.isDiv && !divReady

  // branches and jumps
  val eq  = a === b
  val lt  = a.asSInt < b.asSInt
  val ltu = a < b
  val brCond = WireDefault(false.B)
  switch(exF3) {
    is("b000".U) { brCond := eq }
    is("b001".U) { brCond := !eq }
    is("b100".U) { brCond := lt }
    is("b101".U) { brCond := !lt }
    is("b110".U) { brCond := ltu }
    is("b111".U) { brCond := !ltu }
  }
  val pcPlus4    = exPc + 4.U
```

```
val brTarget   = exPc + idex.imm
val jalrTarget = Cat(aluY(31, 1), 0.U(1.W))
val ctlTaken   = (c.branch && brCond) || c.jal || c.jalr
val ctlTarget  = Mux(c.jalr, jalrTarget, brTarget)

// address checks
val misalignedAccess = (c.memSize === 2.U && aluY(1, 0) =/= 0.U) || (c.memSize === 1.U && aluY(0))
val loadMis  = c.memRead && misalignedAccess
val storeMis = c.memWrite && misalignedAccess
val fetchMis = ctlTaken && ctlTarget(1)

// CSRs
val rs1Field = exInst(19, 15)
csr.io.addr := exInst(31, 20)
csr.io.en   := exValid && c.csr
csr.io.op   := exF3(1, 0)
csr.io.src  := Mux(exF3(2), Cat(0.U(27.W), rs1Field), a)
csr.io.wrEn := exF3(1, 0) === 1.U || rs1Field =/= 0.U

// exceptions (all known here) and commit
val illegal = c.illegal || csr.io.illegal
val trapNow = exValid && (illegal || c.ecall || c.ebreak || loadMis || storeMis || fetchMis)
div.io.kill := trapNow && !freeze

val cause = PriorityMux(Seq(
  illegal    -> Cause.IllegalInstruction.U(4.W),
  c.ecall    -> Cause.EcallFromM.U(4.W),
  c.ebreak   -> Cause.Breakpoint.U(4.W),
  loadMis    -> Cause.LoadAddrMisaligned.U(4.W),
  storeMis   -> Cause.StoreAddrMisaligned.U(4.W),
  fetchMis   -> Cause.InstrAddrMisaligned.U(4.W)
))
val tval = PriorityMux(Seq(
  illegal               -> exInst,
  c.ecall               -> 0.U(32.W),
  c.ebreak              -> exPc,
  (loadMis || storeMis) -> aluY,
  fetchMis              -> ctlTarget
))

// The EX instruction is committed here: nothing after EX can fail.
val exCommit = exValid && !trapNow && !divStall && !freeze
csr.io.commit    := exCommit
csr.io.trapEnter := trapNow && !freeze
csr.io.trapCause := Cat(0.U(28.W), cause)
csr.io.trapPc    := exPc
csr.io.trapVal   := tval
csr.io.mretEnter := exValid && c.mret && !trapNow && !freeze

val nextPc = Mux(trapNow, csr.io.mtvecBase,
             Mux(c.mret, csr.io.mepc,
             Mux(ctlTaken, ctlTarget, pcPlus4)))
val redirect = exValid && !divStall && (trapNow || c.mret || ctlTaken)

val exResult = WireDefault(aluY)
switch(c.wbSel) {
  is(WbSel.ALU) { exResult := aluY }
  is(WbSel.PC4) { exResult := pcPlus4 }
  is(WbSel.CSR) { exResult := csr.io.rdata }
  is(WbSel.MEM) { exResult := aluY }        // replaced by the load data in MEM
}
when(c.isMul) { exResult := mul.io.result }
when(c.isDiv) { exResult := divResult }

// the held divider result belongs to the division in EX: set on the response, cleared when it leaves EX
val exLeaves = exValid && !divStall && !freeze
when(div.io.resp.valid) { divHave := true.B; divHeld := div.io.resp.bits }
when(exLeaves || !exValid) { divHave := false.B }

// ============================================================================
// pipeline control: IF, ID and EX
// ============================================================================
when(freeze) {
  // a cache is busy: every pipeline register holds
}.elsewhen(divStall) {
  // freeze IF, IF/ID and ID/EX while the divider works on the EX instruction
}.elsewhen(redirect) {
  fetchPc    := nextPc
  ifid.valid := false.B                      // flush the two wrong-path instructions
  idex.valid := false.B
}.elsewhen(loadUse) {
  idex.valid := false.B                      // bubble into EX; IF and IF/ID hold
}.otherwise {
  fetchPc     := fetchPc + 4.U
  ifid.valid  := true.B
  ifid.pc     := fetchPc
  ifid.inst   := io.imem.data
  idex.valid  := ifid.valid
  idex.pc     := ifid.pc
  idex.inst   := idInst
```

```
    idex.rs1    := rs1
    idex.rs2    := rs2
    idex.rd     := idInst(11, 7)
    idex.rs1Val := rs1Val
    idex.rs2Val := rs2Val
    idex.imm    := immg.io.imm
    idex.ctrl   := idCtrl
  }

  // EX -> MEM: a bubble while the divider works, otherwise the executed instruction (or its trap)
  when(!freeze) { exmem.valid := exValid && !divStall }
  when(!divStall && !freeze) {
    exmem.exCycle     := cycleCount
    exmem.trap        := trapNow
    exmem.cause       := cause
    exmem.pc          := exPc
    exmem.pcNext      := nextPc
    exmem.inst        := exInst
    exmem.rd          := idex.rd
    exmem.regWrite    := c.regWrite && !trapNow
    exmem.result      := exResult
    exmem.memRead     := c.memRead && !trapNow
    exmem.memWrite    := c.memWrite && !trapNow
    exmem.memSize     := c.memSize
    exmem.memUnsigned := c.memUnsigned
    exmem.addr        := aluY
    exmem.storeData   := b
  }

  // ================================================================================
  // MEM
  // ================================================================================
  val mAddr = exmem.addr
  val size  = exmem.memSize
  io.dmem.addr := mAddr
  io.dmem.re   := exmem.valid && exmem.memRead
  io.dmem.we   := exmem.valid && exmem.memWrite

  val sbyte = exmem.storeData(7, 0)
  val shalf = exmem.storeData(15, 0)
  val wdata = WireDefault(exmem.storeData)
  val wstrb = WireDefault("b1111".U(4.W))
  switch(size) {
    is(0.U) { wdata := Fill(4, sbyte); wstrb := (1.U(4.W) << mAddr(1, 0))(3, 0) }
    is(1.U) { wdata := Fill(2, shalf); wstrb := Mux(mAddr(1), "b1100".U(4.W), "b0011".U(4.W)) }
  }
  io.dmem.wdata := wdata
  io.dmem.wstrb := Mux(io.dmem.we, wstrb, 0.U)

  val rdata = io.dmem.rdata
  val bytes = VecInit(Seq.tabulate(4)(k => rdata(8 * k + 7, 8 * k)))
  val lb    = bytes(mAddr(1, 0))
  val lh    = Mux(mAddr(1), rdata(31, 16), rdata(15, 0))
  val uns   = exmem.memUnsigned
  val loadData = WireDefault(rdata)
  switch(size) {
    is(0.U) { loadData := Mux(uns, Cat(0.U(24.W), lb), Cat(Fill(24, lb(7)), lb)) }
    is(1.U) { loadData := Mux(uns, Cat(0.U(16.W), lh), Cat(Fill(16, lh(15)), lh)) }
  }

  when(!freeze) {
    memwb.valid    := exmem.valid
    memwb.trap     := exmem.trap
    memwb.cause    := exmem.cause
    memwb.pc       := exmem.pc
    memwb.pcNext   := exmem.pcNext
    memwb.inst     := exmem.inst
    memwb.rd       := exmem.rd
    memwb.regWrite := exmem.regWrite
    memwb.wdata    := Mux(exmem.memRead, loadData, exmem.result)
    memwb.memRe    := io.dmem.re
    memwb.memWe    := io.dmem.we
    memwb.memAddr  := mAddr
    memwb.memSize  := size
    memwb.memWdata := Mux(size === 0.U, Cat(0.U(24.W), sbyte), Mux(size === 1.U, Cat(0.U(16.W), shalf),
exmem.storeData))
    memwb.exCycle  := exmem.exCycle
  }

  // ================================================================================
  // WB
  // ================================================================================
  rf.io.we    := memwb.valid && !memwb.trap && memwb.regWrite && !freeze
  rf.io.waddr := memwb.rd
  rf.io.wdata := memwb.wdata

  io.trace.valid    := memwb.valid && !memwb.trap && !freeze
  io.trace.trap     := memwb.valid && memwb.trap && !freeze
  io.trace.exCycle  := memwb.exCycle
  io.trace.cause    := memwb.cause
```

```
  io.trace.pc       := memwb.pc
  io.trace.pcNext   := memwb.pcNext
  io.trace.inst     := memwb.inst
  io.trace.rdAddr   := Mux(wbWrites, memwb.rd, 0.U)
  io.trace.rdData   := Mux(wbWrites, memwb.wdata, 0.U)
  io.trace.memAddr  := memwb.memAddr
  io.trace.memRe    := memwb.valid && memwb.memRe && !freeze   // only in the cycle the instruction is reported
  io.trace.memWe    := memwb.valid && memwb.memWe && !freeze
  io.trace.memSize  := memwb.memSize
  io.trace.memWdata := memwb.memWdata

  // ============================================================================================
  // assertions (compiled out of synthesis builds)
  // ============================================================================================
  assert(fetchPc(1, 0) === 0.U, "fetch pc must stay 4-byte aligned")
  assert(!(io.dmem.we && io.dmem.re), "simultaneous load and store")
  assert(!(exmem.valid && exmem.trap && (exmem.memRead || exmem.memWrite || exmem.regWrite)),
    "trapping instruction accesses memory or writes a register")
  assert(!(exValid && exmem.valid && exmem.memRead && exmem.rd =/= 0.U &&
    ((c.usesRs1 && idex.rs1 === exmem.rd) || (c.usesRs2 && idex.rs2 === exmem.rd))),
    "load-use hazard reached EX (missing stall)")
  assert(!(divStall && trapNow), "a division cannot trap")
  assert(!(csr.io.commit && !exValid), "bubble committed")
  assert(!(freeze && (rf.io.we || csr.io.commit || csr.io.trapEnter || io.trace.valid || io.trace.trap)),
    "state changed during a cache freeze")
}

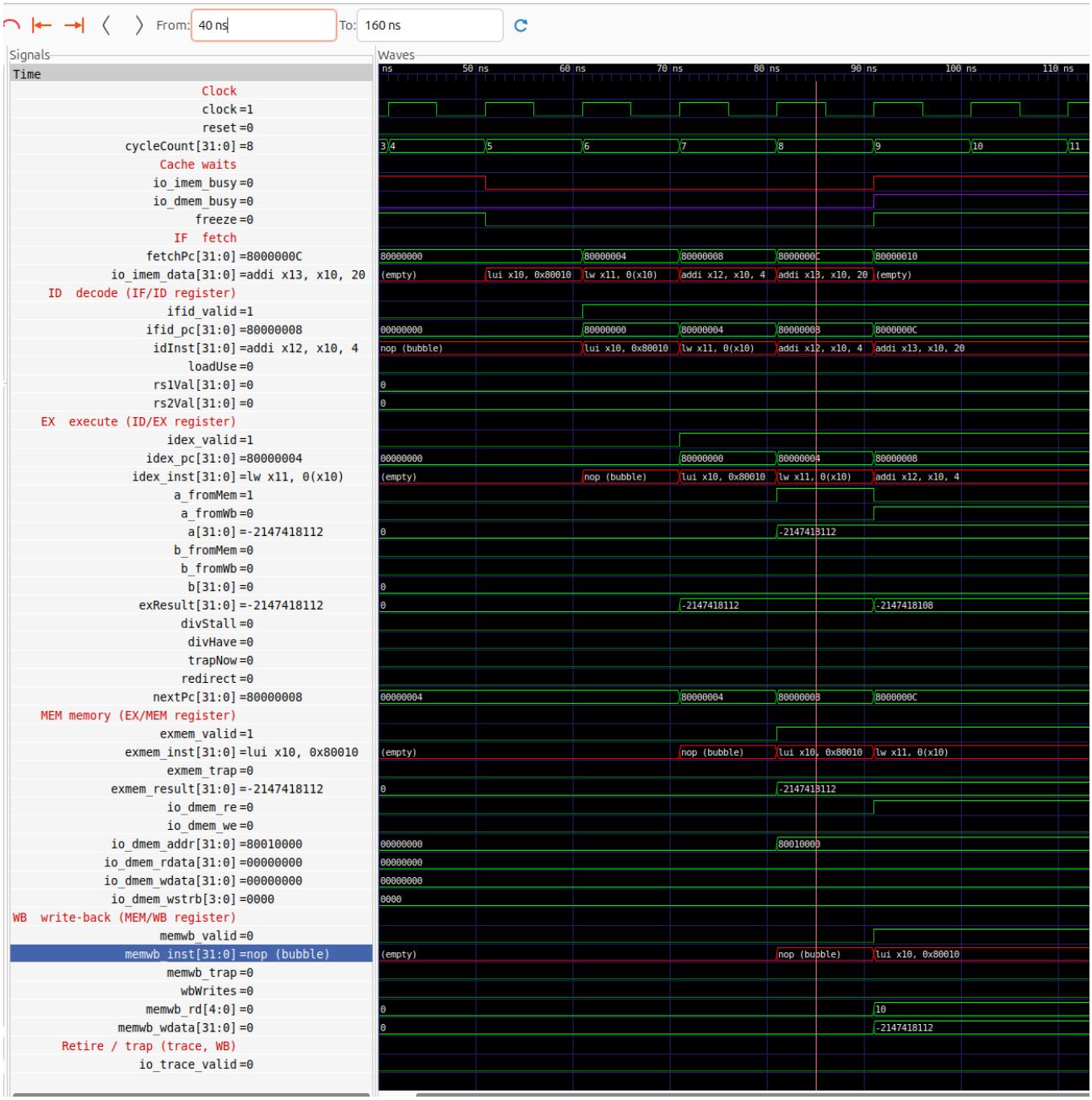
object Core {
  /** `addi x0, x0, 0`: what ID decodes when IF/ID holds a bubble. */
  val Nop = 0x00000013L
  /** Cycles from EX to WB: an instruction executed in EX in cycle t is reported in the trace in cycle t + 2. */
  val ExToWb = 2
  /** Empty cycles before the first instruction reaches WB after reset. */
  val FillCycles = 4
}
```

---

```
$ sbt test
[info] welcome to sbt 1.10.2 (Ubuntu Java 17.0.21.0.3)
[info] loading project definition from /media/bako/0969-2ABE/books_2026/CL.Book.Chisel.2026/RV32.Steps/rv32im-
stage8/project
[info] loading settings for project root from build.sbt ...
[info] set current project to rv32im-stage8 (in build
file:/media/bako/0969-2ABE/books_2026/CL.Book.Chisel.2026/RV32.Steps/rv32im-stage8/)
[info] DecoderSpec:
[info] Decoder
[info] - should decode every RV32I, Zicsr and M-mode SYSTEM instruction
[info] - should flag illegal encodings and assert no other control signal for them
[info] - should classify every one of the 128 major opcodes
[info] AluSpec:
[info] Alu
[info] - should compute every operation on all edge-value pairs (incl. overflow, shift by 31/32, sign)
[info] - should match the reference on random operands
[info] WaveformSpec:
[info] Waveforms
[info] - should show the Stage 8 demo (program.hex + data.hex) through the I-cache and D-cache (waves/stage8_demo)
[info] - should show LUI, AUIPC, JAL and all six branches (waves/upper_jumps)
[info] - should show loads, stores, a loop and a call (waves/load_store_branch)
[info] - should show the four multiplications on corner values, each in one cycle (waves/mul)
[info] - should show DIV/REM waiting on the sequential divider and MUL in one cycle (waves/div_rem)
[info] - should show the divider on its own: request, 32 iterations, response (waves/div_unit)
[info] - should show an ECALL, an illegal instruction (FENCE.I), the handler and MRET (waves/trap)
[info] DivUnitSpec:
[info] DivUnit
[info] - should handle every edge-value pair for all four instructions
[info] - should match the reference on random operands
[info] - should abandon a division on kill and accept the next one
[info] - should run back-to-back requests from a core that holds valid high
[info] FreezeSpec:
[info] Core
[info] - should give the same results with random instruction-port and data-port busy cycles
[info] - should add exactly one cycle per busy cycle when nothing else is in progress, and read mcycle right
[info] - should keep the divider working during a freeze and hold its result until the division leaves EX
[info] CsrFileSpec:
[info] CsrFile
[info] - should implement CSRRW / CSRRS / CSRRC semantics
[info] - should report misa = RV32IM, mhartid and fixed values, and apply WARL masks
[info] - should flag unknown CSRs and writes to read-only CSRs as illegal, and not write them
[info] - should count cycles always and instructions on commit, and let a write replace one half
[info] - should save state on trap entry and restore it on mret
[info] CoreDivSpec:
[info] Core
[info] - should execute DIV, DIVU, REM and REMU including the spec's corner cases
[info] - should use a division result in the next instruction and ignore rd = x0
[info] - should still execute the Step 4 multiplications in one cycle
[info] MultiplierSpec:
[info] Multiplier
[info] - should compute MUL, MULH, MULHSU and MULHU on edge values and random operands
[info] ImmGenSpec:
[info] ImmGen
```

```
[info] - should build the I, S, B, U and J immediates from instructions the assembler encoded
[info] CacheSpec:
[info] ICache
[info] - should return the right word for 3,000 random fetches and miss exactly when a direct-mapped cache misses
[info] DCache
[info] - should load and store correctly (write-through, no-write-allocate) for 3,000 random accesses
[info] CoreTimingSpec:
[info] Core
[info] - should retire one instruction per cycle except DIV/DIVU/REM/REMU, redirect bubbles and load-use stalls
[info] - should discard the wrong-path instruction behind a taken branch, a jump, a trap and MRET
[info] CoreProgramSpec:
[info] Core
[info] - should run arithmetic and logic instructions (OP, OP-IMM, LUI, AUIPC)
[info] - should run loads and stores of every size, signed and unsigned
[info] - should run every branch both ways, loops, JAL and JALR (incl. clearing bit 0)
[info] - should handle back-to-back dependencies, including a load result used immediately
[info] - should run loops written in RV32I only: gcd by subtraction, a sum, and multiplication by shift-and-add
[info] - should execute MUL, MULH, MULHSU and MULHU, including the corner cases, in one cycle
[info] - should compute 12! with MUL in a loop
[info] - should execute DIV, DIVU, REM and REMU (sequential divider), including the spec's corner cases
[info] - should compute gcd(1071, 462) with REM and use MUL and DIV results immediately
[info] - should implement Zicsr and take every exception precisely (illegal, ECALL, EBREAK, misaligned
load/store/fetch)
[info] - should match the reference model on 3000 random instructions
[info] DivRefSpec:
[info] DivRef
[info] - should follow the RISC-V division-by-zero and overflow table
[info] - should round toward zero and give the remainder the dividend's sign
[info] - should agree with the Int form
[info] RegFileSpec:
[info] RegFile
[info] - should hardwire x0 to zero and store x1..x31
[info] - should ignore writes when we is low and return the old value during a write
[info] MemorySpec:
[info] InstructionMemory.readHex
[info] - should accept comments, blank lines and separators
[info] MainMemory
[info] - should answer a line read after exactly `latency` cycles, write under byte masks, and act as a ROM if
asked
[info] RV32ITestWrapper
[info] - should run program.hex through the I-cache and D-cache: results, cache statistics, cycles
[info] Run completed in 10 minutes, 45 seconds.
[info] Total number of tests run: 52
[info] Suites: completed 15, aborted 0
[info] Tests: succeeded 52, failed 0, canceled 0, ignored 0, pending 0
[info] All tests passed.
[success] Total time: 649 s (10:49), completed 11 nov. 2026, 11:46:55

-----------------------------------------------------------------------------------------------
 cat program.hex
# Stage 5 demo program at 0x80000000: dot product with MUL, MULH/MULHU/MULHSU, then DIV/REM/DIVU/REMU incl. corner
cases (data at 0x80010000)
80010537   # 0x00: lui    x10, 0x80010    # x10 = 0x80010000: data base
00052583   # 0x04: lw     x11, 0(x10)     # n = 4
00450613   # 0x08: addi   x12, x10, 4     # pa = &a[0]
01450693   # 0x0c: addi   x13, x10, 20    # pb = &b[0]
00000713   # 0x10: addi   x14, x0, 0      # acc = 0
00062783   # 0x14: lw     x15, 0(x12)     # loop (0x14): x15 = *pa
0006a803   # 0x18: lw     x16, 0(x13)     # x16 = *pb
030788b3   # 0x1c: mul    x17, x15, x16   # 1 cycle (parallel multiplier)
01170733   # 0x20: add    x14, x14, x17   # acc += a[i] * b[i]
00460613   # 0x24: addi   x12, x12, 4
00468693   # 0x28: addi   x13, x13, 4
fff58593   # 0x2c: addi   x11, x11, -1
fe0592e3   # 0x30: bne    x11, x0, loop   # 4 iterations
02e52223   # 0x34: sw     x14, 36(x10)    # dot product (-20) at 0x80010024
02452903   # 0x38: lw     x18, 36(x10)    # read it back
800009b7   # 0x3c: lui    x19, 0x80000    # x19 = 0x80000000 (MIN)
03399a33   # 0x40: mulh   x20, x19, x19   # 0x40000000
0339bab3   # 0x44: mulhu  x21, x19, x19   # 0x40000000
fff00b13   # 0x48: addi   x22, x0, -1
033b2bb3   # 0x4c: mulhsu x23, x22, x19   # 0xffffffff
036b0c33   # 0x50: mul    x24, x22, x22   # 1
00300c93   # 0x54: addi   x25, x0, 3
03974d33   # 0x58: div    x26, x14, x25   # -20 / 3 = -6 (34 cycles, sequential divider)
03976db3   # 0x5c: rem    x27, x14, x25   # -20 % 3 = -2 (sign of the dividend)
03975e33   # 0x60: divu   x28, x14, x25   # 0xffffffec / 3 = 0x5555554e
03977eb3   # 0x64: remu   x29, x14, x25   # 0xffffffec % 3 = 2
02074f33   # 0x68: div    x30, x14, x0    # divide by zero: -1 (2 cycles)
0369efb3   # 0x6c: rem    x31, x19, x22   # MIN % -1 = 0, signed overflow (2 cycles)
0000006f   # 0x70: jal    x0, 0           # halt: jump to itself
-----------------------------------------------------------------------------------------------
```


From: 40 ns
To: 160 ns
Signals
Waves
Time
Clock
clock =1
reset =0
cycleCount[31:0] =8
Cache waits
io_imem_busy =0
io_dmem_busy =0
freeze =0
IF fetch
fetchPc[31:0] =8000000C
io_imem_data[31:0] =addi x13, x10, 20
ID decode (IF/ID register)
ifid_valid =1
ifid_pc[31:0] =80000008
idInst[31:0] =addi x12, x10, 4
loadUse =0
rs1Val[31:0] =0
rs2Val[31:0] =0
EX execute (ID/EX register)
idex_valid =1
idex_pc[31:0] =80000004
idex_inst[31:0] =lw x11, 0(x10)
a_fromMem =1
a_fromWb =0
a[31:0] =-2147418112
b_fromMem =0
b_fromWb =0
b[31:0] =0
exResult[31:0] =-2147418112
divStall =0
divHave =0
trapNow =0
redirect =0
nextPc[31:0] =80000008
MEM memory (EX/MEM register)
exmem_valid =1
exmem_inst[31:0] =lui x10, 0x80010
exmem_trap =0
exmem_result[31:0] =-2147418112
io_dmem_re =0
io_dmem_we =0
io_dmem_addr[31:0] =80010000
io_dmem_rdata[31:0] =00000000
io_dmem_wdata[31:0] =00000000
io_dmem_wstrb[3:0] =0000
WB write-back (MEM/WB register)
memwb_valid =0
memwb_inst[31:0] =nop (bubble)
memwb_trap =0
wbWrites =0
memwb_rd[4:0] =0
memwb_wdata[31:0] =0
Retire / trap (trace, WB)
io_trace_valid =0
ns
50 ns
60 ns
70 ns
80 ns
90 ns
100 ns
110 ns
3 4 5 6 7 8 9 10 11
80000000 80000004 80000008 8000000C 80000010
(empty) lui x10, 0x80010 lw x11, 0(x10) addi x12, x10, 4 addi x13, x10, 20 (empty)
00000000 80000000 80000004 80000008 8000000C
nop (bubble) lui x10, 0x80010 lw x11, 0(x10) addi x12, x10, 4 addi x13, x10, 20
0
0
00000000 80000000 80000004 80000008
(empty) nop (bubble) lui x10, 0x80010 lw x11, 0(x10) addi x12, x10, 4
0 -2147418112
0
0 -2147418112 -2147418108
00000004 80000004 80000008 8000000C
(empty) nop (bubble) lui x10, 0x80010 lw x11, 0(x10)
0 -2147418112
00000000 80010000
00000000
00000000
0000
(empty) nop (bubble) lui x10, 0x80010
0 10
0 -2147418112


From: 40 ns
To: 1600 ns
Signals
Waves
Time
390 ns
400 ns
410 ns
420 ns
430 ns
440 ns
450 ns
Clock
clock =1
reset =0
cycleCount[31:0] =40
Cache waits
io_imem_busy =0
io_dmem_busy =0
freeze =0
IF fetch
fetchPc[31:0] =80000038
io_imem_data[31:0] =lw x18, 36(x10)
ID decode (IF/ID register)
ifid_valid =1
ifid_pc[31:0] =80000034
idInst[31:0] =sw x14, 36(x10)
loadUse =0
rs1Val[31:0] =-2147418112
rs2Val[31:0] =10
EX execute (ID/EX register)
idex_valid =1
idex_pc[31:0] =80000030
idex_inst[31:0] =bne x11, x0, 0x800
a_fromMem =1
a_fromWb =0
a[31:0] =3
b_fromMem =0
b_fromWb =0
b[31:0] =0
exResult[31:0] =3
divStall =0
divHave =0
trapNow =0
redirect =1
nextPc[31:0] =80000014
MEM memory (EX/MEM register)
exmem_valid =1
exmem_inst[31:0] =addi x11, x11, -1
exmem_trap =0
exmem_result[31:0] =3
io_dmem_re =0
io_dmem_we =0
io_dmem_addr[31:0] =00000003
io_dmem_rdata[31:0] =00000004
io_dmem_wdata[31:0] =00000000
io_dmem_wstrb[3:0] =0000
WB write-back (MEM/WB register)
memwb_valid =1
memwb_inst[31:0] =addi x13, x13, 4
memwb_trap =0
wbWrites =1
memwb_rd[4:0] =13
memwb_wdata[31:0] =-2147418088
Retire / trap (trace, WB)
io_trace_valid =1

## Table of Contents


**1. Introduction: AI-Assisted Generation of RISC-V Processor Architectures** 1
- 1.1 Introduction: AI Agents for Processor Design 1
- 1.2 Two Hardware Design Approaches: Chisel and Verilog 1
  - Chisel: Hardware Generation Through Scala 1
  - Verilog: Direct RTL Generation 1
- 1.3 Progressive Construction of the RISC-V Instruction Set 1
  - Stage 1: Foundations of RV32I 1
  - Stage 2: Complete RV32I Implementation 1
  - Stage 3: RV32I with Multiplication 2
  - Stage 4: Complete RV32IM with Sequential Division 2
- 1.4 Evolution Toward Pipelined Processor Architectures 2
  - Stage 5: Two-Stage RV32IM Pipeline 2
  - Stage 6: Five-Stage RV32IM Pipeline 2
  - Stage 7: Five-Stage Pipeline with Cache Memories 2
- 1.5 Fully Automated Generation, Testing, and Waveform Analysis 3
- 1.6 Conclusion 3

**2. The RISC-V Ecosystem and Its Importance for Digital Systems Development** 4
- 2.1 Introduction: An Open Instruction Set Architecture 4
- 2.2 A Modular and Extensible Architecture 4
- 2.3 A Rich Hardware Development Ecosystem 4
- 2.4 Software Tools, Simulation, and Verification 5
- 2.5 Importance for Digital Systems Development 5
  - Technological Independence 5
  - Reduced Development Barriers 5
  - Application-Specific Processor Design 5
  - Hardware/Software Co-Design 5
- 2.6 RISC-V as a Platform for Education, Research, and AI-Assisted Design 5
- 2.7 Conclusion 6

**3. The RISC-V: Digital System and RISC-V Processor Design: Chisel versus Verilog** 7
  - Principles, Design Methodologies, Advantages, and Limitations 7
- 3.1 Introduction 7
- 3.2 Verilog: Direct Description of Digital Hardware 7
  - Advantages of Verilog 7
  - Drawbacks of Verilog 7
- 3.3 Chisel: Hardware Construction Using Scala 7
  - Advantages of Chisel 8
  - Drawbacks of Chisel 8
- 3.4 Comparison for RISC-V Processor Development 8
  - Instruction Decoder and Datapath 8
  - Multiplication and Sequential Division 8
  - Pipeline Development 8
  - Cache Memory Integration 9
- 3.5 Synthesis, Performance, and Verification 9
- 3.6 Comparative Summary 9
- 3.7 AI-Assisted RISC-V Development: A Complementary Approach 9
- 3.8 Conclusion 10
  - Automated RISC-V Architecture Generation 11
  - Automatic Testing and Functional Verification 11
  - Waveform Generation and Visual Analysis with GTKWave 11
  - Conclusion 11

**GLab 0. Installing Chisel 3 on Ubuntu Linux** 13
  - Development Tools, Installation Procedure, Simulation, and Waveform Visualization 13
- 0.1 Introduction 13
- 2. Installing the Basic Development Tools 13
- 3. Installing Scala and sbt 14
- 4. Installing the RTL Simulation and Visualization Tools 14
- 5. Creating a Chisel 3 Project 14
  - Configure the Scala project 15
- 6. Creating a First Chisel Module 15
- 7. Creating Automated Unit Tests 16
- 8. Generating VCD Waveforms 16
- 9. Recommended Workflow for RISC-V Processor Development 17
- 10. Summary 18
  - Official documentation 18

**GLab 1: Building simple RV32I (R-type only) RISC-V architecture** 18
  - Test class: 23
- Understanding the Chisel ALU Test File (ALUSpec.scala) 24
  - Purpose of the test file 24
  - Testbench organization 24
  - Definition of the ALU operations 24
  - First test: Basic functional verification 25
  - Second test: Boundary-value verification 25
  - Third test: Randomized verification 25
  - Waveform generation 25
  - Summary of the verification strategy 26
- Complete project: rv32i-stage1.zip 26

Understanding the Files Generated by a Chisel-Based RISC-V Processor Project....27
Main File Types....27
Chisel Hardware Description Files....27
ALU.scala....27
RegisterFile.scala....28
Decoder.scala....28
ImmediateGenerator.scala....28
RV32IRCore.scala....28
RV32ITestWrapper.scala....28
CoreParams.scala....28
RV32IEncoding.scala....28
Trace.scala....28
Emit.scala....28
RefModel.scala....28
Generated SystemVerilog Files....29
RV32IRCore.sv....29
RV32ITestWrapper.sv....29
Why generate SystemVerilog?....29
Automated Test Files....29
Supporting test utilities....29
Running the tests....30
Simulation Waveform Files....30
Files within each waveform directory....30
Opening the waveforms....31
Project Configuration and Documentation....31
Overall Organization of the Project....31
Conclusion....32

**GLab 2: Building simple RV32I (R and I types) RISC-V architecture....34**

**GLab 3: Building full RV32I RISC-V architecture....42**

**GLab 4: Building full RV32IM (MUL only) architecture....53**

**GLab 5: Building full RV32IM architecture with sequential DIV/REM unit....59**

**GLab 6: Building full RV32IM architecture with 2-stage pipeline....67**

**GLab 7: Building full RV32IM architecture with 5-stage pipeline....75**

**GLab 8: 5-stage pipeline with I-cache and D-cache....83**